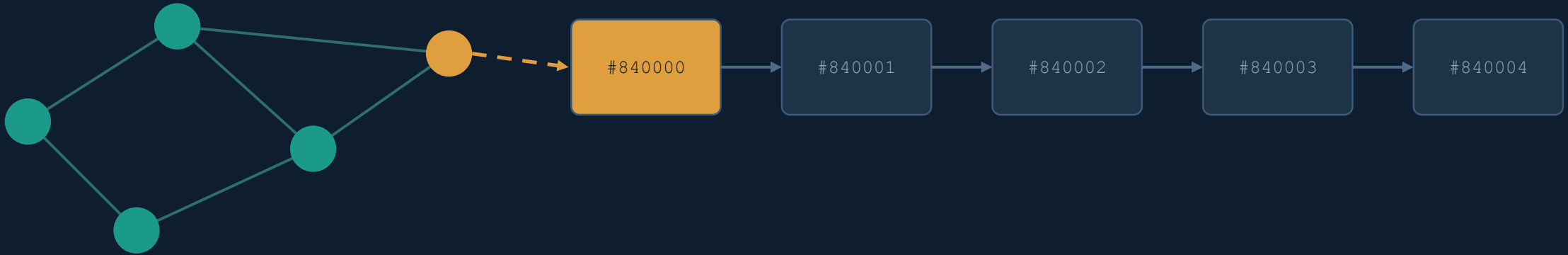


Introduction to Knowledge Graphs & Blockchain Technology

Graphs model what we know. Blockchains prove who agreed, what happened, and when.



Every slide is a self-contained unit: concept → illustration → key insight

1 Layered progression

Each part starts with intuition, formalises it, then stresses it with attacks, trade-offs and edge cases.

2 Visual-first

Diagrams are built from native shapes and text boxes, so you can edit, recolour and re-use every one of them.

3 Real numbers

Hashes, probabilities and supply curves in this deck are computed, not invented — you can reproduce them.

4 Checkpoints

Knowledge checks close each part. Attempt them before reading the answers alongside.

5 Runnable labs

Python and SPARQL labs whose outputs were produced by actually running the code (slides 149, 157–158).

6 Emphasis

Roughly two-thirds of the deck is blockchain. Knowledge graphs provide the modelling lens and return in Part III.

KEY INSIGHT

Read the badge in the top-left corner: it names the part you are in and the slide number — like a block height.

Learning outcomes

By the end you should be able to...

MODEL

Represent a domain as a knowledge graph using RDF, RDFS/OWL and property graphs, and query it with SPARQL and Cypher.

EXPLAIN

Explain hashing, Merkle trees, digital signatures and why they make a ledger tamper-evident.

ANALYSE

Analyse consensus protocols (PoW, PoS, BFT) in terms of safety, liveness, finality and attack cost.

BUILD

Read a block header, trace a UTXO transaction, and write and audit a simple smart contract.

EVALUATE

Evaluate scaling approaches (rollups, channels, sharding) and decide when a blockchain is the wrong tool.

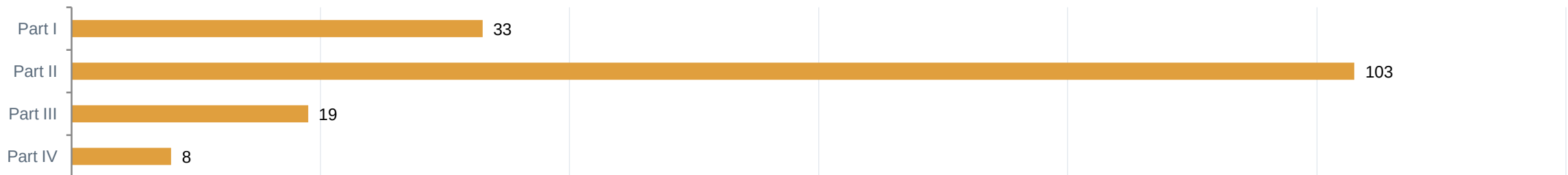
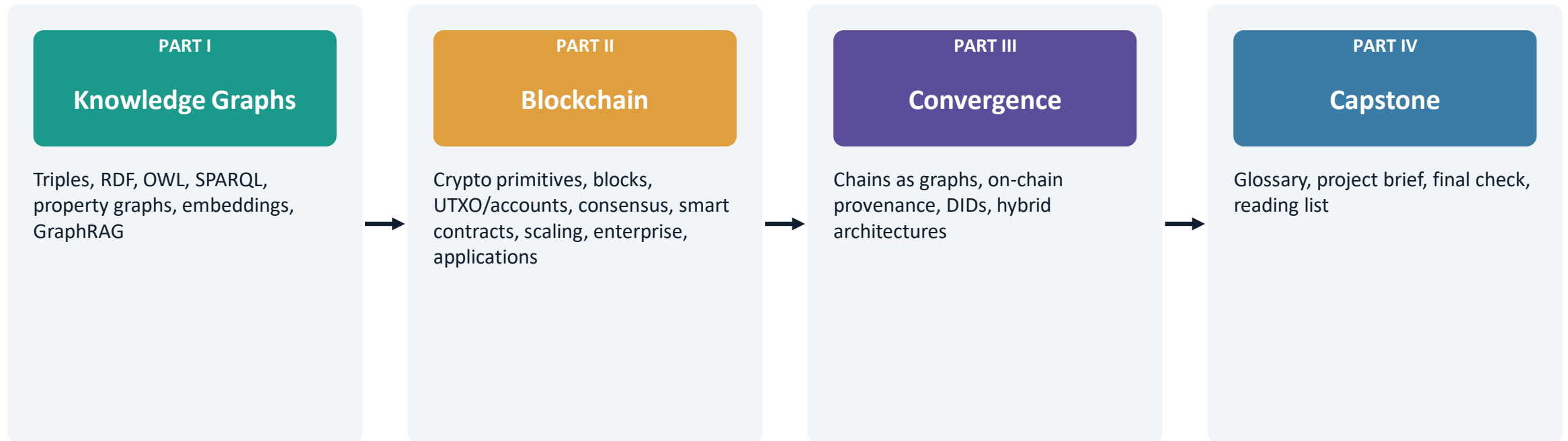
INTEGRATE

Design hybrid systems that anchor knowledge-graph provenance on-chain and analyse chains as graphs.

Verbs follow Bloom's taxonomy — from recall to design.

Course map

Four parts, one narrative: represent knowledge → secure agreement on it → combine both

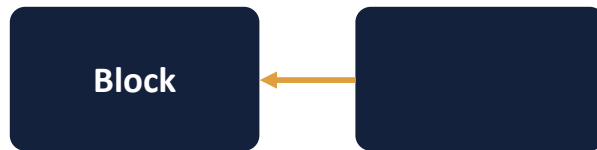
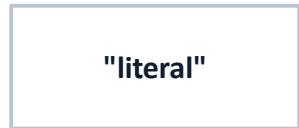


Notation & conventions used throughout

A small, consistent visual language



Ellipse = resource/entity · labelled arrow = relationship · plain rectangle = literal value



Navy rounded box = block or process step · gold arrow = hash pointer / flow of control



`monospace` = code, hex, identifiers, byte fields. **Bold** = defined term. *Gold italic* = emphasis.

Prerequisites self-check

Rate yourself 1–5; anything below 3 has a refresher slide referenced

Topic	What you need	Refresher
Discrete maths	Sets, relations, functions, directed graphs	Slide 10
Probability	Independent events, Poisson intuition	Slide 71
Programming	Read Python / JavaScript; JSON	Labs: slides 149, 157–158
Networking	Client–server vs peer-to-peer; latency	Slides 91–94
Algebra	Modular arithmetic, exponents, logs	Slides 54–57
Databases	Tables, keys, indexes, transactions (ACID)	Slides 11, 29

KEY INSIGHT

No cryptography background is assumed: every primitive is built up from its security properties, not its mathematics.

PART I

Knowledge Graph Foundations

How machines represent entities, relationships and meaning

▸ Graph theory ▸ RDF & triples ▸ RDFS / OWL ▸ SPARQL & Cypher ▸ Construction ▸ Embeddings & GNNs ▸ GraphRAG



From data to knowledge: the DIKW pyramid

Knowledge graphs sit where data acquires explicit, machine-readable meaning

Wisdom

----- Judgement: which action to take

Knowledge

----- Connected, contextualised facts + rules → inference

Information

----- Data with structure & meaning (schema)

Data

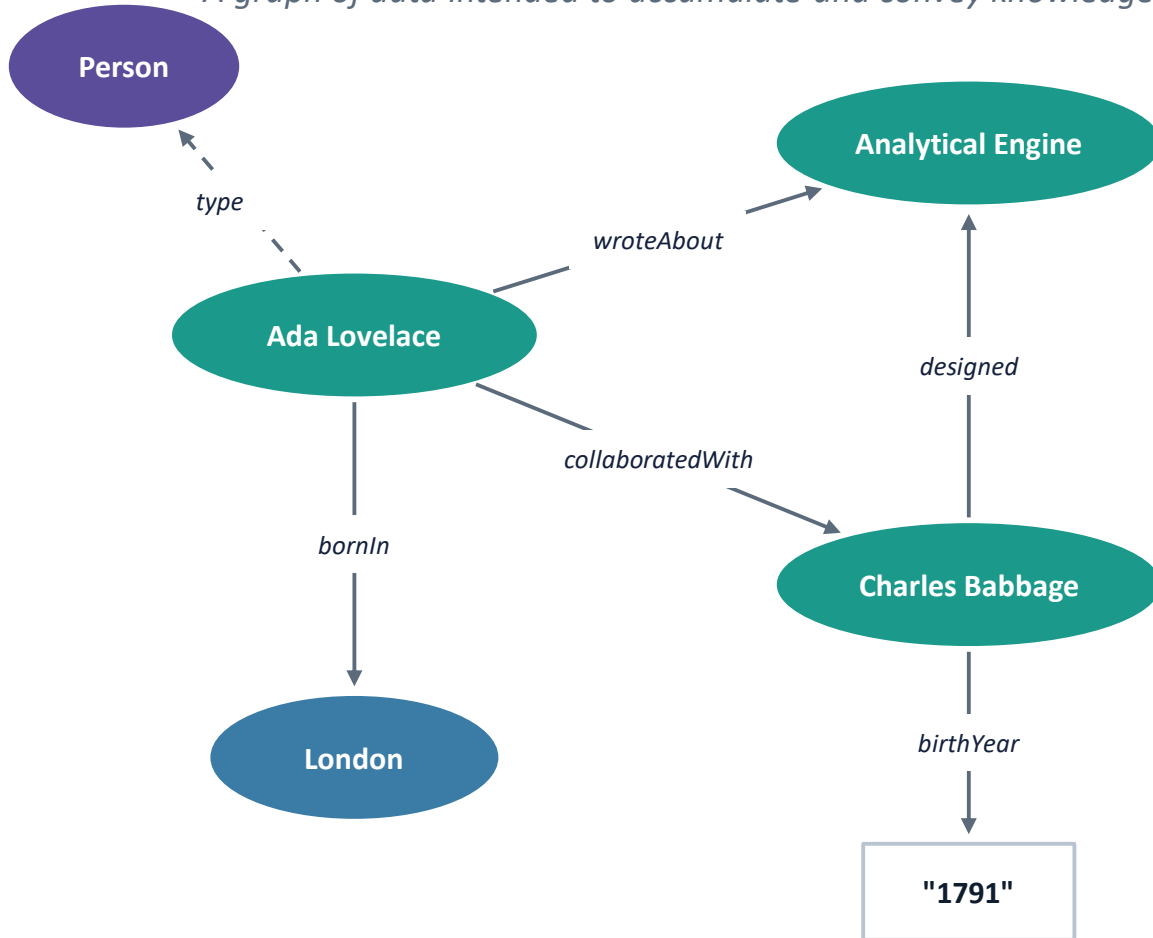
--- Raw symbols: 51.48, 'Salford', 1894

KEY INSIGHT

A knowledge graph lives on the 'Knowledge' tier: facts are linked, typed and governed by a schema that permits reasoning.

What is a knowledge graph?

"A graph of data intended to accumulate and convey knowledge of the real world" — Hogan et al., 2021



- **Nodes** = entities (things, not strings)
- **Edges** = typed, directed relationships
- **Types** = classes from an ontology (dashed)
- **Literals** = values: dates, numbers, text
- **Identity** = every entity has a global ID (IRI / QID)

KEY INSIGHT

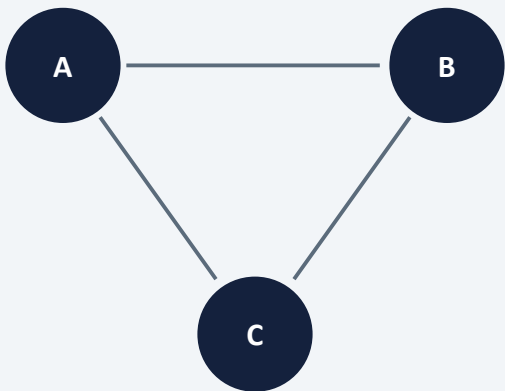
Graph + schema + identity = knowledge graph.
Remove any one and you have merely a graph, a schema, or a list of names.

Graph theory primer

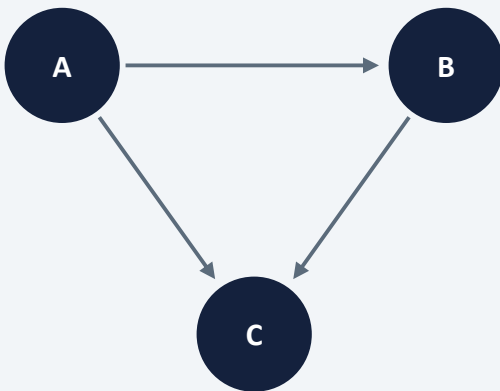
The mathematical object underneath every knowledge graph

A graph $G = (V, E)$: a set of vertices V and edges $E \subseteq V \times V$

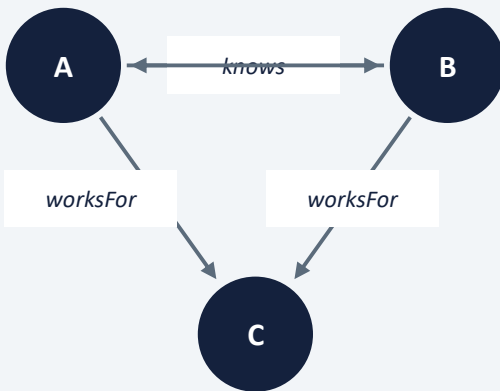
Undirected



Directed



Labelled multigraph



Term	Meaning	KG relevance
Degree	Number of incident edges (in/out)	Hub entities, popularity
Path	Sequence of connected edges	Multi-hop reasoning, property paths
Subgraph	Subset of V and E	Named graphs, context, provenance

Two dominant data models

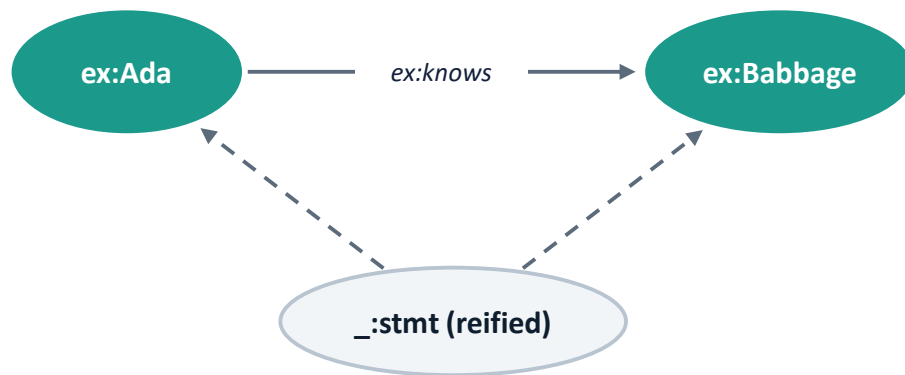
RDF graphs (W3C standard) vs labelled property graphs (industry practice)

RDF graph

- Everything is a **triple** (s, p, o)
- Global identifiers (IRIs) → web-scale merging
- Formal semantics: RDFS, OWL, SHACL
- Query: **SPARQL**; stores: GraphDB, Jena, Virtuoso

Labelled property graph (LPG)

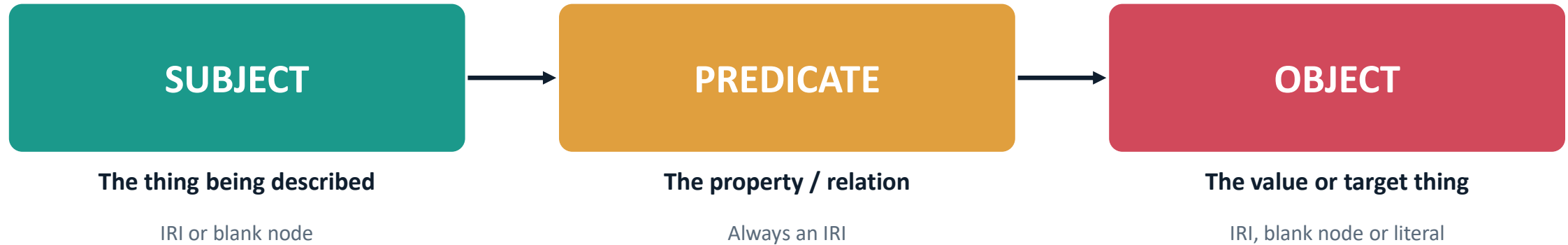
- Nodes & edges both carry **key-value properties**
- Local IDs; schema optional
- Pragmatic traversal performance
- Query: **Cypher / GQL**; stores: Neo4j, Memgraph



Edge metadata in RDF needs reification or RDF-star (`<< s p o >> :since 1833`); LPGs store it on the edge natively.

The atomic unit: the triple

Subject — predicate — object: a sentence that a machine can join with any other sentence



N-Triples — one triple per line

```
<http://ex.org/Ada> <http://ex.org/bornIn> <http://ex.org/London> .  
<http://ex.org/Ada> <http://xmlns.com/foaf/0.1/name> "Ada Lovelace"@en .  
<http://ex.org/Ada> <http://ex.org/birthDate> "1815-12-10"^^xsd:date .
```

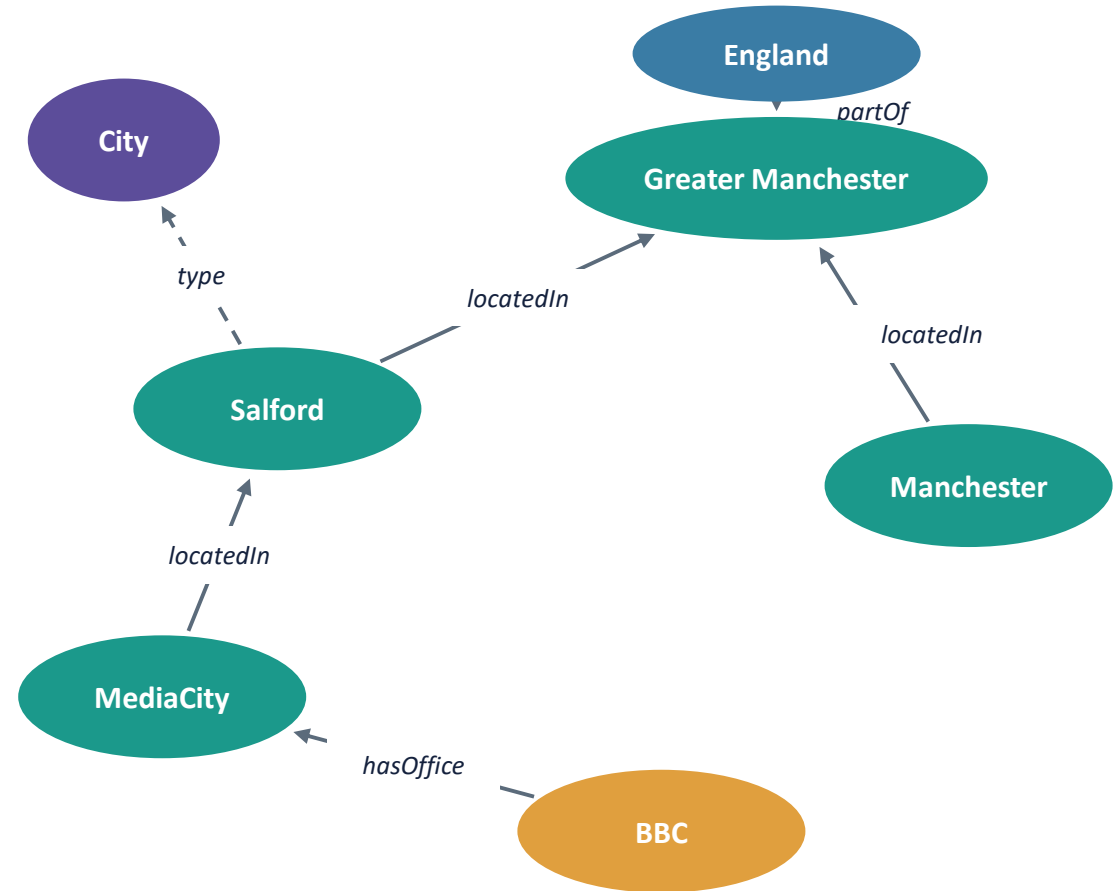
KEY INSIGHT

Because every triple is self-contained, two datasets merge by simple set union — no table redesign, no joins to configure.

Growing a graph, one triple at a time

Six triples → a connected, queryable structure

- 1 `ex:Salford ex:locatedIn ex:GreaterManchester`
- 2 `ex:Manchester ex:locatedIn ex:GreaterManchester`
- 3 `ex:GreaterManchester ex:partOf ex:England`
- 4 `ex:MediaCity ex:locatedIn ex:Salford`
- 5 `ex:BBC ex:hasOffice ex:MediaCity`
- 6 `ex:Salford rdf:type ex:City`



KEY INSIGHT

Query 'organisations with an office in England' now succeeds via a 4-hop path even though no single triple states it.

Identity: IRIs, literals and blank nodes

Getting identity right is 80% of knowledge-graph engineering

I IRI

- Globally unique name:
`https://www.wikidata.org/entity/Q7259`
- Dereferenceable → Linked Data
- Prefixes shorten them: `wd:Q7259`
- Two IRIs may denote the same thing →
`owl:sameAs`

L Literal

- Lexical form + datatype or language tag
- `"42"^^xsd:integer`, `"Londres"@fr`
- Can only be an **object**
- Typed literals enable range filters & arithmetic

B Blank node

- Anonymous, locally scoped: `_:b0`
- Useful for n-ary structures (an address)
- Hard to merge & diff — **skolemise** when publishing
- Avoid for anything you will reference later

Linked Data principles (Berners-Lee, 2006): 1) use URIs as names; 2) use HTTP URIs; 3) return useful RDF when dereferenced; 4) link to other URIs.

Serialising RDF: one graph, many syntaxes

Turtle for humans · JSON-LD for web developers · N-Triples for streaming

ada.ttl (Turtle)

```
@prefix ex:    <http://ex.org/> .
@prefix foaf:  <http://xmlns.com/foaf/0.1/> .
@prefix xsd:   <http://www.w3.org/2001/XMLSchema#> .

ex:Ada a foaf:Person ;
    foaf:name "Ada Lovelace"@en ;
    ex:bornIn ex:London ;
    ex:birthDate "1815-12-10"^^xsd:date ;
    foaf:knows ex:Babbage , ex:DeMorgan .
```

; repeats the subject, , repeats subject + predicate

ada.jsonld (JSON-LD)

```
{
  "@context": {"foaf": "http://xmlns.com/foaf/0.1/",
               "ex": "http://ex.org/"},
  "@id": "ex:Ada",
  "@type": "foaf:Person",
  "foaf:name": {"@value": "Ada Lovelace",
                "@language": "en"},
  "ex:bornIn": {"@id": "ex:London"}
}
```

@context maps short JSON keys to full IRIs

Other formats: RDF/XML (legacy), N-Quads & TriG (add a 4th element: the named graph).

Schema vs data: TBox and ABox

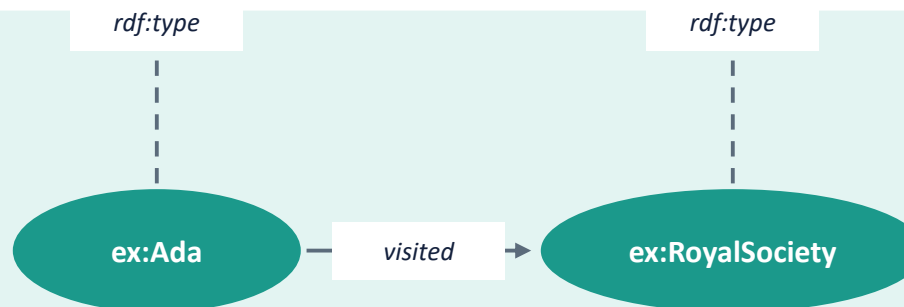
Terminological knowledge (concepts) vs assertional knowledge (facts about individuals)

TBox — ontology / schema



Reasoners use the TBox to derive new ABox facts.

ABox — instance data



TBox answers: what kinds of things exist and how may they relate?
ABox answers: which particular things exist and how do they relate?

RDFS: the lightweight schema language

Six constructs that already allow useful inference

Construct	Meaning	Inference rule (simplified)
<code>rdfs:Class</code>	Declares a class	—
<code>rdfs:subClassOf</code>	Every A is a B	$x \text{ type } A \wedge A \sqsubseteq B \Rightarrow x \text{ type } B$
<code>rdfs:subPropertyOf</code>	p implies q	$x \text{ p } y \wedge p \sqsubseteq q \Rightarrow x \text{ q } y$
<code>rdfs:domain</code>	Subjects of p are Cs	$x \text{ p } y \wedge \text{domain}(p)=C \Rightarrow x \text{ type } C$
<code>rdfs:range</code>	Objects of p are Cs	$x \text{ p } y \wedge \text{range}(p)=C \Rightarrow y \text{ type } C$
<code>rdfs:label</code> / <code>comment</code>	Human-readable annotation	—

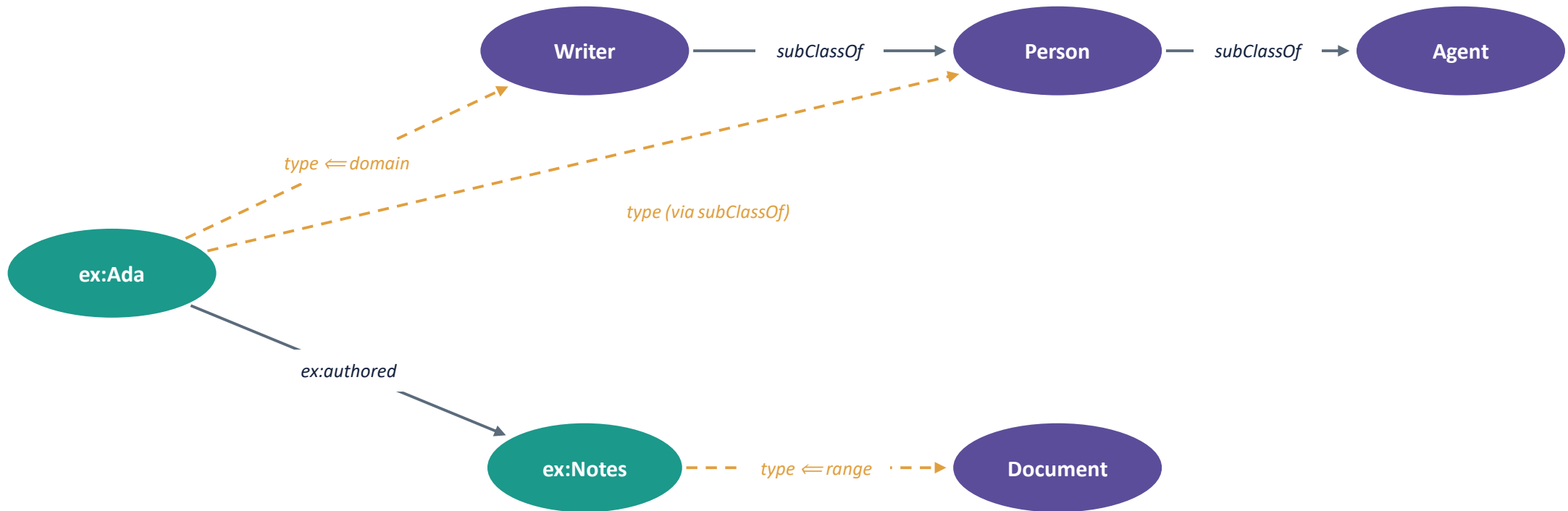
COMMON TRAP

Domain and range are NOT constraints in RDFS — they cause inference. Asserting `ex:London ex:bornIn ex:Ada` makes London a Person! Validation needs SHACL.

Inference in action: entailment

Explicit triples (solid) + schema rules $\Rightarrow$ implicit triples (dashed, gold)

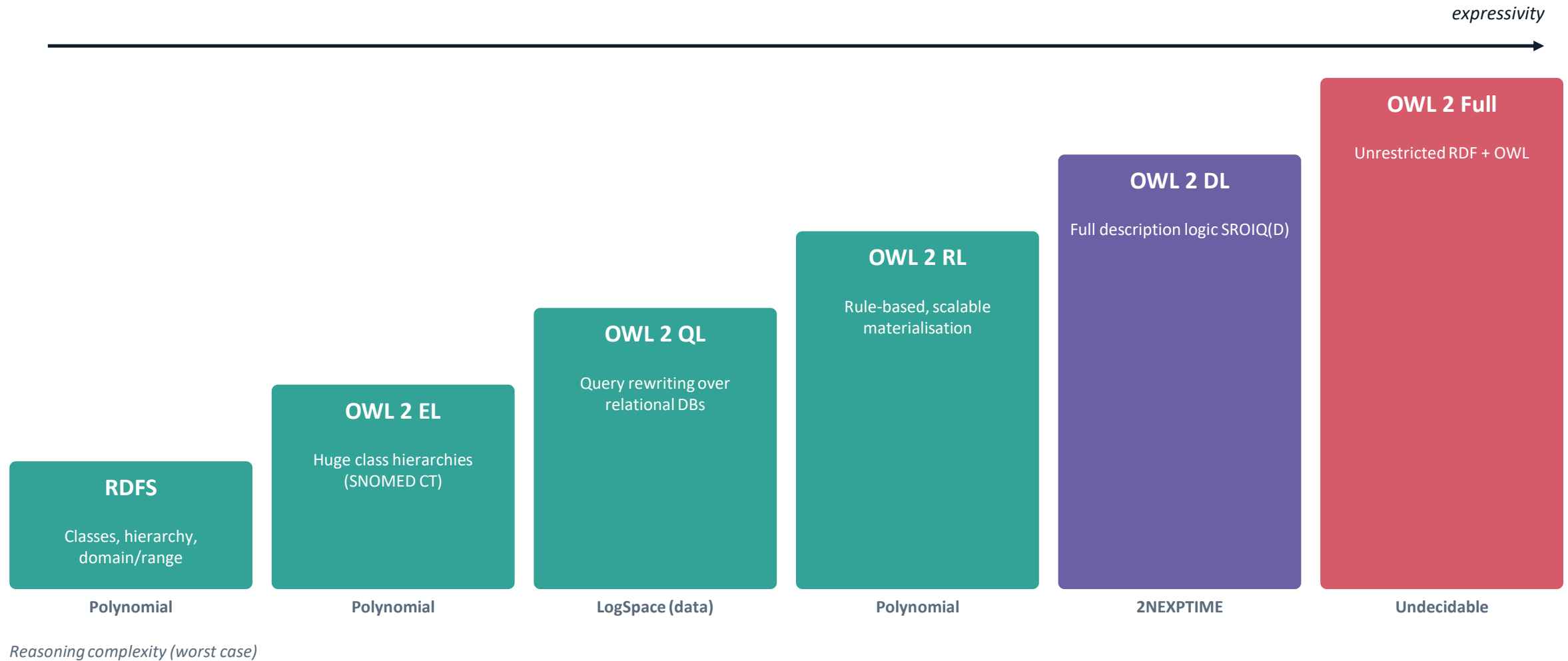
Schema: `ex:authored rdfs:domain ex:Writer ; rdfs:range ex:Document .`



Also entailed: Ada type Agent. Forward-chaining materialises these; backward-chaining derives them at query time.

OWL: the expressivity ladder

More expressive power $\Rightarrow$ more inference $\Rightarrow$ higher reasoning cost



Core OWL constructs

What each construct lets a reasoner conclude

Construct	Example	What it enables
<code>owl:equivalentClass</code>	Human $\equiv$ Person	Merge vocabularies
<code>owl:disjointWith</code>	Person $\sqcap$ Organisation = $\emptyset$	Detect contradictions
<code>owl:TransitiveProperty</code>	partOf	Salford partOf England (via GM)
<code>owl:SymmetricProperty</code>	marriedTo	$a \rightarrow b \Rightarrow b \rightarrow a$
<code>owl:inverseOf</code>	parentOf $\leftrightarrow$ childOf	Navigate both directions
<code>owl:FunctionalProperty</code>	hasBirthDate	Two values $\Rightarrow$ same individual
<code>owl:someValuesFrom</code>	Parent $\equiv \exists \text{hasChild. Person}$	Classify individuals automatically
<code>owl:sameAs</code>	wd:Q7259 = dbr:Ada_Lovelace	Cross-dataset identity

KEY INSIGHT

`owl:sameAs` is the most abused OWL property: it asserts that ALL statements about one IRI hold for the other.

Open-world vs closed-world assumption

The single most important semantic difference between KGs and databases

c

Closed World (SQL, most apps)

- What is not stated is **false**
- No row for Ada's phone $\Rightarrow$ Ada has no phone
- Unique Name Assumption: different IDs $\Rightarrow$ different things
- Natural for complete, controlled data

o

Open World (RDF / OWL)

- What is not stated is **unknown**
- No phone triple $\Rightarrow$ we don't know
- No Unique Name Assumption (unless declared)
- Natural for the web: data is always incomplete

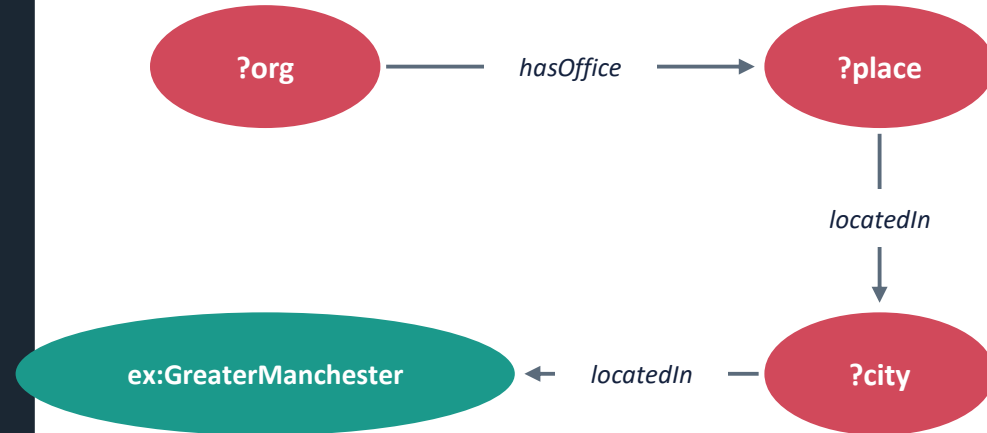
Scenario	Closed world answers	Open world answers
Is Ada a mother? (no data)	No	Unknown
Functional hasMother has two values	Constraint violation	The two mothers are the same individual

SPARQL fundamentals: querying by graph pattern

A query is a graph with holes (variables); the engine finds every matching subgraph

basic graph pattern (BGP)

```
PREFIX ex: <http://ex.org/>
SELECT ?org ?place
WHERE {
  ?org ex:hasOffice ?place .
  ?place ex:locatedIn ?city .
  ?city ex:locatedIn ex:GreaterManchester .
}
ORDER BY ?org
LIMIT 10
```



Solution: ?org = ex:BBC, ?place = ex:MediaCity (?city = ex:Salford)

KEY INSIGHT

Each triple pattern is matched against an index; shared variables become joins. SPARQL is essentially relational algebra over a three-column table.

SPARQL beyond the basics

Operators you need for real-world, incomplete data

OPTIONAL

```
?p ex:name ?n . OPTIONAL { ?p ex:phone ?t }
```

Left outer join: keep ?p even with no phone

FILTER

```
FILTER(?year > 1800 && lang(?n) = "en")
```

Restrict solutions by expression

Property paths

```
?x ex:locatedIn+ ex:England
```

Transitive closure: 1..n hops

Aggregates

```
SELECT ?c (COUNT(?p) AS ?n) ... GROUP BY ?c
```

Summaries per group; HAVING filters groups

UNION / MINUS

```
{ A } UNION { B }      { A } MINUS { B }
```

Alternatives and set difference

CONSTRUCT

```
CONSTRUCT { ?b ex:hub ?a } WHERE { ... }
```

Output a new graph, not a table

SERVICE

```
SERVICE <https://query.wikidata.org/sparql> { ... }
```

Federated query across endpoints

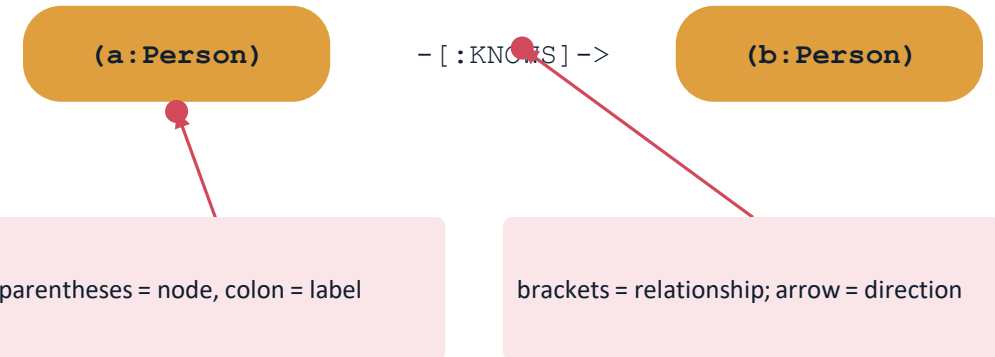
Property-graph querying with Cypher / GQL

ASCII-art patterns: (node)-[:REL]->(node). GQL became ISO/IEC 39075 in 2024

cypher

```
// Who collaborated with someone Ada knows?
MATCH (a:Person {name:'Ada'})-[:KNOWS]->(b:Person)
  -[c:COLLABORATED_WITH]->(x:Person)
WHERE c.since < 1850
RETURN x.name, count(*) AS paths
ORDER BY paths DESC;

// Variable-length path, 1 to 3 hops
MATCH p = (a)-[:KNOWS*1..3]->(z)
RETURN length(p), z.name;
```



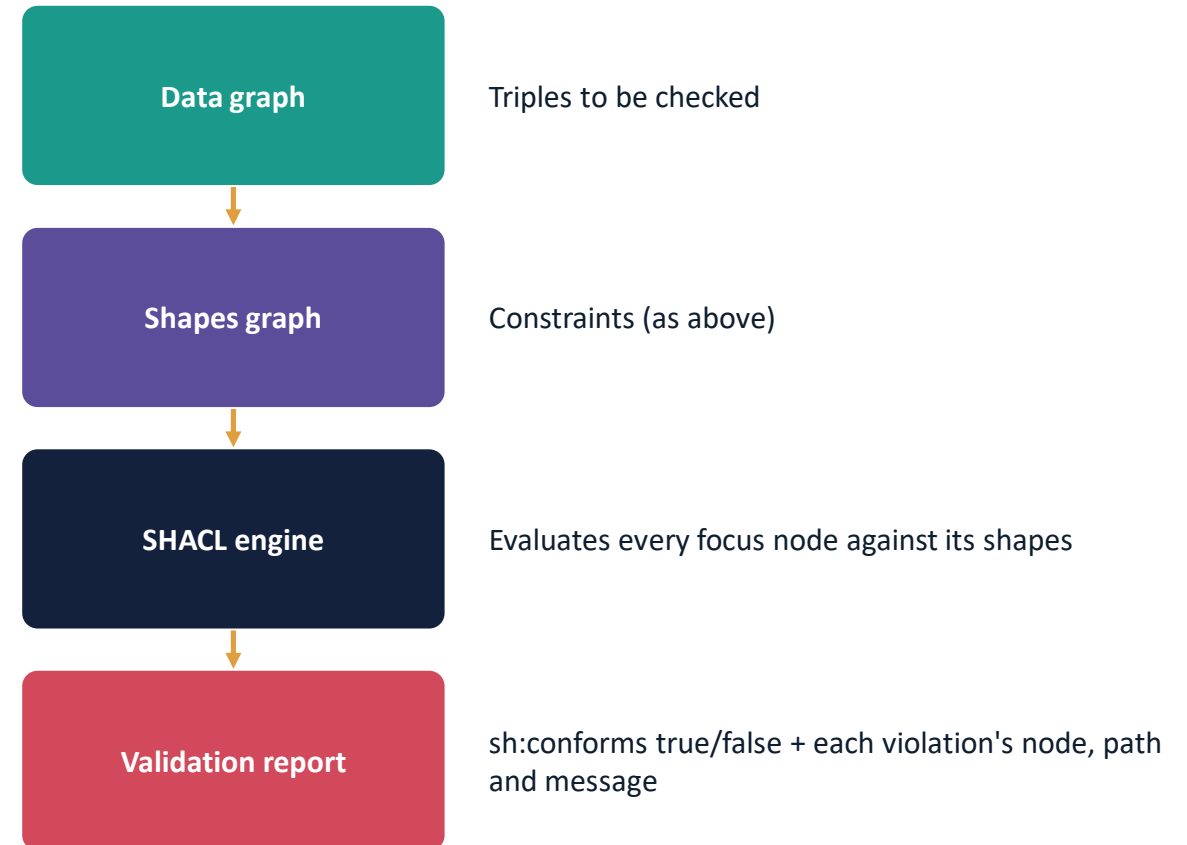
	SPARQL	Cypher
Pattern	triple patterns	ASCII-art paths
Edge props	RDF-star	native
Reasoning	RDFS/OWL	procedural

SHACL: validating the graph

Closed-world constraints layered on top of open-world data

shapes.ttl

```
ex:PersonShape a sh:NodeShape ;  
  sh:targetClass ex:Person ;  
  sh:property [  
    sh:path ex:birthDate ;  
    sh:datatype xsd:date ;  
    sh:maxCount 1 ;  
  ] ;  
  sh:property [  
    sh:path ex:bornIn ;  
    sh:class ex:Place ;  
    sh:minCount 1 ;  
  ] .
```

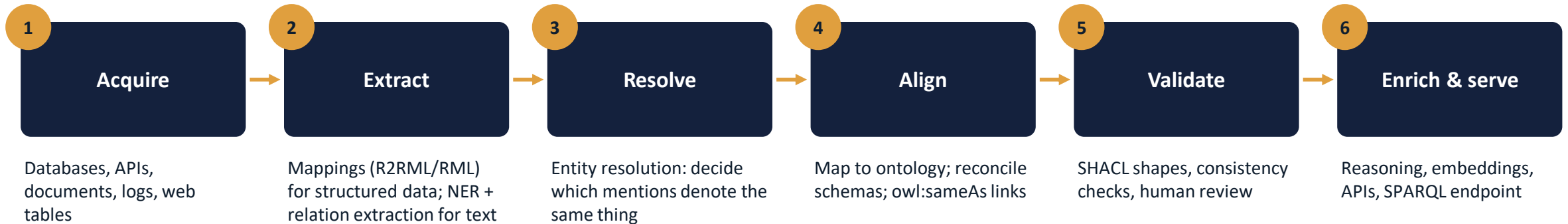


KEY INSIGHT

Use OWL to infer, SHACL to enforce. Mixing them up is the classic source of 'why didn't my constraint fire?'

Knowledge-graph construction pipeline

From heterogeneous sources to a curated, versioned graph



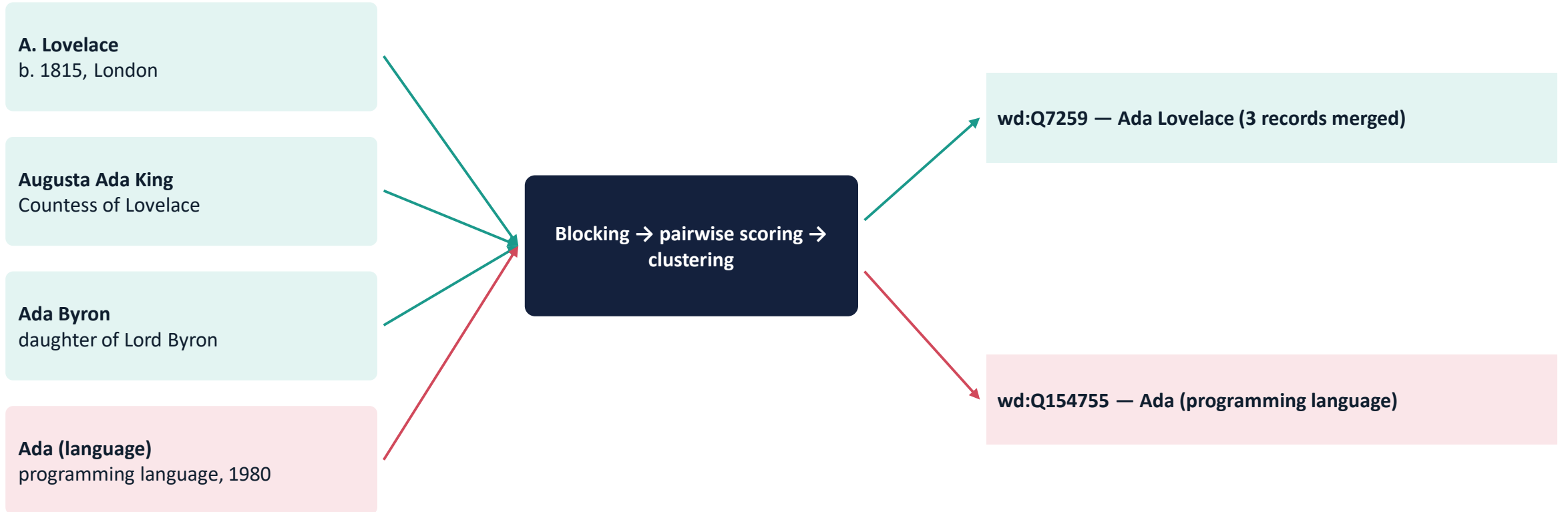
Feedback loop: quality metrics and user corrections flow back to extraction and resolution models

KEY INSIGHT

Provenance (who asserted what, from which source, when) must be captured at extraction time — retrofitting it is nearly impossible. Part III anchors it on-chain.

Entity resolution

Deciding when two records denote the same real-world entity



Signals: string similarity (Jaro–Winkler), attribute agreement, graph neighbourhood overlap, embeddings, LLM adjudication. Measure with precision / recall on labelled pairs.

Extracting triples from text — classic NLP and LLMs

Named-entity recognition → linking → relation extraction

“In 1843, **Ada Lovelace** published notes on **Babbage's Analytical Engine** in **London**.”

NER

Spans: PERSON(Ada Lovelace),
PERSON(Babbage), MACHINE(Analytical
Engine), GPE(London), DATE(1843)

Entity linking

Map spans to KG IDs: Q7259, Q46633,
Q468...; resolve ambiguity with context

Relation extraction

(Q7259, wroteAbout, AnalyticalEngine),
(AnalyticalEngine, designedBy, Q46633)

Confidence & provenance

Score each triple; attach source sentence,
model version, timestamp

CAUTION

LLMs make extraction cheap but not reliable: they hallucinate relations. Constrain outputs to the ontology schema and validate every triple with SHACL.

Storing graphs: triple stores and indexes

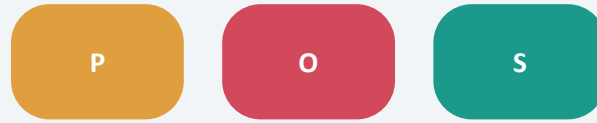
How a triple store answers patterns with any combination of bound terms

Dictionary encoding: every IRI/literal → integer ID. Then store the same triples sorted several ways:



(ex:Ada, ?, ?)

Everything about a subject



(?, ex:bornIn, ex:London)

Who was born in London?



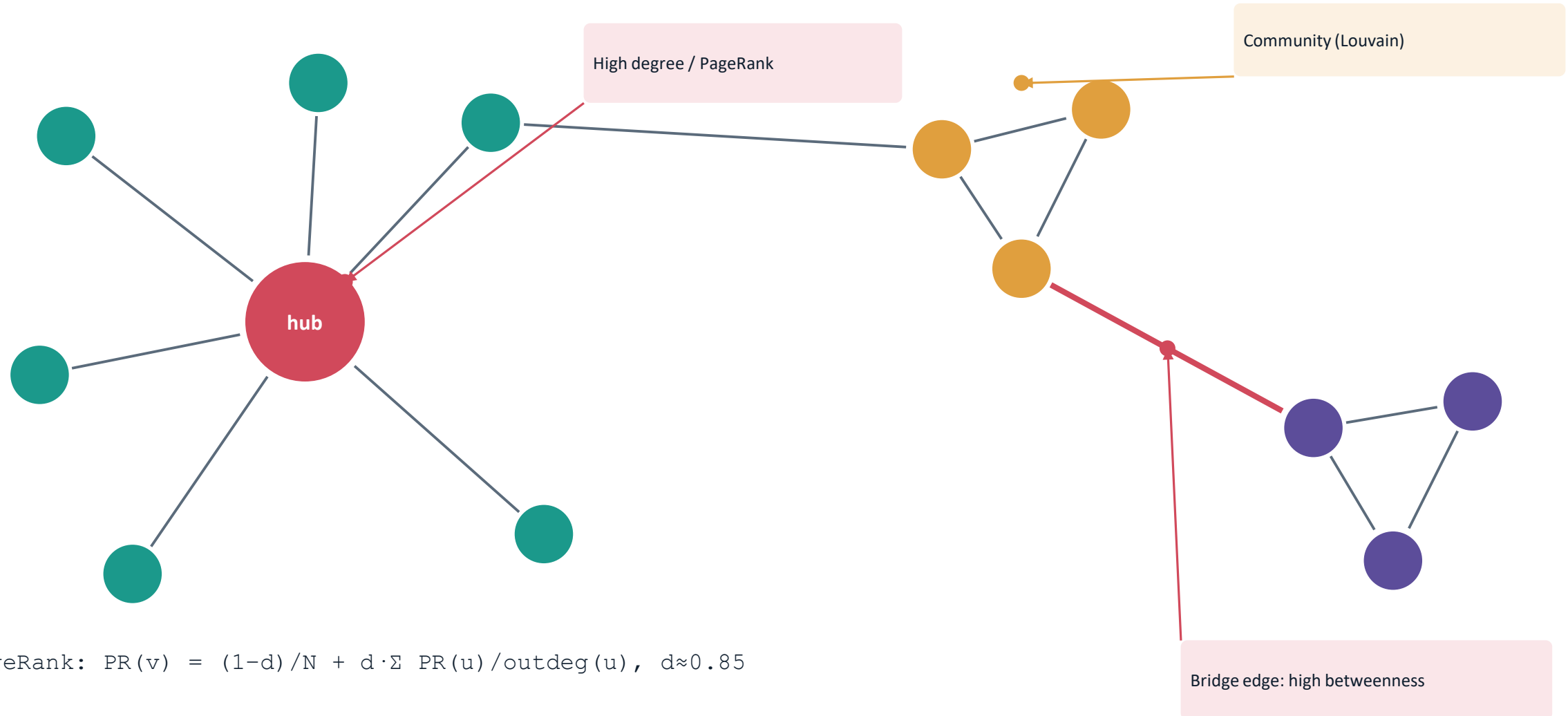
(?, ?, ex:London)

Everything pointing at London

System family	Examples	Strength
RDF triple stores	GraphDB, Apache Jena TDB, Virtuoso, Blazegraph	Standards, reasoning, federation
Native property graphs	Neo4j, Memgraph, TigerGraph	Index-free adjacency; deep traversals
Multi-model / cloud	Amazon Neptune, ArangoDB, Stardog	Operational convenience

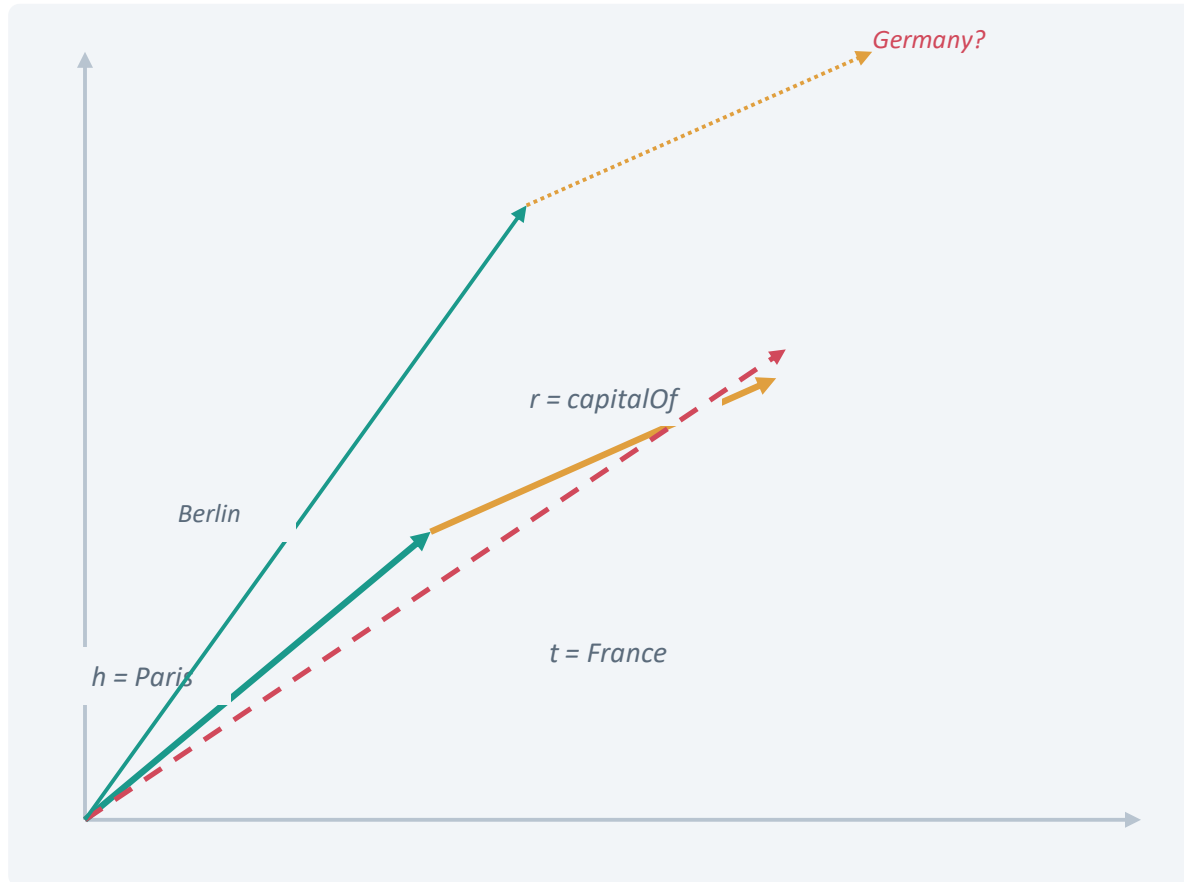
Graph algorithms: extracting structure

Centrality, community and path analytics turn a KG into insight



Knowledge-graph embeddings: TransE

Learn vectors so that $head + relation \approx tail$ (Bordes et al., 2013)

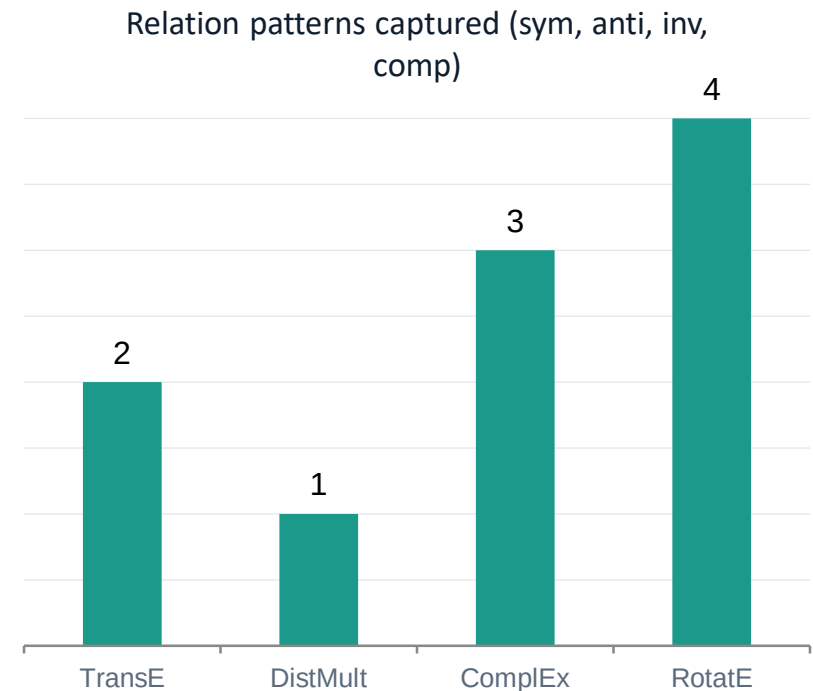


- **Score** $f(h,r,t) = -\|h + r - t\|$
- **Loss**: margin ranking — push true triples closer than corrupted ones by margin γ
- **Link prediction**: rank all t for $(Berlin, capitalOf, ?)$
- **Limitation**: cannot model 1-to-N or symmetric relations well $\rightarrow$ TransH, RotatE

Families of KG embedding models

Translational, bilinear, rotational and neural approaches

Model	Scoring idea	Handles	Year
TransE	$\ h + r - t\ $	Hierarchies, compositions	2013
DistMult	$\langle h, r, t \rangle$ (tri-linear)	Symmetric relations only	2015
Complex	$\text{Re}(\langle h, r, \bar{t} \rangle)$, complex vectors	Symmetric + antisymmetric	2016
RotatE	$h \circ r \approx t, r_i = 1$	Symmetry, inversion, composition	2019
R-GCN	Relational graph convolution	Uses neighbourhood structure	2018
LLM-based	Text + structure encoders	Unseen entities (inductive)	2020s

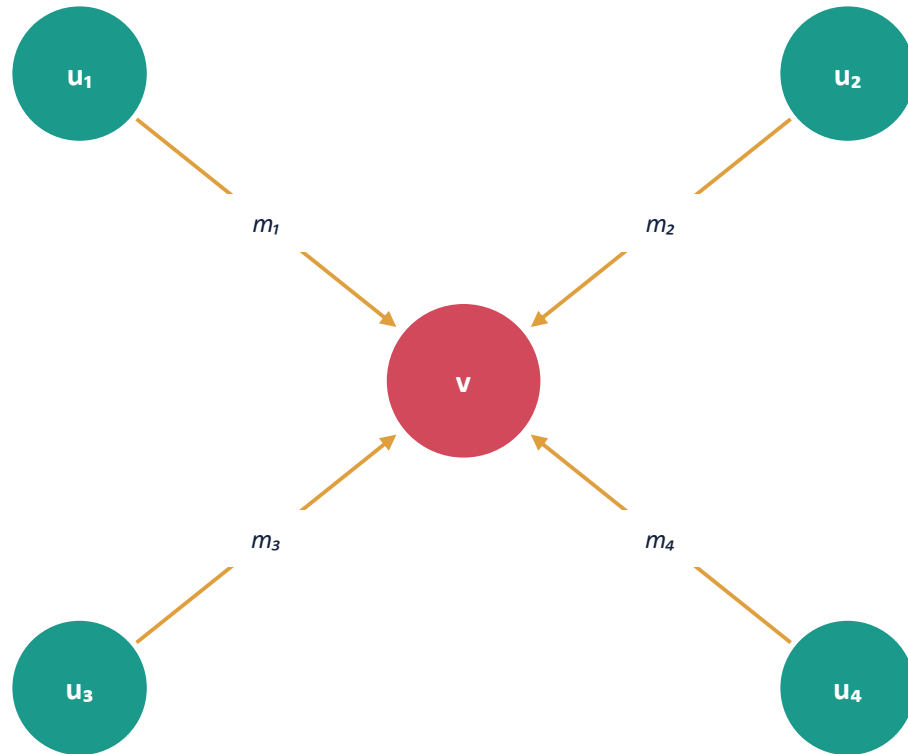


KEY INSIGHT

Choose by relation mix: RotatE captures symmetry, antisymmetry, inversion and composition, per Sun et al. (2019).

Graph neural networks: message passing

Each node's representation is updated from its neighbours' messages, layer by layer



$$h_v^{(k+1)} = \sigma \left(W_{\text{self}} \cdot h_v^{(k)} + \sum_r \frac{1}{c_{\{v, r\}}} \cdot W_r \cdot \sum_{u \in N_r(v)} h_u^{(k)} \right)$$

R-GCN update (Schlichtkrull et al., 2018): a separate weight matrix W_r per relation type r

Aggregate

Sum / mean / attention over neighbour messages

Update

Combine with self-state; apply non-linearity

Stack k layers

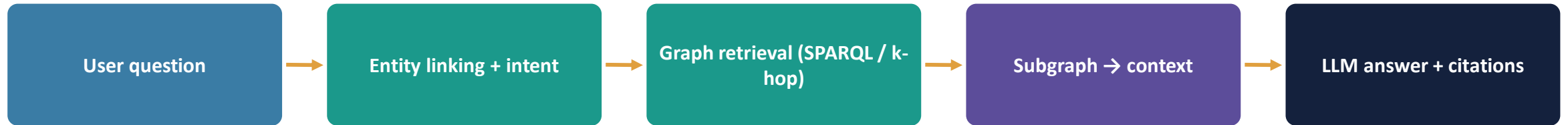
k-hop receptive field; beware over-smoothing

Tasks

Node classification, link prediction, fraud detection

Knowledge graphs meet LLMs: GraphRAG

Ground generative answers in explicit, traceable facts



Why graphs help

- Multi-hop facts retrieved precisely
- Every sentence traceable to triples
- Access control at triple level

Why LLMs help graphs

- Text-to-SPARQL/Cypher
- Extraction and schema drafting
- Natural-language summaries of subgraphs

Open problems

- Retrieval recall on large graphs
- Stale or conflicting facts
- Evaluating faithfulness

KEY INSIGHT

Faithfulness improves only if the provenance of retrieved triples is trustworthy — a direct bridge to blockchain-anchored provenance in Part III.

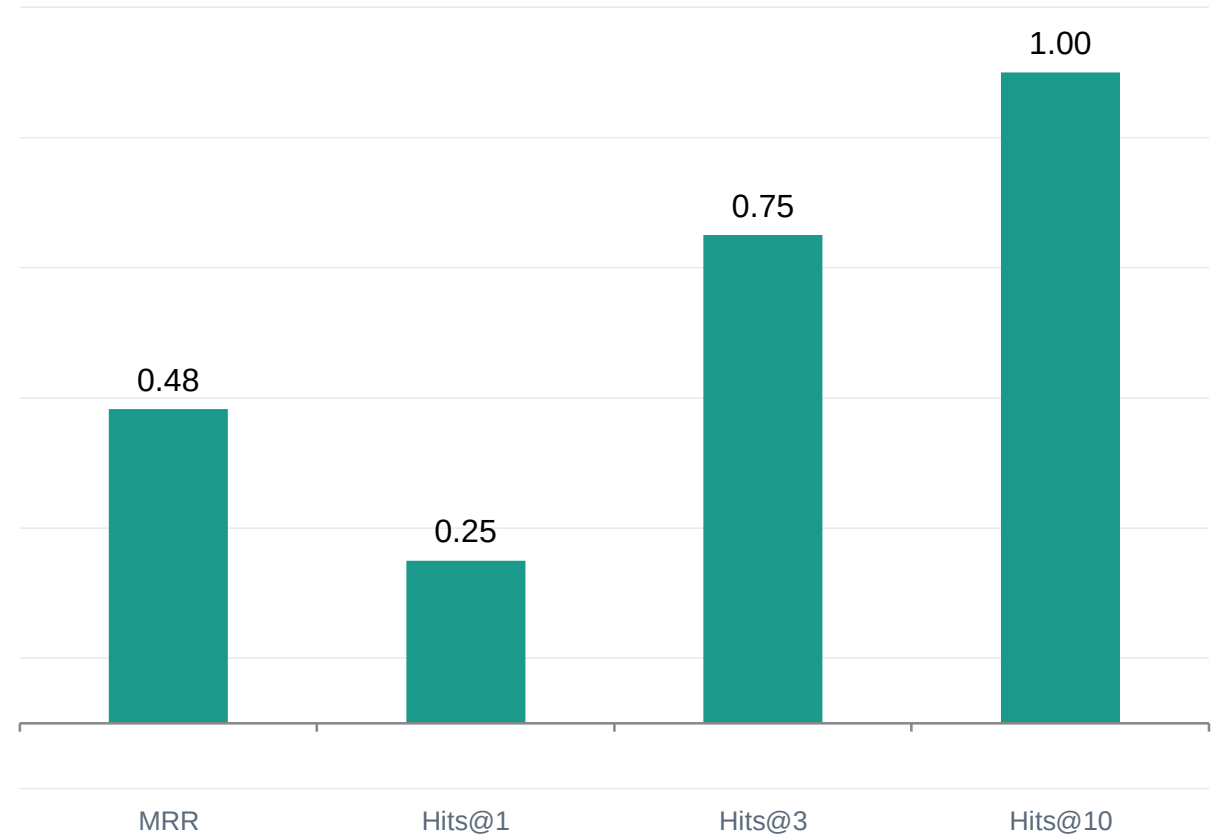
Evaluating link prediction

Rank the true answer among all candidates; report MRR and Hits@k (filtered setting)

Test query	Rank of true tail
(Paris, capitalOf, ?)	1
(Ada, bornIn, ?)	3
(Babbage, designed, ?)	2
(Salford, partOf, ?)	10

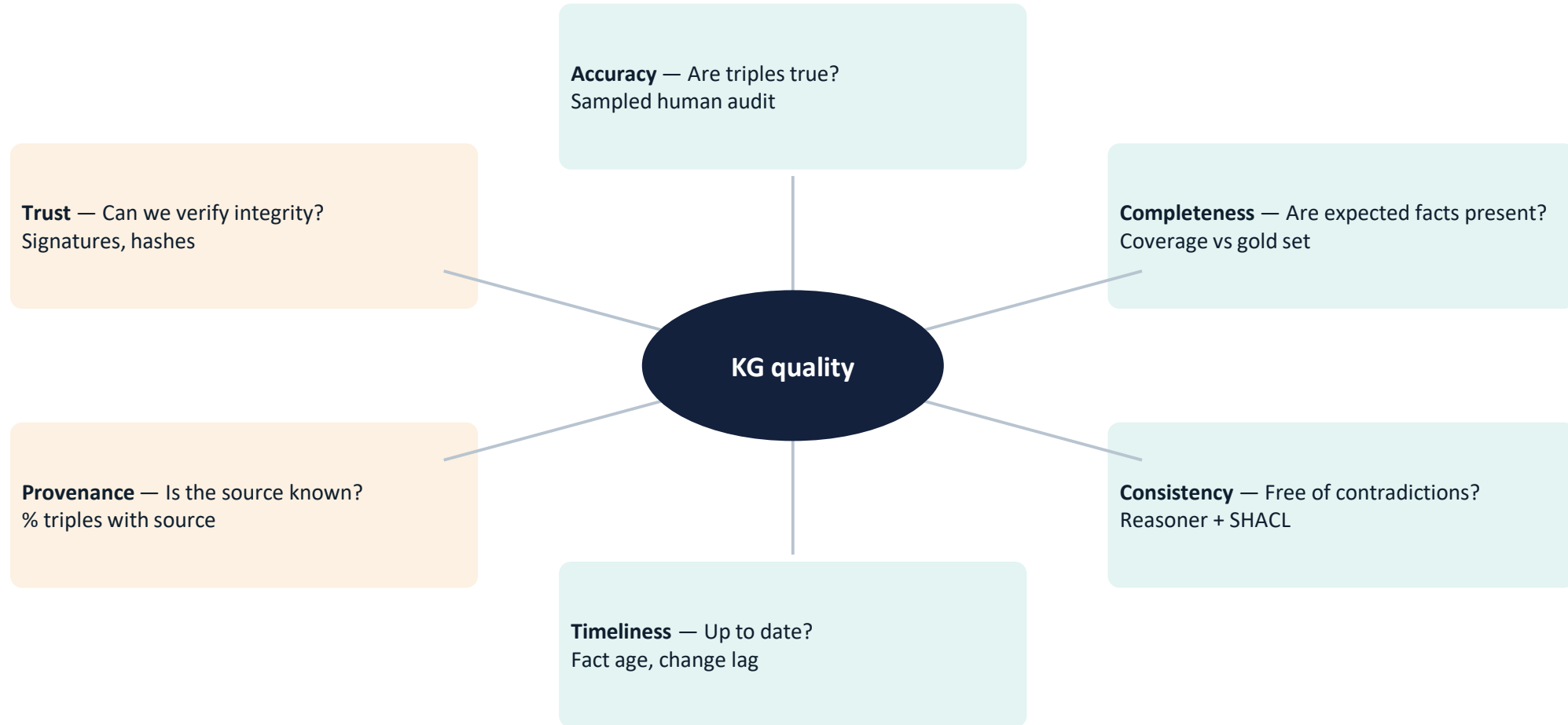
- **MRR** = $\text{mean}(1/\text{rank}) = (1 + 1/3 + 1/2 + 1/10)/4 = \mathbf{0.483}$
- **Hits@1** = $1/4 = 0.25$ **Hits@3** = $3/4 = 0.75$ **Hits@10** = $4/4 = 1.0$
- **Filtered**: remove other known-true answers before ranking so correct alternatives are not penalised

Metrics for the toy test set (computed)



Knowledge-graph quality dimensions

What 'good' means — and how each is measured



Gold dimensions are where cryptographic ledgers add capability KGs lack natively.

Landmark knowledge graphs

Public and enterprise graphs that shaped the field

Graph	Origin	Model	Notable for
Wikidata	Wikimedia (2012)	RDF-exportable statements with qualifiers & references	Collaborative, CC0, QIDs
DBpedia	Leipzig / Mannheim (2007)	RDF extracted from Wikipedia infoboxes	Linked Open Data hub
YAGO	Max Planck Institute (2007)	RDF + schema.org taxonomy	High accuracy, temporal facts
Google Knowledge Graph	Google (2012)	Proprietary	Popularised the term
Schema.org	Search engines (2011)	Shared vocabulary	Web markup in JSON-LD
SNOMED CT / UMLS	Healthcare	OWL EL / metathesaurus	Clinical terminology

KEY INSIGHT

Wikidata's statement model (value + qualifiers + references) is effectively provenance-aware — a design pattern Part III revisits.

Part I summary

Six ideas to carry into the blockchain part

01 Triples

Facts are atomic, mergeable statements.

02 Identity

Global IDs make data joinable across owners.

03 Semantics

Schemas enable inference; OWL is open-world.

04 Validation

SHACL enforces closed-world constraints.

05 Learning

Embeddings & GNNs predict missing links.

06 Provenance

Trustworthiness depends on knowing origins — the gap blockchains fill.

Knowledge check — Part I

Q1

Why is a literal never the subject of an RDF triple?

Answer: Literals are values, not resources; RDF 1.1 allows them only as objects so they cannot carry properties.

Q2

ex:bornIn rdfs:range ex:Place. What does asserting ex:Ada ex:bornIn ex:Bob entail?

Answer: That ex:Bob is a Place — RDFS range infers types; it does not reject data.

Q3

Name one query SPARQL property paths can express that a fixed BGP cannot.

Answer: Transitive reachability of unknown length, e.g. `?x ex:partOf+ ex:England`.

Q4

Why can TransE not model a symmetric relation well?

Answer: $h + r = t$ and $t + r = h$ imply $r = 0$, collapsing h and t together.

Q5

Open world: no phone number for Ada. Answer to “Does Ada have a phone?”

Answer: Unknown — absence of a triple is not evidence of falsity.

PART II

Blockchain Technology

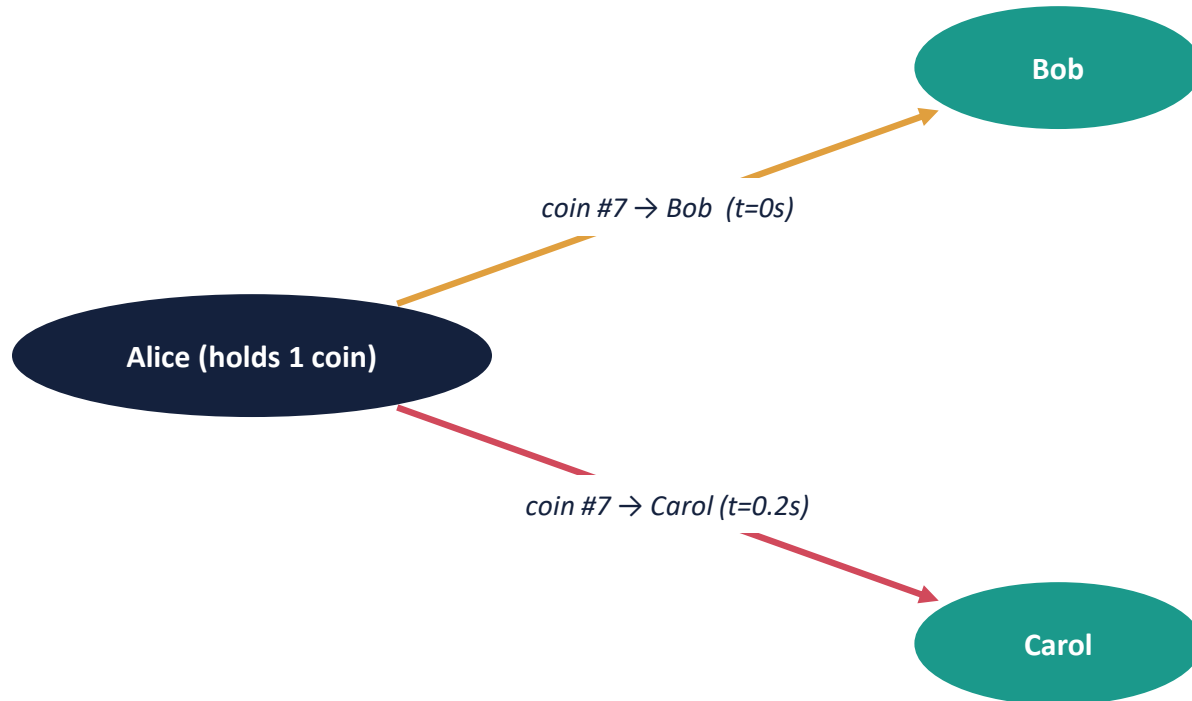
A shared, append-only ledger that strangers can trust without trusting each other

▸ Why ▸ Cryptographic primitives ▸ Blocks & transactions ▸ Consensus ▸ Networks ▸ Smart contracts ▸ Scaling ▸ Enterprise ▸ Applications ▸ Security & regulation



The core problem: agreeing on a history without a referee

Digital value is just bits — and bits can be copied



Traditional fix

A trusted intermediary (bank, Visa, registry) keeps THE ledger and orders transactions.

Blockchain fix

Everyone keeps a copy; a consensus protocol makes all honest copies converge on one order.

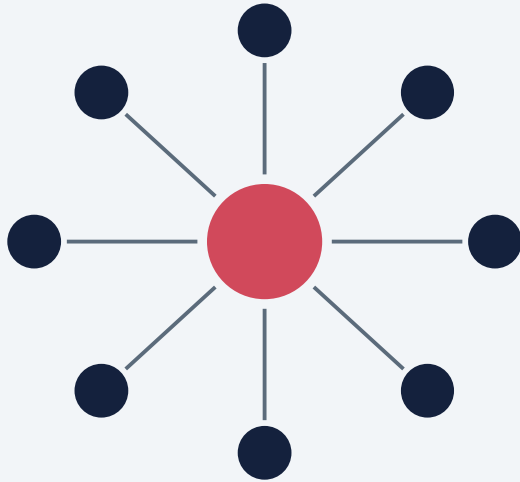
KEY INSIGHT

The double-spend problem is an ORDERING problem: if everyone agrees which transaction came first, the second is simply invalid.

Centralised, decentralised, distributed

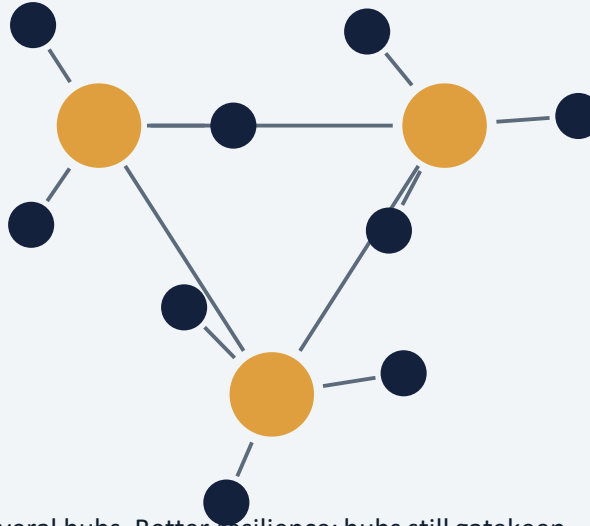
Baran's 1964 network taxonomy — still the clearest way to see what blockchains change

Centralised



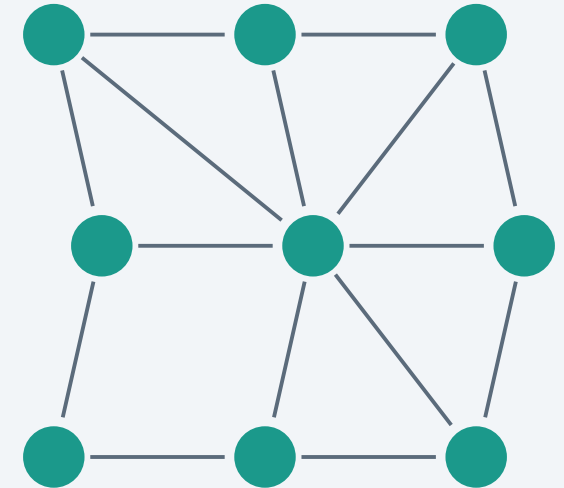
One node controls state. Efficient; single point of failure & control.

Decentralised



Several hubs. Better resilience; hubs still gatekeep.

Distributed (P2P)



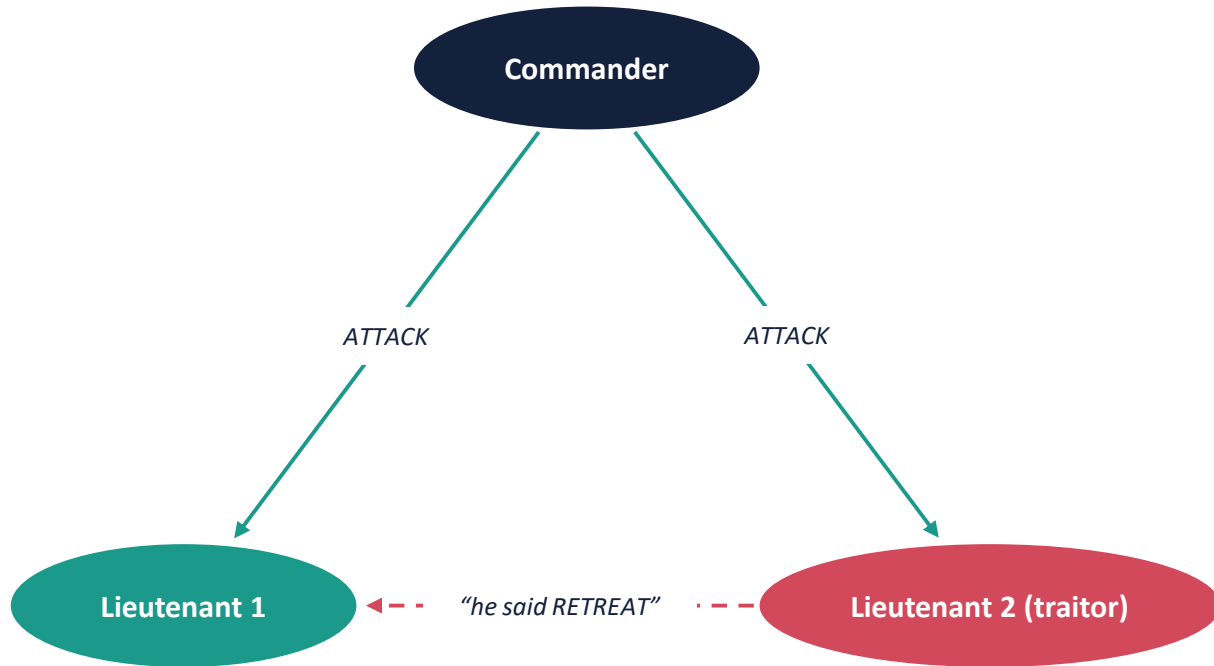
Every node peers. Blockchains aim here for control, not just storage.

KEY INSIGHT

‘Decentralised’ in blockchain discourse means no single party controls the ledger’s rules or history — not merely that copies are replicated.

The Byzantine Generals Problem

Lamport, Shostak & Pease (1982): agreement when some participants may lie



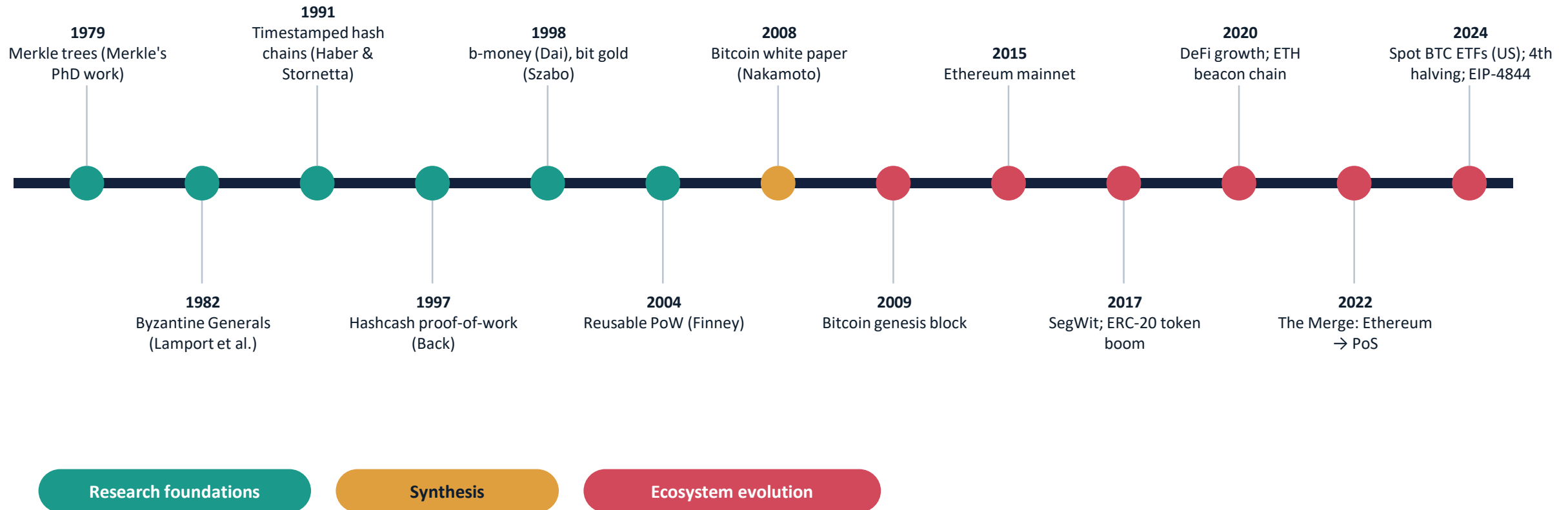
- **Setting:** n generals must agree on a plan; messages may be forged by traitors.
- **Result:** with oral (unsigned) messages, agreement needs $n \geq 3f + 1$ to tolerate f traitors — 3 generals cannot tolerate 1.
- **With signatures:** forgery becomes detectable; the bound improves (any n with $f < n$ under synchrony).
- **Blockchain link:** nodes are generals, blocks are plans, and Bitcoin (2008) gave the first practical open-membership solution.

KEY INSIGHT

Byzantine = arbitrary faults: crash, delay, lie, collude. Crash-fault protocols (Raft, Paxos) are not enough for open networks.

Timeline: 30 years of ideas converging in 2008

Bitcoin was an integration of existing research, not a single invention



Defining blockchain precisely

An annotated definition — every clause does work

A blockchain is a replicated, append-only ledger of cryptographically linked blocks, whose order is agreed by a consensus protocol among mutually distrustful parties.

1

Replicated

Every full node holds a copy —
no master copy exists

2

Append-only

History is extended, never
edited; corrections are new
entries

3

Cryptographically linked

Each block commits to its
predecessor's hash → tamper-
evidence

4

Consensus

Rules decide which valid
extension becomes canonical

5

Mutually distrustful

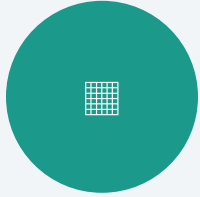
Security must hold even if some
parties are adversarial

KEY INSIGHT

Drop 'mutually distrustful' and a replicated database does the job faster. That clause is why blockchains exist.

Three lenses on the same system

Confusion usually comes from mixing lenses



Data structure

A hash-linked list of blocks, each holding a Merkle-committed batch of transactions.

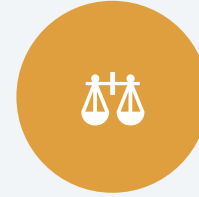
Provides: Integrity, tamper-evidence



Network

Thousands of peers gossiping transactions and blocks over the internet.

Provides: Availability, censorship resistance



Protocol / institution

Rules for validity, consensus and incentives that define ownership and state.

Provides: Agreement, economic security

KEY INSIGHT

Part II follows these lenses in order: primitives & data structures → transactions → consensus → networks → programmability.

Cryptographic Primitives

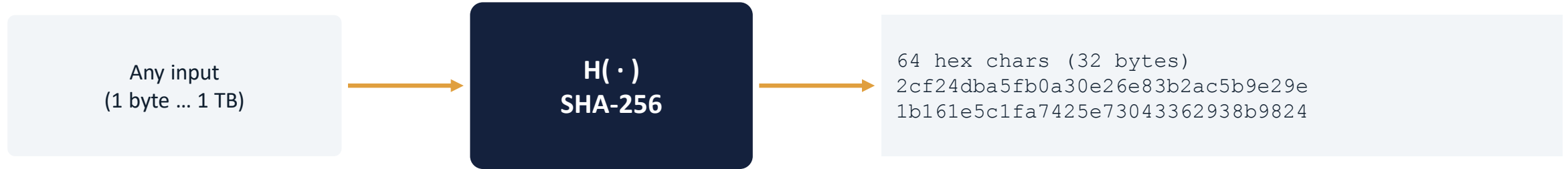
The four tools that make a ledger tamper-evident and ownership provable

▸ Hash functions ▸ Merkle trees ▸ Public-key cryptography ▸ Digital signatures ▸ Addresses & wallets



Cryptographic hash functions

A deterministic fingerprint: any input $\rightarrow$ fixed-size output (SHA-256: 256 bits)



1 Pre-image resistance

Given y , infeasible to find x with $H(x) = y$. (Hides data; basis of commitments.)

2 Second pre-image resistance

Given x , infeasible to find $x' \neq x$ with $H(x') = H(x)$. (Stops substitution.)

3 Collision resistance

Infeasible to find ANY pair $x \neq x'$ with equal hashes. Birthday bound $\approx 2^{128}$ work for SHA-256.

4 Puzzle-friendliness

No shortcut to find x such that $H(k||x) < \text{target}$ — only brute force. (Enables proof-of-work.)

The avalanche effect — computed live

One-character changes flip about half the output bits; outputs look unrelated

"hello"

2cf24dba5fb0a30e26e83b2ac5b9e29e
1b161e5c1fa7425e73043362938b9824

reference

"Hello"

185f8db32271fe25f561a6fc938b2e26
4306ec304eda518007d1764826381969

125 of 256 bits differ (48.8%)

"hello!"

ce06092fb948d9ffac7d1a376e404b26
b7575bcc11ee05a4615fef4fec3a308b

130 of 256 bits differ (50.8%)

Red characters differ from the reference hash. Values produced with Node.js crypto.createHash('sha256') at build time.

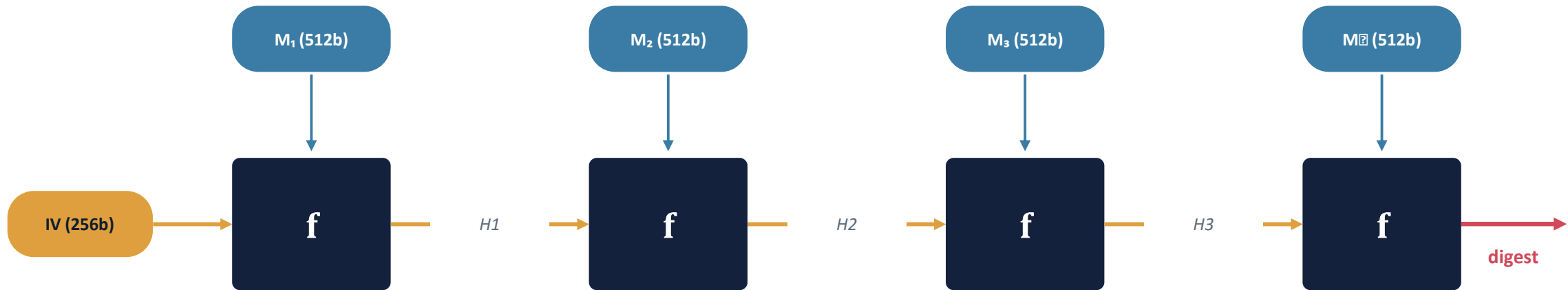
KEY INSIGHT

Because tiny edits scramble the fingerprint, comparing two 32-byte hashes verifies gigabytes of data.

Inside SHA-256: the Merkle–Damgård construction

Pad → split into 512-bit blocks → chain a compression function

message || 1 || 0...0 || length(64 bits)



Compression f

64 rounds of 32-bit additions, rotations, Σ/σ functions, Ch and Maj over 8 working registers.

Design goal

Collision resistance of $f \Rightarrow$ collision resistance of H (Merkle–Damgård theorem).

Known weakness

Length extension: $H(m)$ lets you compute $H(m||pad||x)$. Hence HMAC, SHA-3 or double-SHA-256 in Bitcoin.

Hash pointers: the chain in 'blockchain'

A pointer that says where data is AND what it must look like



Each 'prev' equals the previous block's 'hash' (real SHA-256 values, truncated). The final hash commits to the entire history.

Tamper-evident, not tamper-proof

Anyone can edit bytes; nobody can edit them without the hashes changing.

Commitment

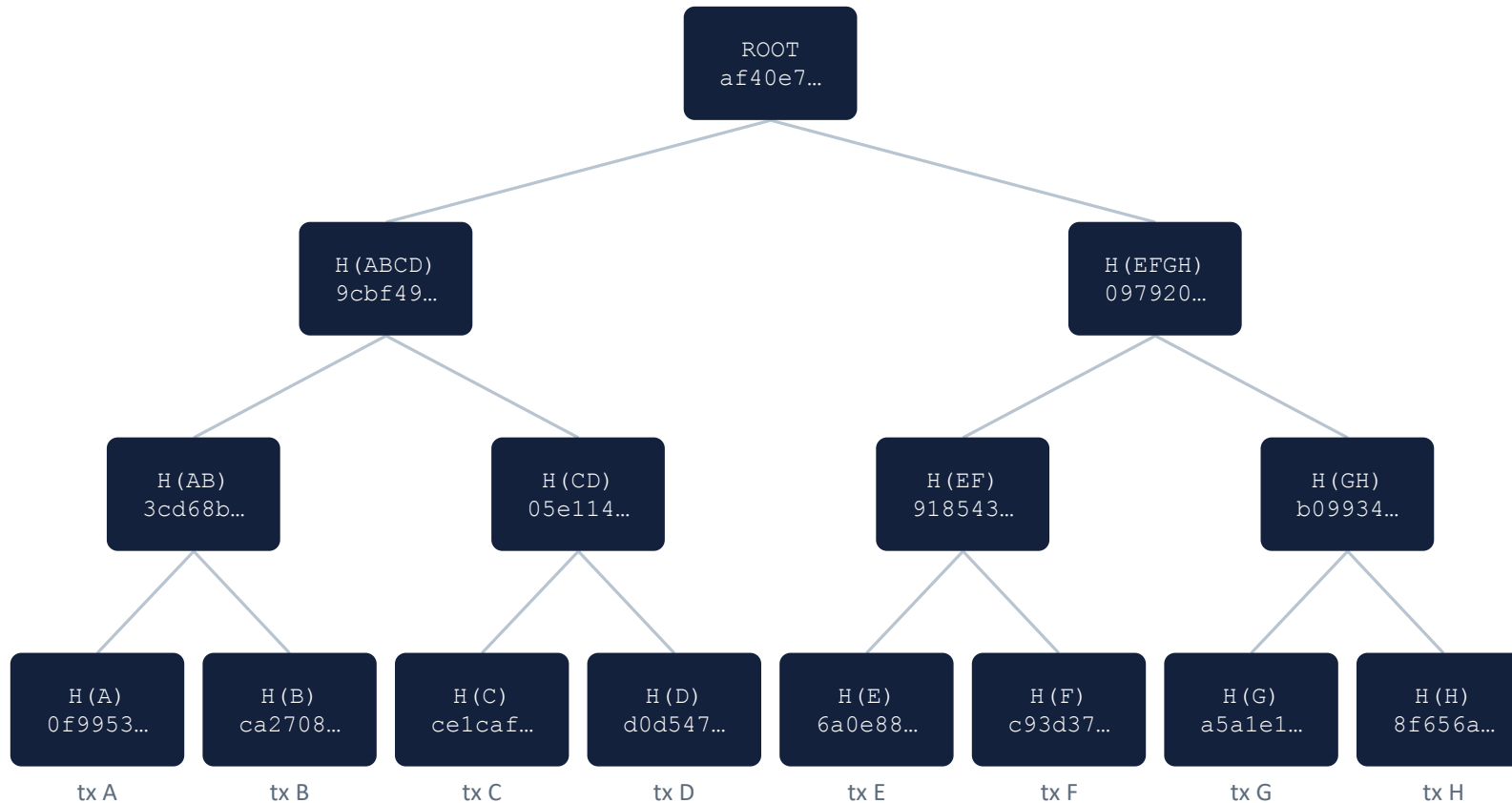
Keeping only the latest hash (32 bytes) lets you verify any claimed history.

Origin

Haber & Stornetta (1991) used exactly this to timestamp documents.

Merkle trees: committing to a set with one hash

Pairwise hashing up to a single Merkle root stored in the block header

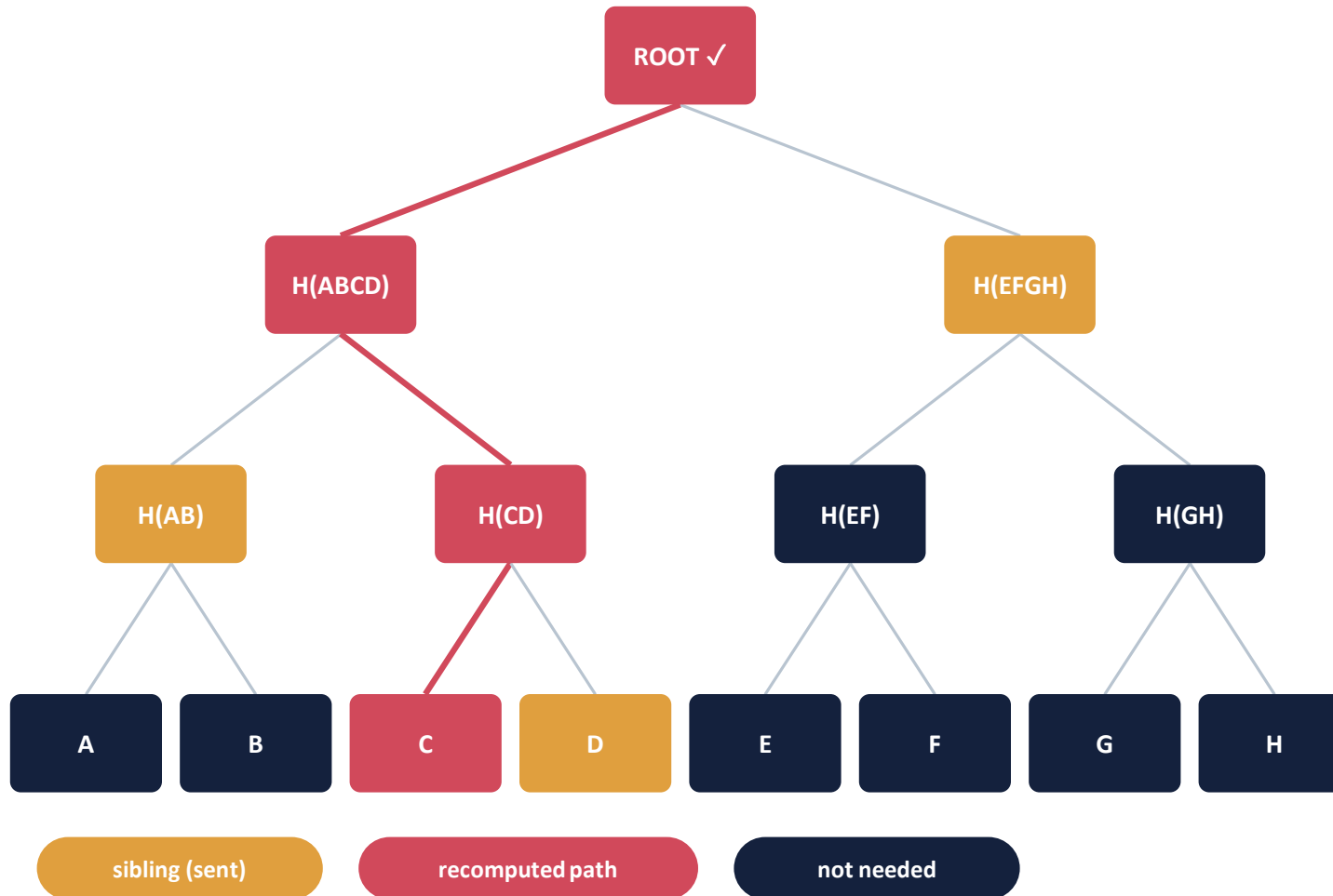


Hash prefixes are real SHA-256 outputs of the labels shown.

- **Root** changes if any transaction changes, is added, removed or reordered.
- **Header stays small** (80 bytes in Bitcoin) whatever the block size.
- **Odd count:** Bitcoin duplicates the last hash at that level.
- **Bitcoin** uses double SHA-256; Ethereum uses Keccak-256 in Merkle Patricia tries.

Merkle proofs: verifying membership in $O(\log n)$

Prove 'tx C is in the block' with 3 hashes instead of 8 transactions



- **Verifier holds:** only the block header (with root).
- **Prover sends:** C, plus sibling hashes **H(D)**, **H(AB)**, **H(EFGH)** (gold).
- **Verifier recomputes** (red path): $H(C) \rightarrow H(CD) \rightarrow H(ABCD) \rightarrow \text{root}$.
- **Match** $\Rightarrow$ C is included. Size = $\log_2 n$ hashes: 20 hashes for a million transactions.
- Used by **SPV** light clients (Nakamoto §8) and in rollups, bridges, certificate transparency.

Public-key cryptography

A key pair: the private key signs, the public key verifies — and cannot be reversed

Random 256-bit private key k

Must be generated from a CSPRNG. Anyone who learns k controls the funds — forever.

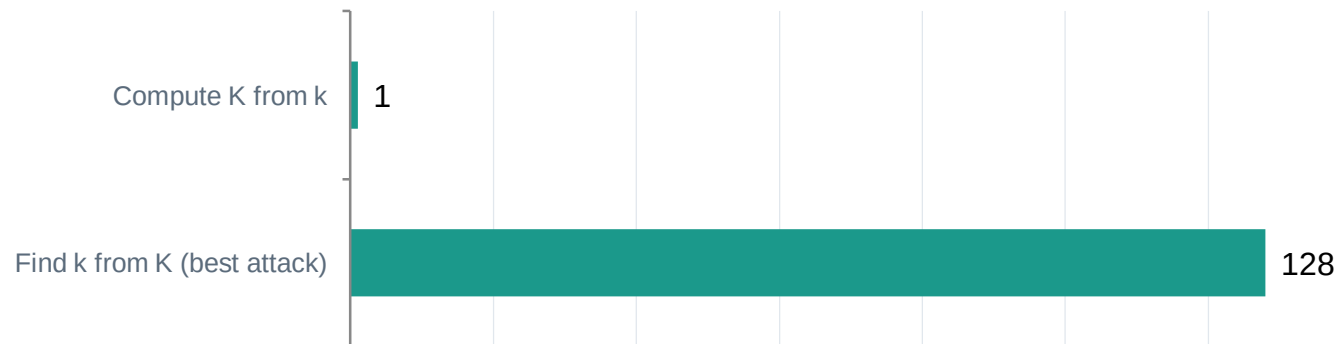
Public key $K = k \cdot G$

Elliptic-curve scalar multiplication: fast one way, infeasible to invert (ECDLP).

Address = $\text{hash}(K)$

A short, checksummed identifier shared publicly to receive funds.

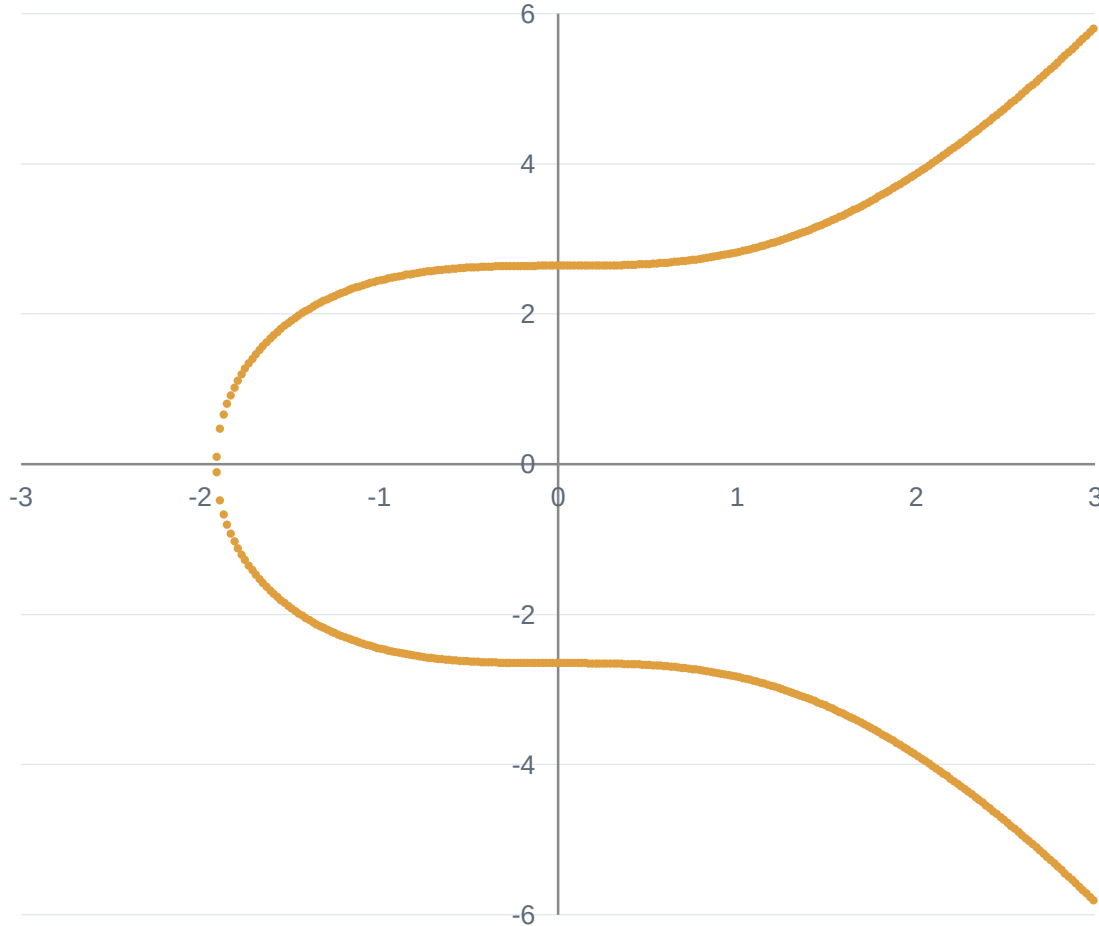
Security in $\log_2(\text{operations})$ — secp256k1



Deriving K takes $\sim$ one scalar multiplication (think ' $\approx 2^0$ ' units of work); the best known attack (Pollard's rho) needs $\approx 2^{128}$ group operations. That asymmetry is the whole security model.

Elliptic curves, visualised: secp256k1

Bitcoin and Ethereum use $y^2 = x^3 + 7$ — shown here over the real numbers for intuition



- **Point addition:** draw a line through P and Q; it meets the curve at a third point; reflect it $\rightarrow P + Q$.
- **Doubling:** tangent at P gives 2P. Repeat $\rightarrow k \cdot G$ via double-and-add in ~ 256 steps.
- **In practice** the curve is over a prime field $\mathbb{F}_p$, $p = 2^{256} - 2^{32} - 977$: a scatter of points, not a smooth curve.
- **G** is a fixed generator point; the group order $n \approx 1.158 \times 10^{77}$.

KEY INSIGHT

Why EC? 256-bit EC keys give security comparable to $\sim 3,072$ -bit RSA keys — small keys and signatures.

Digital signatures (ECDSA)

Proves that the holder of k authorised exactly this message — without revealing k

SIGN (wallet, private)

$$z = H(tx)$$

random nonce r_k

$$R = r_k \cdot G; r = R.x$$

$$s = r_k^{-1}(z + r \cdot k) \bmod n$$

Output signature (r, s)

$(r, s), K, tx$

VERIFY (every node, public)

$$z = H(tx)$$

$$u_1 = z \cdot s^{-1}, u_2 = r \cdot s^{-1}$$

$$R' = u_1 \cdot G + u_2 \cdot K$$

$$\text{valid} \Leftrightarrow R'.x \equiv r$$

PITFALL

Nonce reuse is fatal: two signatures with the same r_k reveal k by simple algebra (the 2010 PlayStation 3 key leak). Use RFC 6979 deterministic nonces.

Beyond ECDSA: Schnorr and BLS signatures

Linearity enables aggregation — many signers, one signature

Property	ECDSA	Schnorr (BIP-340)	BLS12-381
Used by	Bitcoin (legacy), Ethereum txs	Bitcoin Taproot (2021)	Ethereum consensus layer
Signature size	~71–72 B (DER)	64 B	96 B (48 B pubkey)
Security proof	Weaker (generic group)	Clean proof in ROM	Pairing assumptions
Linear / aggregatable	No	Yes (MuSig2 multisig)	Yes, non-interactive
Batch verification	No	Yes	Yes
Key win	Ubiquity	n-of-n looks like 1-of-1: privacy + fee savings	Thousands of validator votes → one signature

KEY INSIGHT

Aggregation is why Ethereum can have hundreds of thousands of validators voting every epoch without flooding the network.

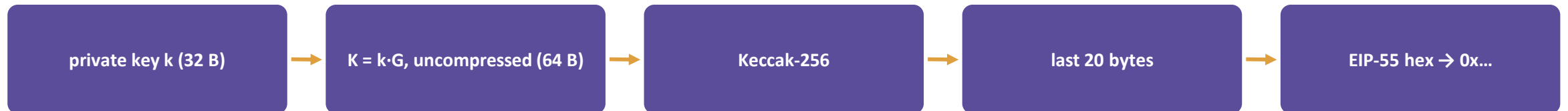
From private key to address

Two pipelines, same idea: hash the public key, add a checksum and an encoding

Bitcoin (P2WPKH / SegWit)



Ethereum



Why hash the key?

Shorter identifiers, and quantum-hedging: K is not exposed until you spend.

Checksums

Bech32 and EIP-55 mixed-case detect typos before funds are lost.

Address $\neq$ account

An address is a hash commitment; in Bitcoin it is best used once.

Wallets: hierarchical deterministic keys

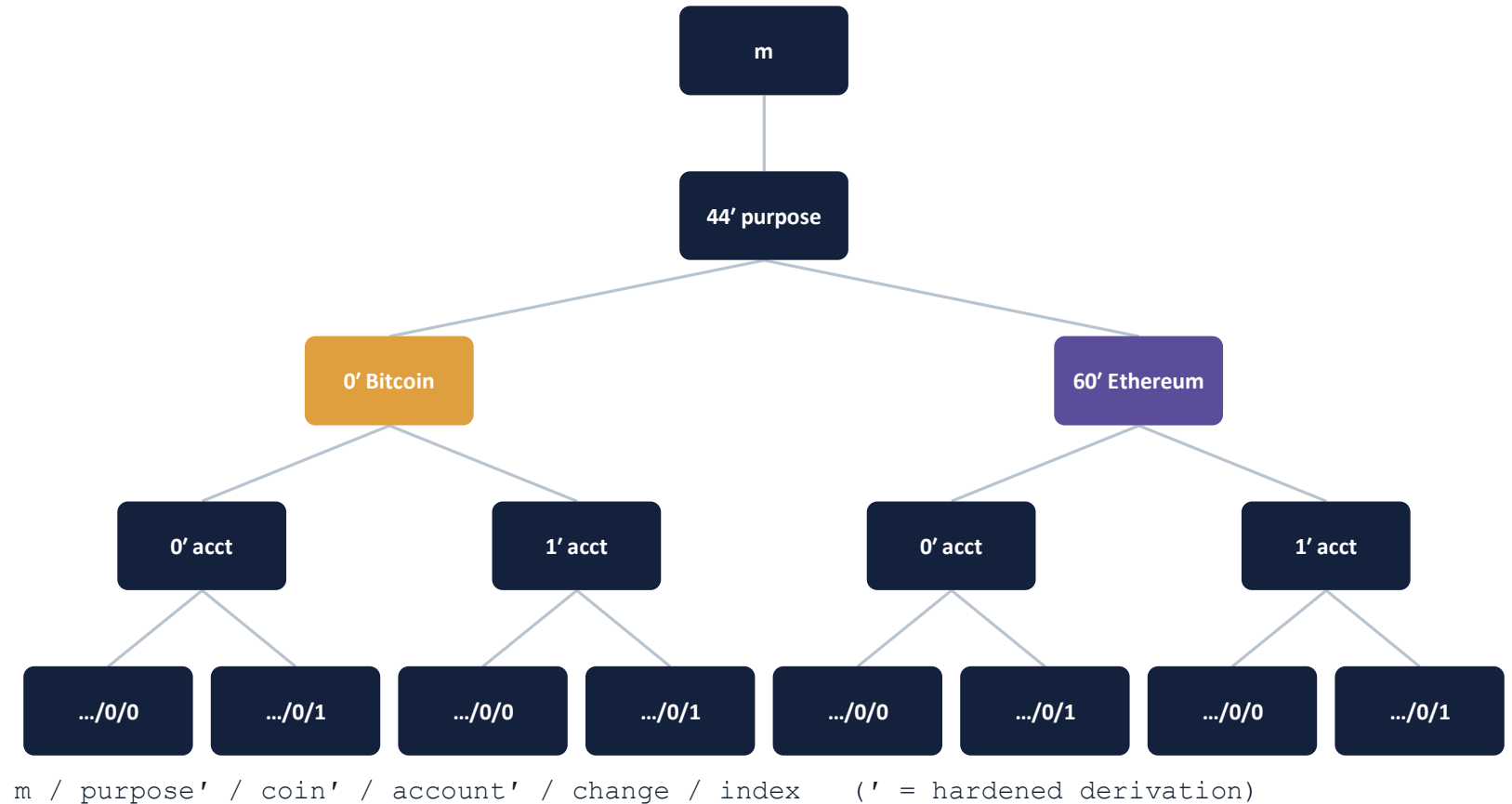
BIP-39 mnemonic → BIP-32 key tree → BIP-44 paths. One backup, unlimited keys

12 / 24 words from a 2,048-word list
(128–256 bits entropy + checksum)

PBKDF2-HMAC-SHA512 (2,048 rounds) →
512-bit seed

HMAC-SHA512 → master key *m* + chain code

*Custody spectrum: hot wallet → hardware wallet →
multisig → MPC / custodial*



Anatomy of Blocks & Transactions

What is actually inside a block, and how value moves

▸ Block header ▸ Chaining & tampering ▸ UTXO model ▸ Account model ▸ Lifecycle ▸ Fees ▸ Finality ▸ Script ▸ Forks



Anatomy of a Bitcoin block header — exactly 80 bytes

Every field, its size and its job



- Version (4 B)** Signals soft-fork readiness (BIP-9 bits)
- Prev block hash (32 B)** Hash pointer to parent — the chain link
- Merkle root (32 B)** Commits to every transaction in the block
- Timestamp (4 B)** Unix time; must exceed median of last 11 blocks
- nBits (target) (4 B)** Compact encoding of the difficulty target
- Nonce (4 B)** Free field miners vary to search for a valid hash

KEY INSIGHT

Block ID = SHA-256(SHA-256(header)). The body (transactions) is bound to the header only via the Merkle root.

Chaining blocks: what a node actually stores

Headers form the chain; bodies hang off them via Merkle roots

← toward genesis

chain tip →

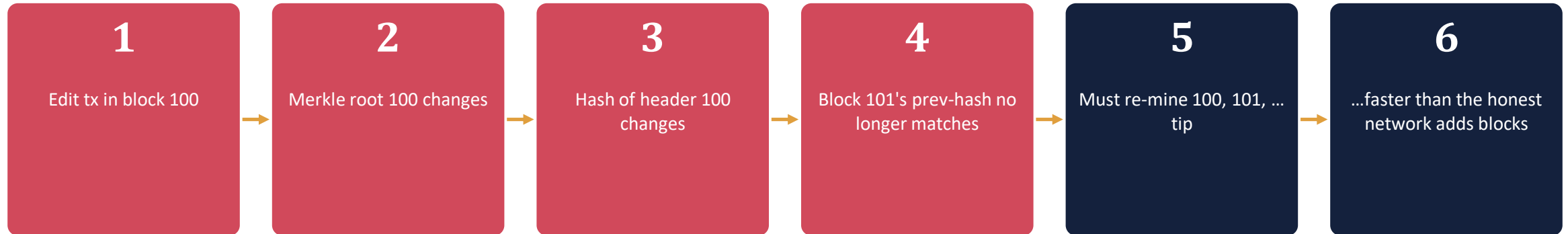


KEY INSIGHT

Header-only chains are tiny (80 B × ~900k blocks ≈ 72 MB) — this is what SPV light clients download.

Why tampering fails: the cascade

Edit one old transaction and every later block becomes invalid



```
original tx hash: 7c1366a320c8c064fb4d3d02dbd00b75d7fd0d27...
tampered (5 → 50): d256ebe7f9df01e029d7946c7ce23c5b469b6483...
```

KEY INSIGHT

The chain makes tampering DETECTABLE; proof-of-work (or stake) makes rewriting it EXPENSIVE. Security needs both.

Computed at build time with SHA-256.

The genesis block

Block 0 of Bitcoin, mined 3 January 2009 — hard-coded into every client

0

Height — it has no parent;
prev-hash is all zeros

50 BTC

Coinbase reward — famously unspendable
due to a code quirk

2009-01-03

Timestamp (18:15 UTC)

1

Transaction: the coinbase

Coinbase field message: “The Times 03/Jan/2009 Chancellor on brink of second bailout for banks”

Proof of earliest date

Quoting that day's headline proves the block was not pre-mined earlier.

Trust anchor

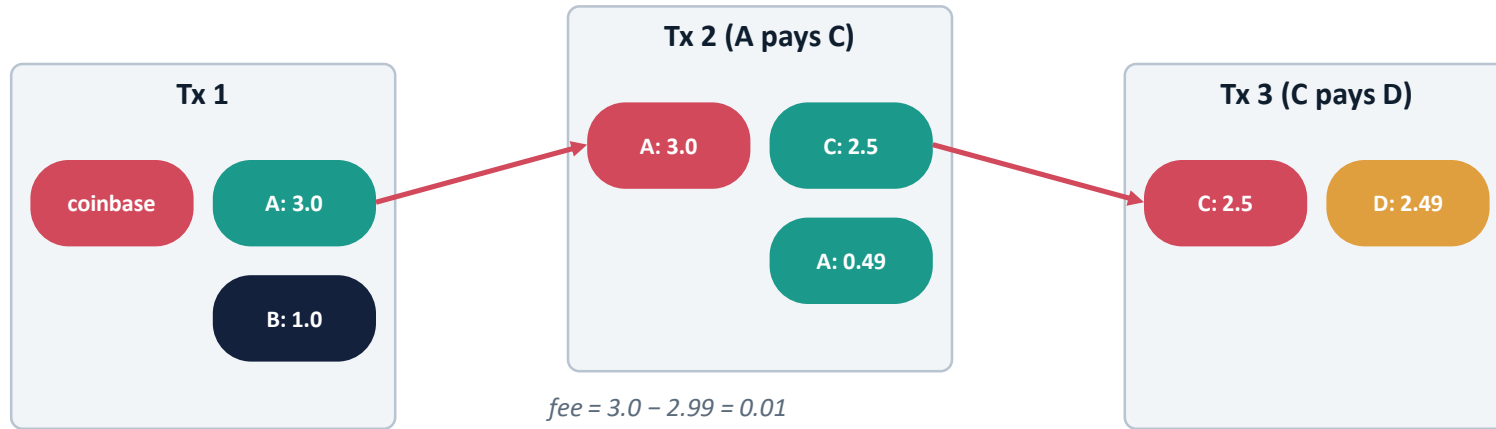
Clients validate the whole chain from this known hash; forks with another genesis are other networks.

Commentary

Widely read as a statement on the fragility of bank-centred money.

The UTXO model (Bitcoin)

No balances are stored — only a set of Unspent Transaction Outputs



- Outputs are **spent whole**; change returns to the sender
- Inputs reference (txid, index) and supply a **signature**
- $\Sigma \text{ inputs} \geq \Sigma \text{ outputs}$; difference = **miner fee**
- Validity check = 'is this outpoint in the UTXO set?'
- Parallel validation & strong privacy hygiene (fresh addresses)

Current UTXO set = { B: 1.0, A: 0.49, D: 2.49 } — every output not yet consumed by a later input

KEY INSIGHT

Like cash: you cannot tear a £20 note, so you hand it over and receive change.

A UTXO transaction, field by field

Simplified serialisation of Tx 2 from the previous slide

tx2.json (illustrative)

```
version      : 2
inputs       : [
  { prev_txid : 9f2c...e1 (Tx 1)
    vout      : 0           // which output
    witness   : <sig_A> <pubkey_A>
    sequence  : 0xffffffff // RBF enabled } ]
outputs      : [
  { value: 250000000 sats script: P2WPKH(hash160(C)) },
  { value: 490000000 sats script: P2WPKH(hash160(A)) } ]
locktime     : 0
```

Outpoint = (txid, vout) uniquely identifies the coin being spent

Witness (SegWit) holds the signature outside the txid → fixes malleability

Sequence < 0xffffffff signals Replace-By-Fee (BIP-125)

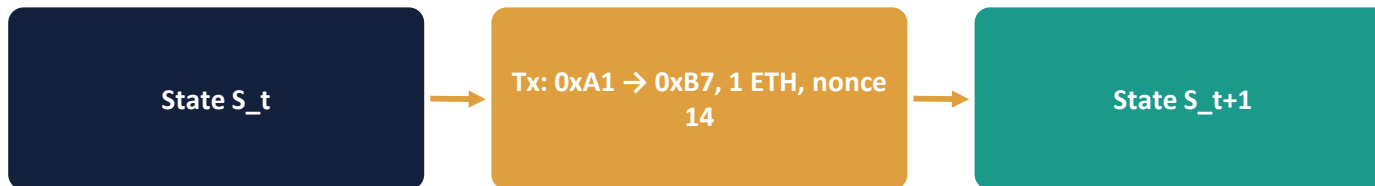
Values in satoshis: 1 BTC = 100,000,000 sats

Locktime can delay validity until a height/time

The account model (Ethereum)

Global state = a map from address $\rightarrow$ account; transactions mutate it

Address	Nonce	Balance (ETH)	Code hash	Storage root
0xA1... (EOA)	14	3.20	empty	empty
0xB7... (EOA)	2	0.75	empty	empty
0xC0... (contract)	1	120.00	0x5d...	0x9a... (key-value trie)



Result: 0xA1 nonce 15, balance 2.20 - fee; 0xB7 balance 1.75

- **EOA**: externally owned, controlled by a key
- **Contract**: controlled by its code
- **Nonce** prevents replay & orders an account's txs
- State transition: $\sigma' = Y(\sigma, T)$ (Yellow Paper)

KEY INSIGHT

Account-based chains are simpler for smart contracts; UTXO chains are simpler to validate in parallel.

UTXO vs account model

Neither is 'better' — they optimise for different workloads

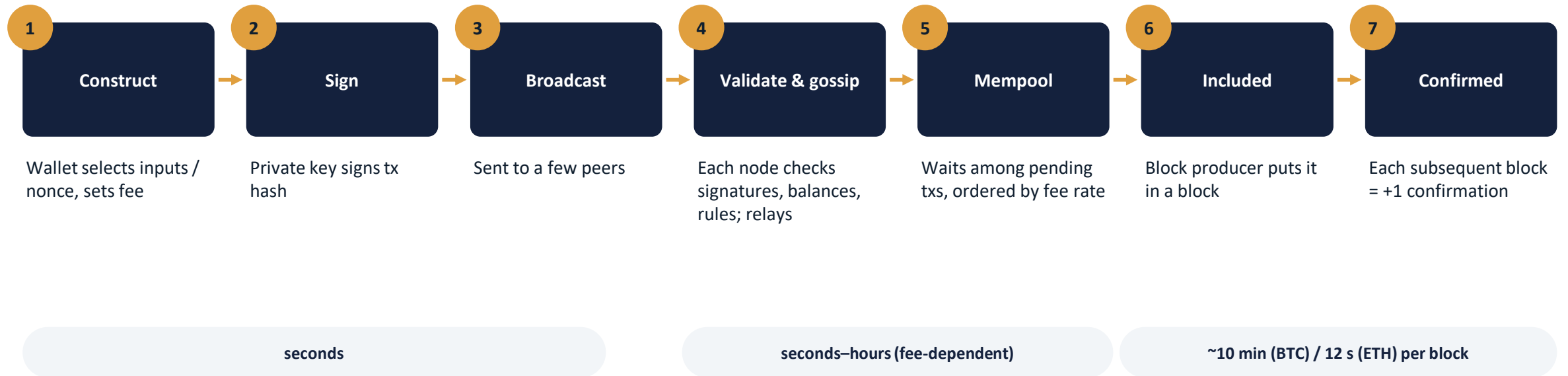
Dimension	UTXO (Bitcoin, Cardano eUTXO)	Account (Ethereum, Solana)
State	Set of unspent outputs	Map of balances + storage
Double-spend check	Is the outpoint unspent?	Is the nonce next & balance sufficient?
Parallelism	Natural: disjoint inputs	Needs access lists / scheduling
Privacy	Fresh addresses per payment	Address reuse by default
Smart contracts	Limited or eUTXO style	Rich shared mutable state
Replay protection	Outputs can be spent once	Account nonce + chain ID
Wallet complexity	Coin selection required	Just a balance

KEY INSIGHT

Rule of thumb: model assets you hand over like cash → UTXO; model shared programmable state → accounts.

Transaction lifecycle

From a click in a wallet to a probabilistically final entry in the ledger



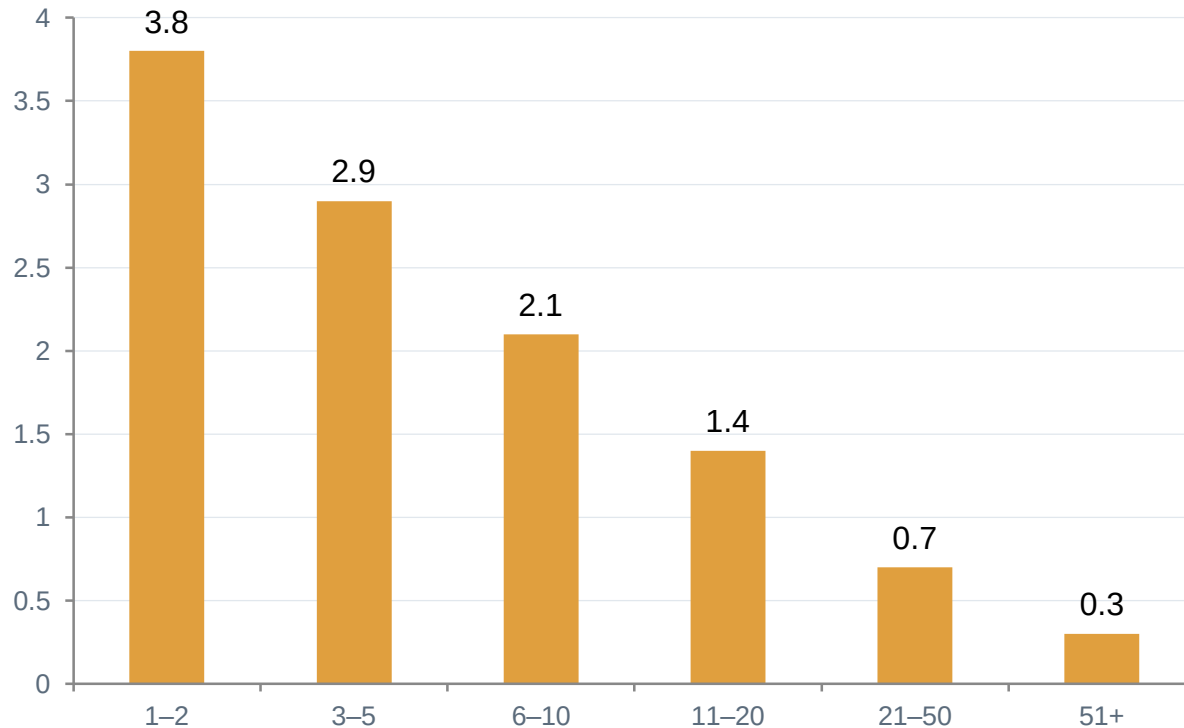
REMEMBER

A transaction can be valid yet never confirm (fee too low, conflicting spend). 'Sent' in a wallet is not 'settled'.

The mempool and fee markets

Block space is scarce; users bid for it

Illustrative mempool: MvB waiting per fee band (sat/vB)

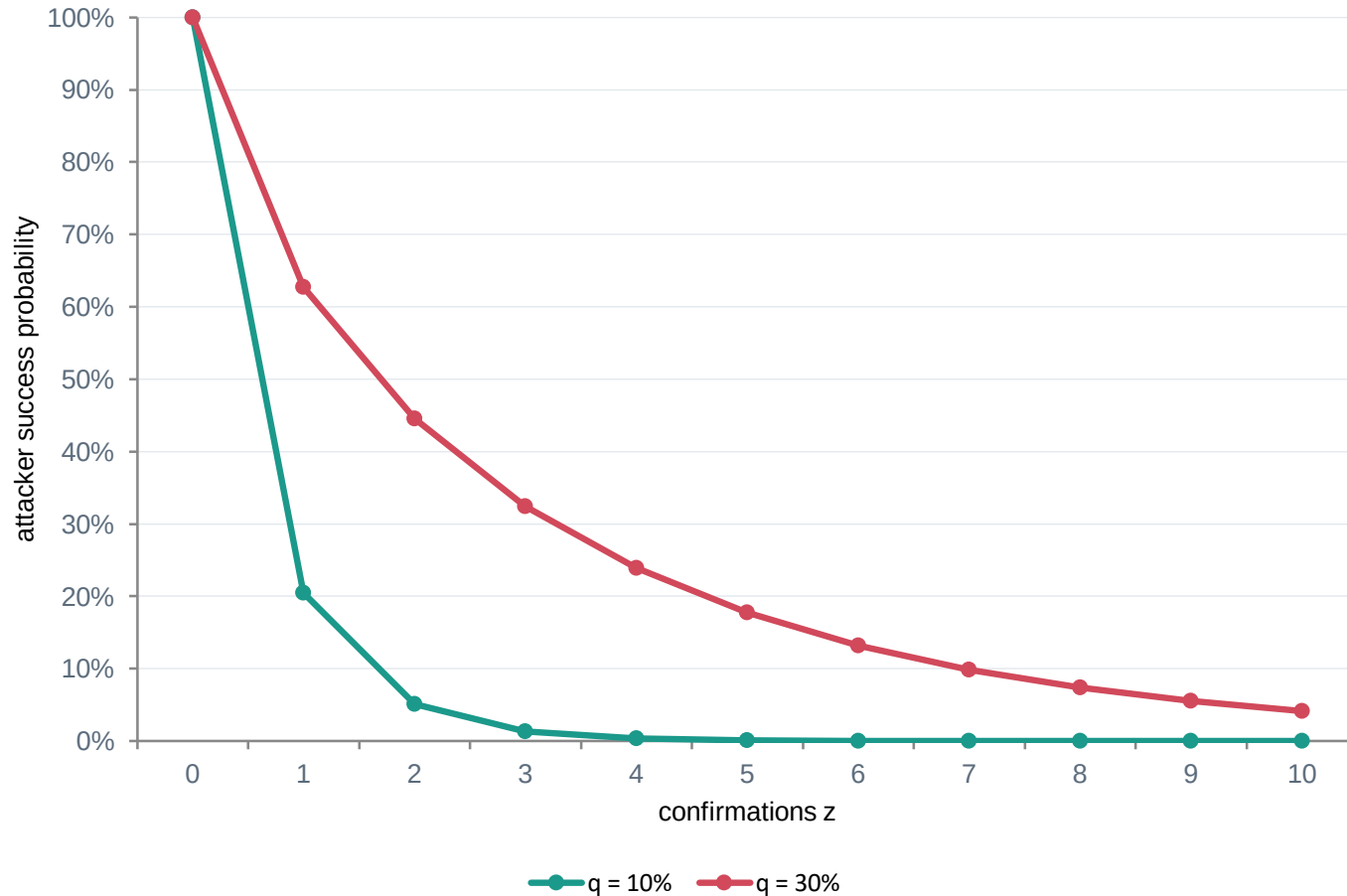


Illustrative shape, not live data

- **Bitcoin:** fee = rate (sat/vB) × virtual size; miners greedily fill ~1 MvB (4M weight units) by fee rate.
- **Replace-By-Fee:** rebroadcast with higher fee.
- **Child-Pays-For-Parent:** a high-fee child pulls its parent in.
- **Ethereum:** EIP-1559 base fee (burned) + priority tip — see Section F.
- **Why fees matter long-term:** block subsidies halve; fees must eventually pay for security.

Confirmations and probabilistic finality

Probability an attacker with share q ever catches up from z blocks behind (Nakamoto 2008, §11) — computed

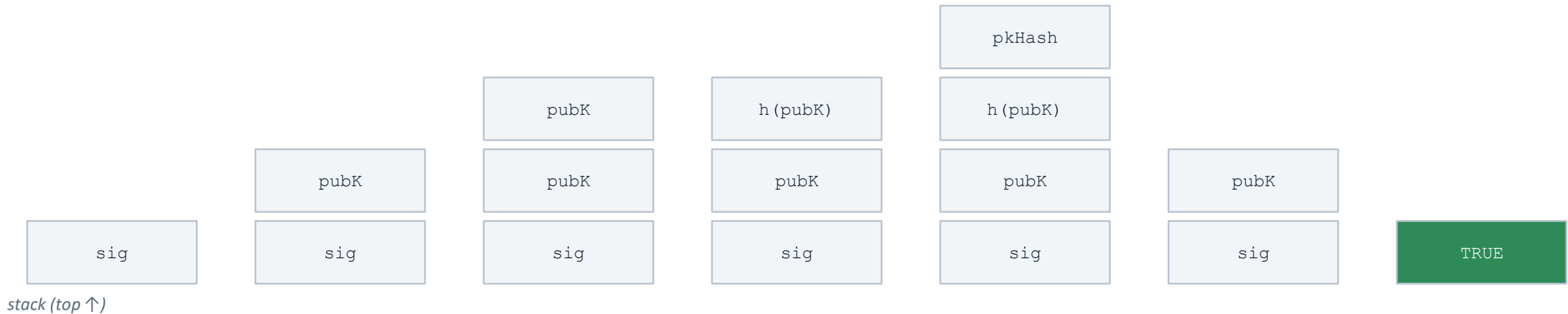


z	q = 10%	q = 30%
1	20.5%	62.8%
3	1.32%	32.5%
6	0.024%	13.2%
10	0.00012%	4.2%

Six confirmations became convention because at $q = 10\%$ risk falls below 0.1%. Against a 30% attacker, six is far from enough — finality is a function of the threat model.

Bitcoin Script: a stack machine for spending conditions

Pay-to-Public-Key-Hash (P2PKH) executed step by step



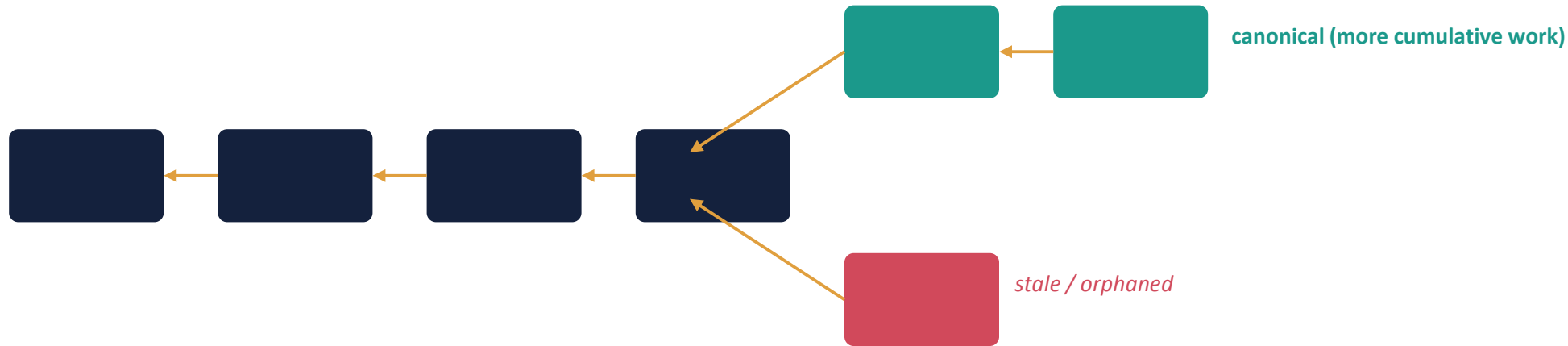
scriptSig (unlocking)

scriptPubKey (locking)

Deliberately not Turing-complete: no loops $\Rightarrow$ bounded execution. Enables multisig, timelocks (CLTV/CSV), hash-locks — the building blocks of Lightning.

Forks: when the chain splits

Temporary forks resolve by the heaviest-chain rule; protocol forks change the rules



Fork type	What changes	Compatibility	Examples
Natural / temporary	Two valid blocks at same height	Resolves in 1–2 blocks	Everyday occurrence
Soft fork	Rules tightened	Old nodes still accept new blocks	P2SH (2012), SegWit (2017), Taproot (2021)
Hard fork	Rules loosened / changed	Old nodes reject new blocks	Bitcoin Cash (2017), ETH/ETC split (2016)

Consensus Mechanisms

How thousands of nodes agree on one history — and what it costs to attack them

▸ Safety & liveness ▸ Proof of Work ▸ Difficulty ▸ 51% & selfish mining ▸ Proof of Stake ▸ Ethereum Gasper ▸ BFT: PBFT, Tendermint, HotStuff ▸ Comparison



What consensus must guarantee

Classic distributed-systems properties, and the impossibility results that bound them

S

Safety (consistency)

Honest nodes never finalise conflicting blocks. ‘Nothing bad happens.’

L

Liveness

Valid transactions are eventually included. ‘Something good eventually happens.’

!

FLP impossibility (1985)

No deterministic protocol guarantees both in a fully asynchronous network with even one crash fault.

Δ

CAP theorem

Under network partition choose consistency or availability. Nakamoto chains lean ‘available’; BFT chains lean ‘consistent’.

Network model	Assumption	Typical protocols
Synchronous	Messages arrive within known Δ	Nakamoto PoW (probabilistic)
Partially synchronous	Unknown GST, then bounded delay	PBFT, Tendermint, HotStuff, Casper FFG
Asynchronous	No timing bounds	Randomised: HoneyBadgerBFT

Proof of Work: one CPU cycle, one vote

Find a nonce so that the block's hash falls below a target

$$\text{SHA-256}(\text{header} \parallel \text{nonce}) < \text{target}$$

nonce = 0

7bcafb76bd71f29532da21dc6bb9090e1a7c8db19d4a...

reject

nonce = 1

c313f583c3e80b73b581f954f9136caa6493c39acf43...

reject

nonce = 2

324731f21ccab8c6c621b752a532d9f71314287c6be7...

reject

nonce = 8560

00008363fb7676adf54e4f7f7ebc4a80abd49a71173c...

VALID ✓

Target here: 4 leading hex zeros (16 bits) → expected $2^{16} = 65,536$ tries; this search found nonce 8,560 — computed at build time.

Hard to produce

Only brute force works (puzzle-friendliness).

Trivial to verify

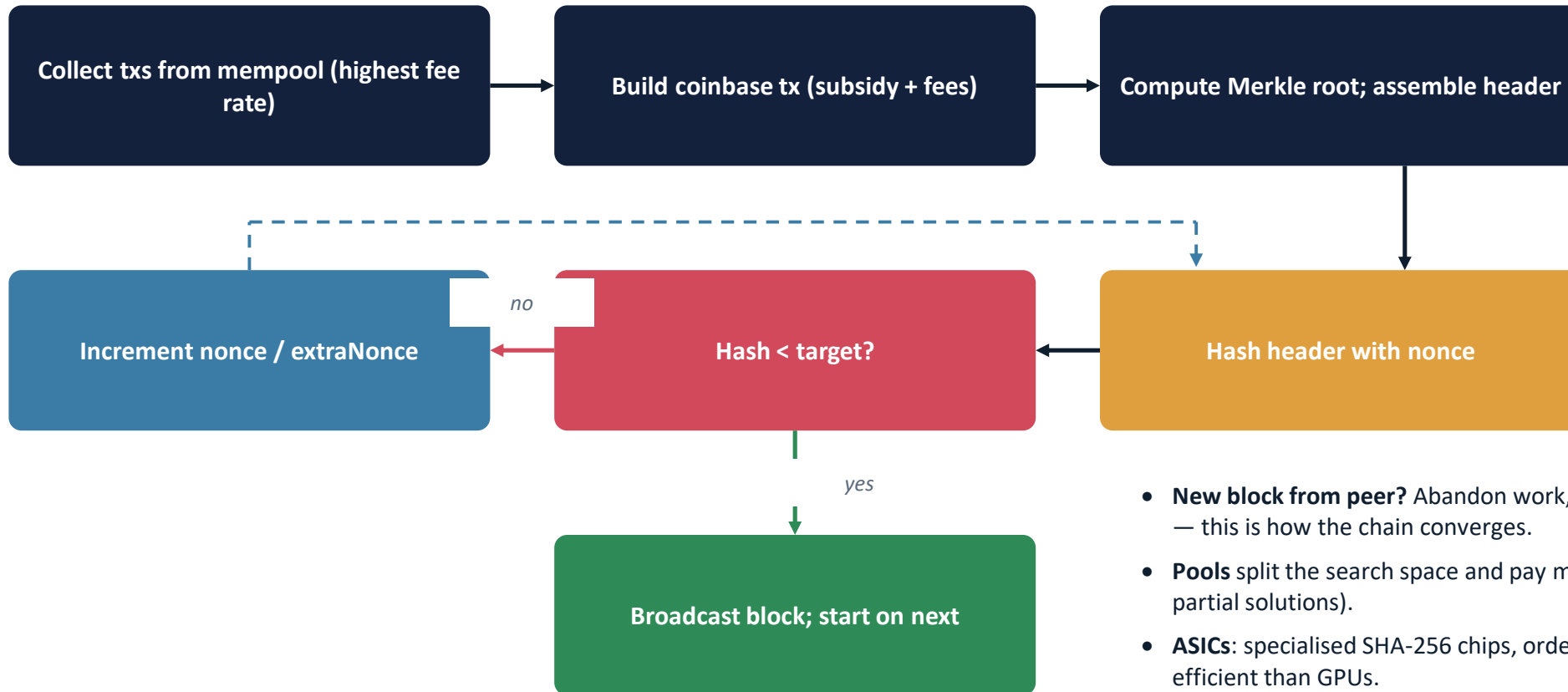
One hash checks the work.

Sybil-resistant

Votes cost energy, not identities.

The mining loop

What a miner's hardware does billions of times per second



- **New block from peer?** Abandon work, validate it, restart on top — this is how the chain converges.
- **Pools** split the search space and pay members by 'shares' (easier partial solutions).
- **ASICs**: specialised SHA-256 chips, orders of magnitude more efficient than GPUs.

Difficulty adjustment

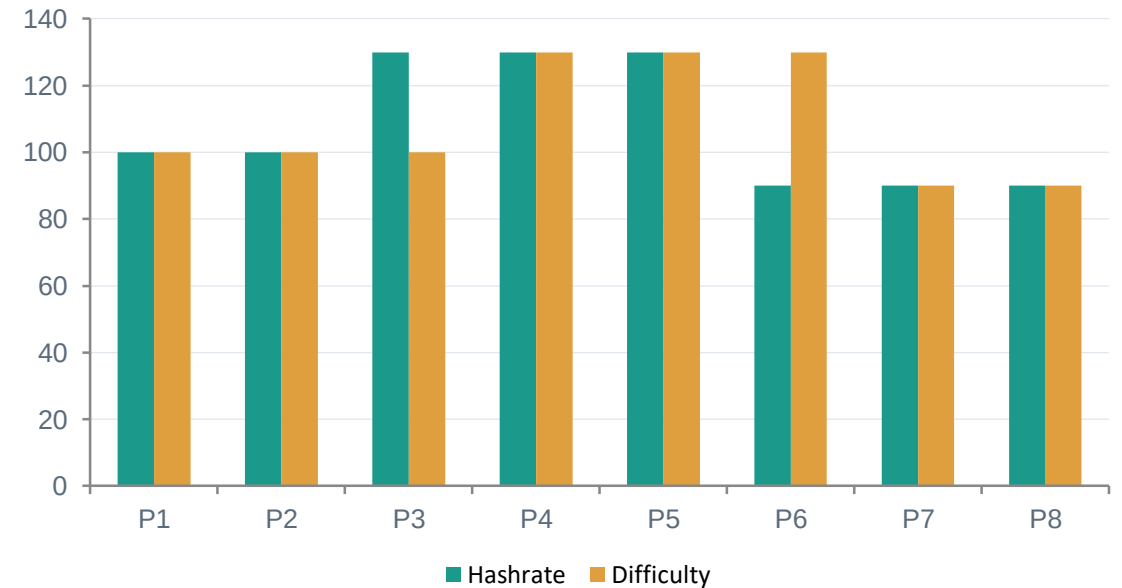
A feedback controller that holds block time near 10 minutes whatever the hashrate

$$\text{new_target} = \text{old_target} \times (\text{actual time of last 2016 blocks}) \div (2016 \times 600 \text{ s})$$

Block time per 2016-block period (simulated)



Hashrate vs difficulty (index)



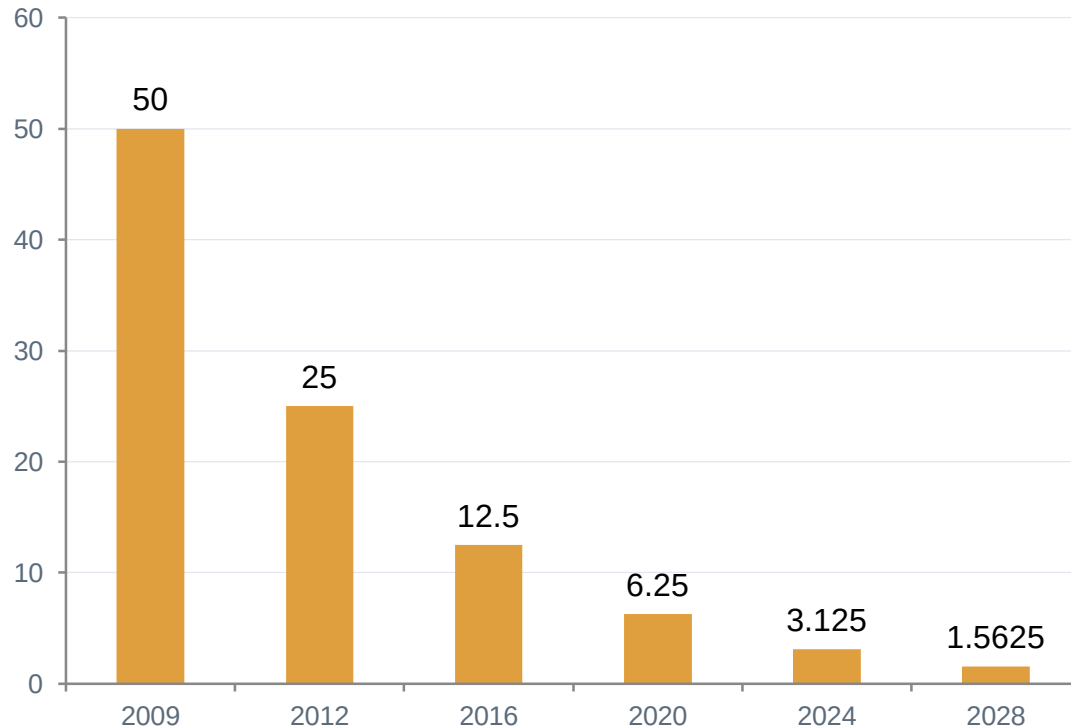
KEY INSIGHT

Hashrate +30% in P3 → 7.7-min blocks → difficulty rises → P4 back to 10.0. Hashrate -31% in P6 → 14.4 min → difficulty eases. Retargets are clamped to x4.

The economics of Proof of Work security

Security budget = what it costs to out-mine the honest network

Block subsidy (BTC) after each halving



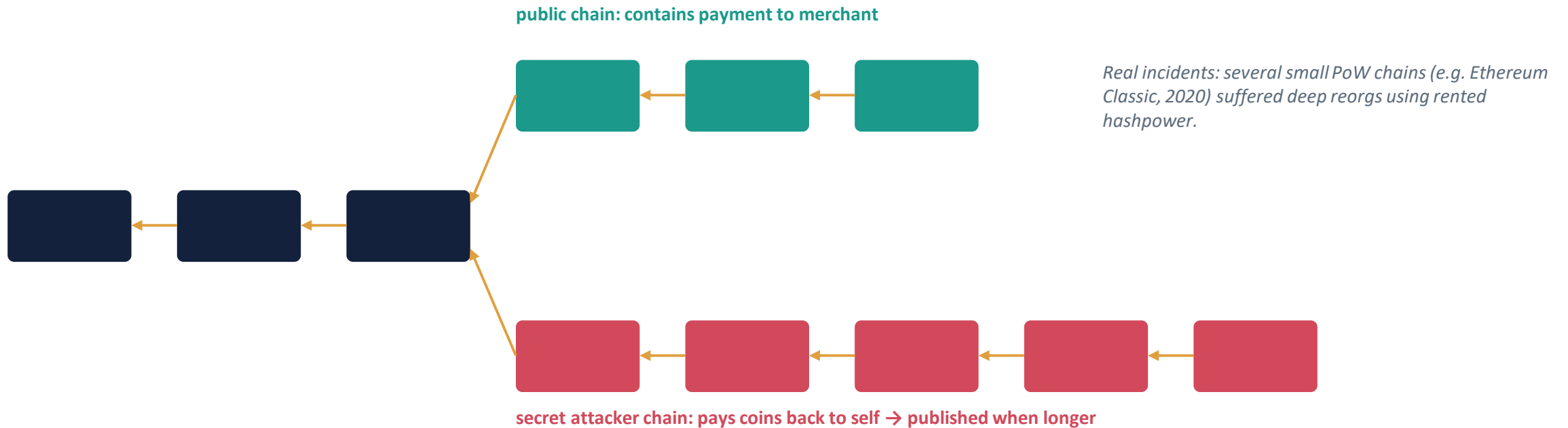
- **Miner revenue** = subsidy + fees, paid via the coinbase transaction.
- **Honest mining is rational**: an attacker with majority hashpower earns more by mining honestly than by destroying confidence in the coin it is paid in.
- **Energy is the point**: expended cost is what makes rewriting history expensive; this is also PoW's main criticism.
- **Fee transition**: as subsidies halve, fees must fund security — an open research question.

KEY INSIGHT

Security is economic, not absolute: a chain is as secure as the cost to acquire and run majority hashpower for the attack's duration.

The 51% attack

A majority miner can rewrite recent history — but only its own transactions' outcomes



Can

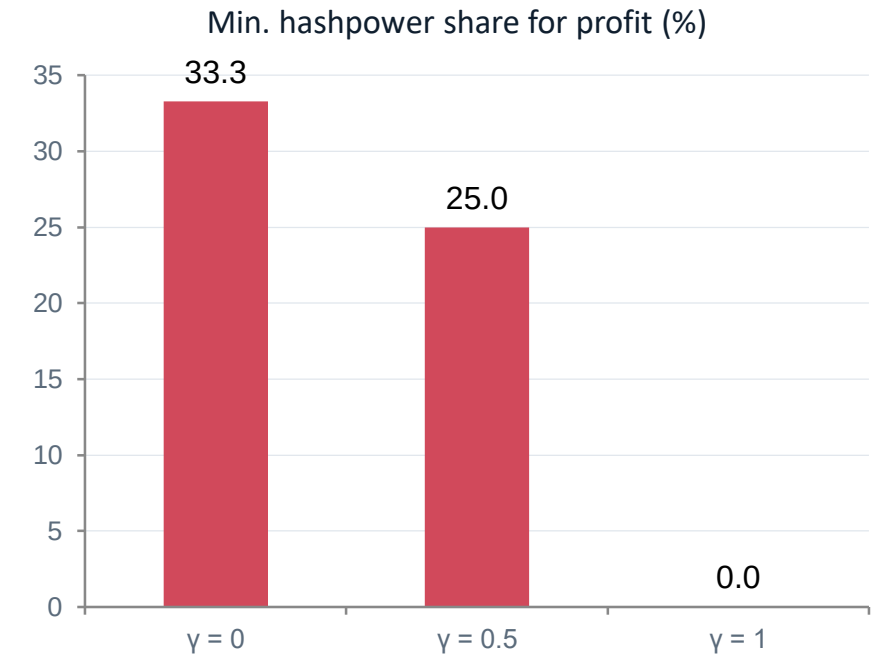
Double-spend own coins; censor or exclude transactions; orphan honest blocks.

Cannot

Forge signatures, steal others' coins, create coins beyond the rules (nodes reject invalid blocks).

Selfish mining

Eyal & Sirer (2014): withholding blocks can earn more than a fair share



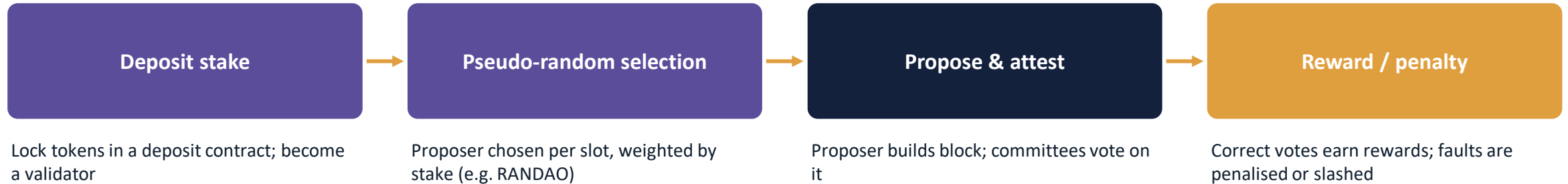
Threshold $\alpha > (1 - \gamma) / (3 - 2\gamma)$, per Eyal & Sirer.

KEY INSIGHT

The 'honest majority' assumption is really 'honest supermajority' once strategic behaviour is modelled.

Proof of Stake: capital instead of electricity

Validators lock (stake) tokens; misbehaviour destroys the stake



	Proof of Work	Proof of Stake
Scarce resource	Energy + hardware	Staked capital
Attack cost	Acquire >50% hashrate	Acquire $\geq \frac{1}{3}$ (liveness) or $\geq \frac{2}{3}$ (safety) of stake
Punishment	Wasted electricity (external)	Slashing: stake burned (internal)
Energy use	Very high	Dramatically lower (Ethereum est. ~99.95% drop)

Ethereum's PoS: slots, epochs and finality

Gasper = LMD-GHOST fork choice + Casper FFG finality gadget

Epoch N (32 slots × 12 s = 6.4 min)



checkpoint → FINALISED

Epoch N+1 (32 slots × 12 s = 6.4 min)



checkpoint → JUSTIFIED

Epoch N+2 (32 slots × 12 s = 6.4 min)



in progress

(each small bar = 2 slots; gold = epoch-boundary checkpoint)

Slot (12 s)

One proposer builds a block; a committee attests to the head.

LMD-GHOST

Fork choice: follow the subtree with the most recent attestation weight.

Casper FFG

$\frac{2}{3}$ of stake votes source → target checkpoints: justify, then finalise.

Economic finality

Reverting a finalised block requires $\geq \frac{1}{3}$ of all stake to be slashed.

KEY INSIGHT

Finality in ~2 epochs (~12.8 min). Validator minimum 32 ETH; since the Pectra upgrade (2025) max effective balance is 2,048 ETH.

Slashing: making equivocation expensive

Provable, attributable misbehaviour burns stake and ejects the validator

1 Double proposal

Signing two different blocks for the same slot.

2 Double vote

Two attestations with the same target epoch.

3 Surround vote

An attestation whose source–target span surrounds or is surrounded by another of yours.

Whistleblower submits two signed messages

Protocol verifies both signatures

Initial penalty + forced exit

Correlation penalty $\propto$ total slashed in window

KEY INSIGHT

Correlation penalties mean a lone mistake costs little, but a coordinated attack by $\frac{1}{3}$ of stake loses nearly everything.

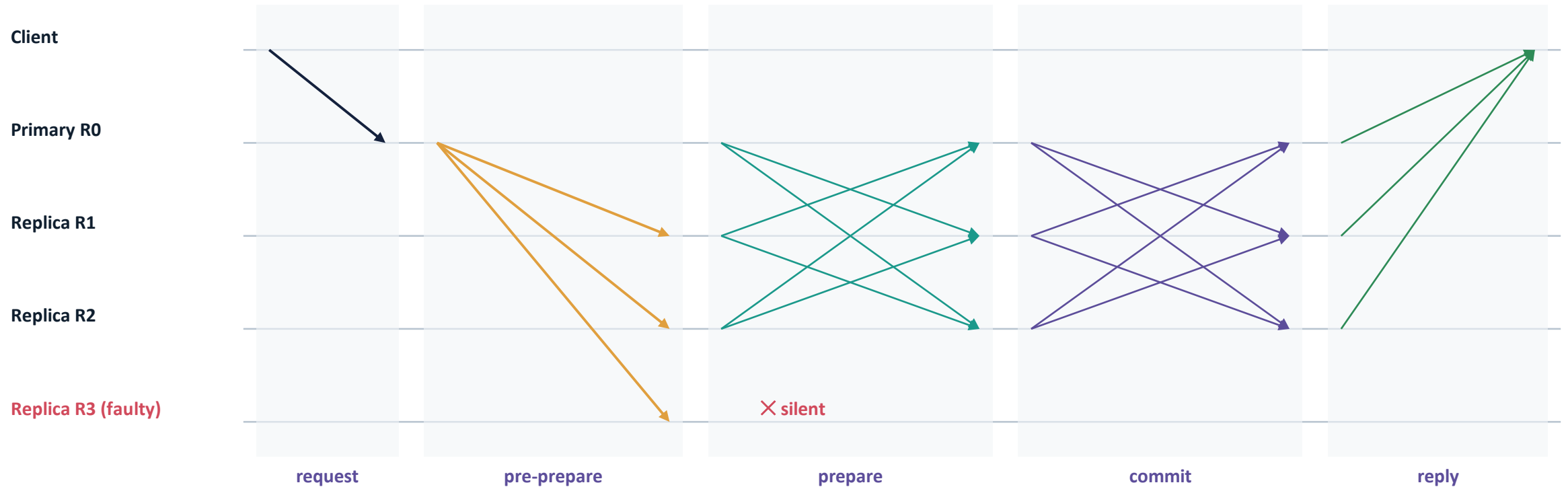
PoW vs PoS — a rigorous comparison

Both work; they fail differently

Criterion	Proof of Work (Bitcoin)	Proof of Stake (Ethereum)
Finality	Probabilistic, deepens with confirmations	Economic finality after ~2 epochs
Block time	~10 min (variable, Poisson)	12 s fixed slots
Sybil resistance	Physical: energy and ASICs	Financial: bonded stake
Recovery from 51%	Change PoW algorithm (bricks ASICs)	Social slashing / inactivity leak
Bootstrapping	Verify from genesis with no trust	Weak subjectivity checkpoint needed
Centralisation pressure	Cheap electricity, ASIC makers, pools	Liquid staking providers, exchanges
Environmental cost	High	Low
Complexity	Simple, battle-tested since 2009	Complex; live since 2020 / Merge 2022

Classical BFT consensus: PBFT

Castro & Liskov (1999). $n = 3f + 1$ replicas tolerate f Byzantine; quorum = $2f + 1$



KEY INSIGHT

Deterministic, immediate finality — but $O(n^2)$ messages per decision limits PBFT to tens of known replicas. Client waits for $f + 1$ matching replies.

Modern BFT: Tendermint and HotStuff

BFT made practical for proof-of-stake validator sets

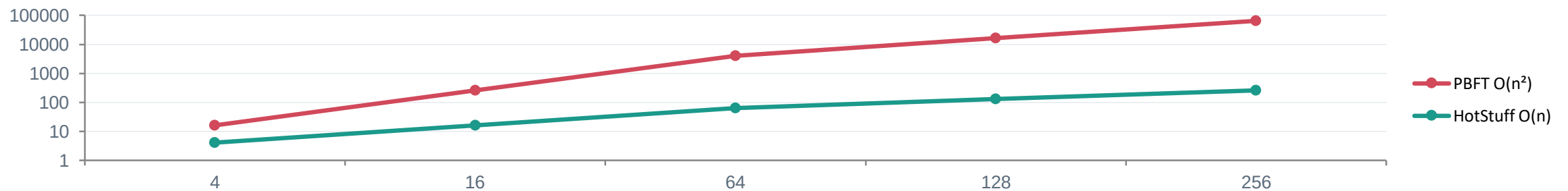
Tendermint / CometBFT (Cosmos)

- Rounds of propose → prevote → precommit
- Block committed with $> \frac{2}{3}$ precommits: **instant finality**
- Locking rules guarantee safety across rounds
- Prioritises safety: halts rather than forks if $> \frac{1}{3}$ offline

HotStuff (and descendants)

- Leader collects votes into a **quorum certificate (QC)**
- Linear $O(n)$ communication via threshold signatures
- Three-chain commit rule; pipelined (chained) phases
- Influenced Diem/Aptos (Jolteon), and many PoS designs

Messages per consensus decision vs number of validators (log scale)



The wider consensus zoo

Different trust models, different trade-offs

D

Delegated PoS

Token holders elect a small set of producers (e.g. EOS). Fast; more centralised.

A

Proof of Authority

Known, reputable validators sign blocks. Common in consortia and testnets.

H

Proof of History

Solana: a verifiable delay (sequential SHA-256) timestamps events; consensus is PoS (Tower BFT).

T

Proof of Elapsed Time

Trusted hardware (Intel SGX) enforces random waits. Hyperledger Sawtooth.

R

Raft / crash-fault

Leader-based, tolerates crashes not lies. Fabric's ordering service.

G

DAG-based

Blocks reference many parents (Narwhal/Bullshark, Avalanche-style sampling) for throughput.

KEY INSIGHT

Always ask: who can join, who can vote, what failures are tolerated, and how final is final?

Consensus comparison matrix

Summary of families discussed (typical characteristics, not guarantees)

Family	Membership	Fault model	Tolerance	Finality	Scale of validators
Nakamoto PoW	Open	Byzantine (rational)	< 50% hashrate*	Probabilistic	Unbounded
Longest-chain PoS	Open (stake)	Byzantine	< 50% stake*	Probabilistic	Large
Gasper (Ethereum)	Open (stake)	Byzantine	< ⅓ safety / liveness	Economic (~13 min)	Very large
PBFT	Permissioned	Byzantine	$f < n/3$	Immediate	Tens
Tendermint / HotStuff	Permissioned or PoS	Byzantine	$f < n/3$	Immediate	Hundreds
Raft / Paxos	Permissioned	Crash only	$f < n/2$	Immediate	Few

* Selfish mining lowers the practical threshold (see earlier slide).

PoS-specific attacks and defences

Costless simulation creates problems PoW does not have

1 Nothing-at-stake

Voting on every fork is free. Defence: slashing for equivocation.

2 Long-range attack

Old keys (already withdrawn) rebuild history from far back. Defence: weak-subjectivity checkpoints.

3 Stake grinding

Manipulating randomness to be selected more often. Defence: RANDAO + VDF research, look-ahead limits.

4 Cartelisation

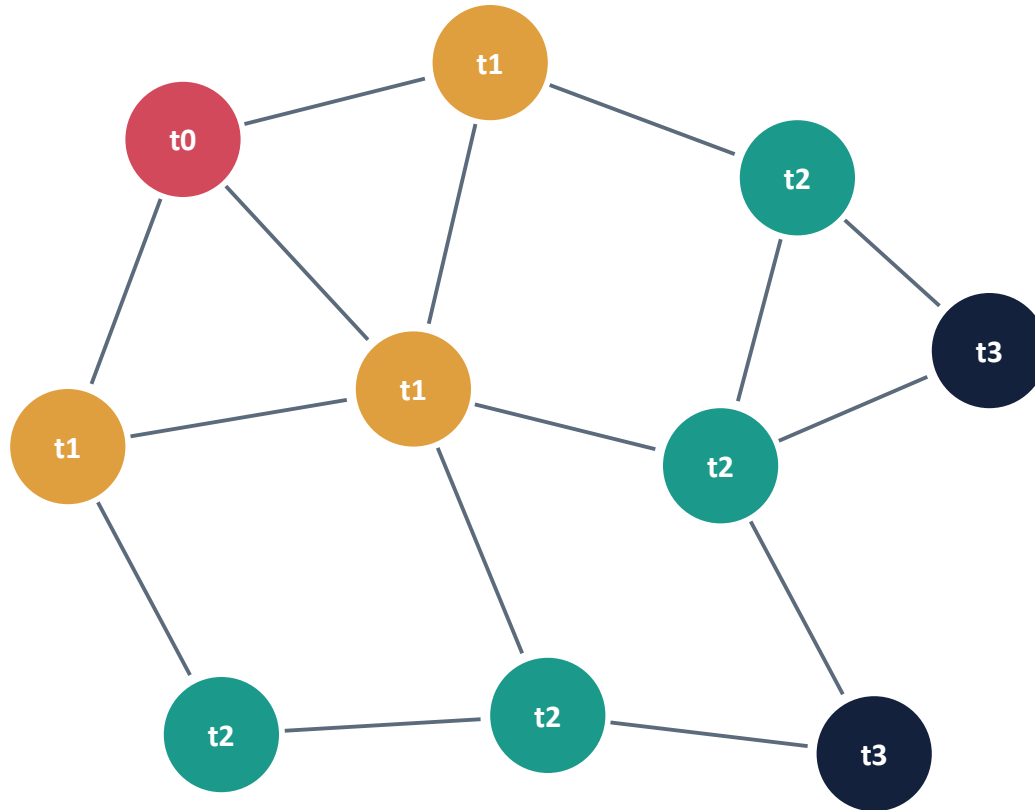
Large staking pools approach $\frac{1}{3}$ or $\frac{2}{3}$. Defence: social pressure, client diversity, distributed validators (DVT).

Weak subjectivity, explained

A new node cannot tell the true chain from a long-range fake using protocol rules alone; it must obtain a recent checkpoint from a trusted source (client release, friends, explorers). PoW needs no such input — a genuine philosophical difference.

The peer-to-peer network: gossip

No central server: each node talks to a handful of peers and relays what it learns



Colour = hop count from origin (red, t0)

Most nodes know within a few hops: gossip reaches N peers in $\sim O(\log N)$ rounds.

- **Discovery:** DNS seeds, hard-coded peers, address gossip; Ethereum uses discv5 (Kademlia-style DHT).
- **Messages:** inv / getdata / tx / block (Bitcoin); devp2p & libp2p gossipsub (Ethereum).
- **Default peers:** Bitcoin Core makes 8 full-relay outbound connections (plus block-relay-only ones).
- **Validation before relay** stops spam from propagating.

Node types

Trade storage and bandwidth for independence of verification

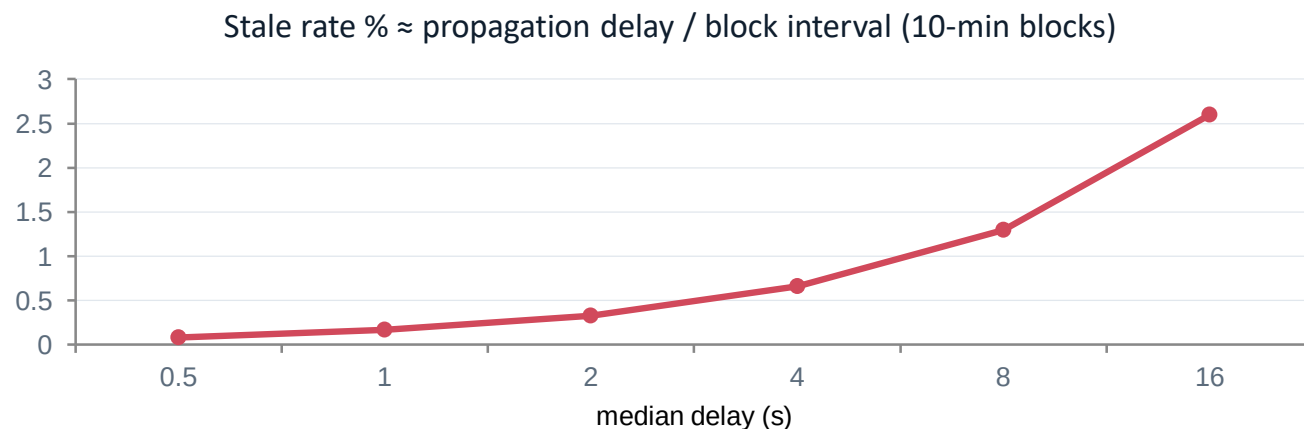
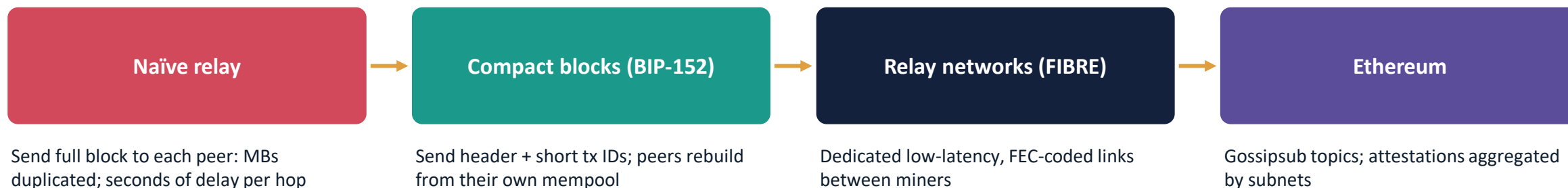
Type	Stores	Verifies	Trust needed	Typical user
Archive node	Full chain + every historical state	Everything	None	Explorers, analytics, indexers
Full node	Full chain (can prune old blocks) + current state	Every rule of every block	None	Exchanges, sovereign users
Light / SPV client	Block headers only	PoW (or sync-committee sigs) + Merkle proofs	Honest-majority of producers	Mobile wallets
Validator / miner	Full node + signing keys	Everything; also produces blocks	None	Block producers

KEY INSIGHT

‘Don't trust, verify’ is only true for people who run a full node. Everyone else trusts someone who does.

Block propagation and why it matters

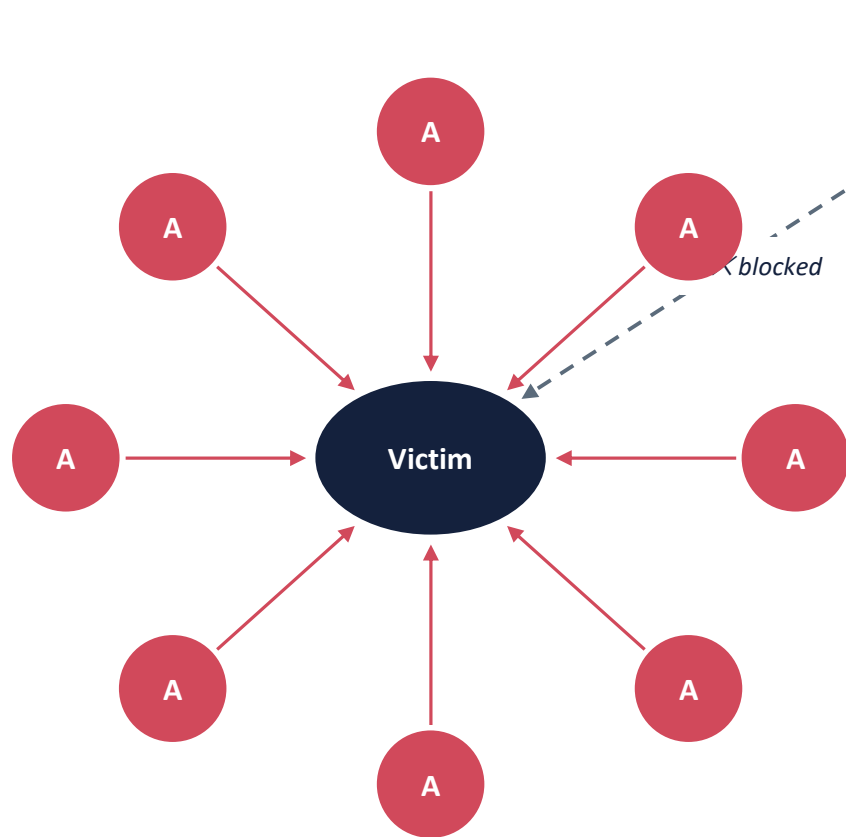
Slow propagation → more stale blocks → advantage to large, well-connected miners



Model: stale probability $\approx 1 - e^{(-d/600)}$ for delay d seconds.
Faster chains (12 s slots) are far more sensitive to latency — one reason block size cannot simply grow.

Network-layer attacks: Sybil and Eclipse

Consensus assumes you see the honest chain — the network can deny you that



- **Sybil**: attacker spins up many cheap identities to gain influence. PoW/PoS make *voting* Sybil-resistant, but peer slots are not.
- **Eclipse** (Heilman et al., 2015): monopolise all of a victim's connections → feed it a fake view, enable double spends, waste its hashpower.
- **Defences**: diverse outbound peers across network groups (ASNs), anchor connections, bucketed address tables, block-relay-only links, Tor/I2P, checking header chain work.

Smart Contracts & Ethereum

Turning a ledger of balances into a shared, programmable computer

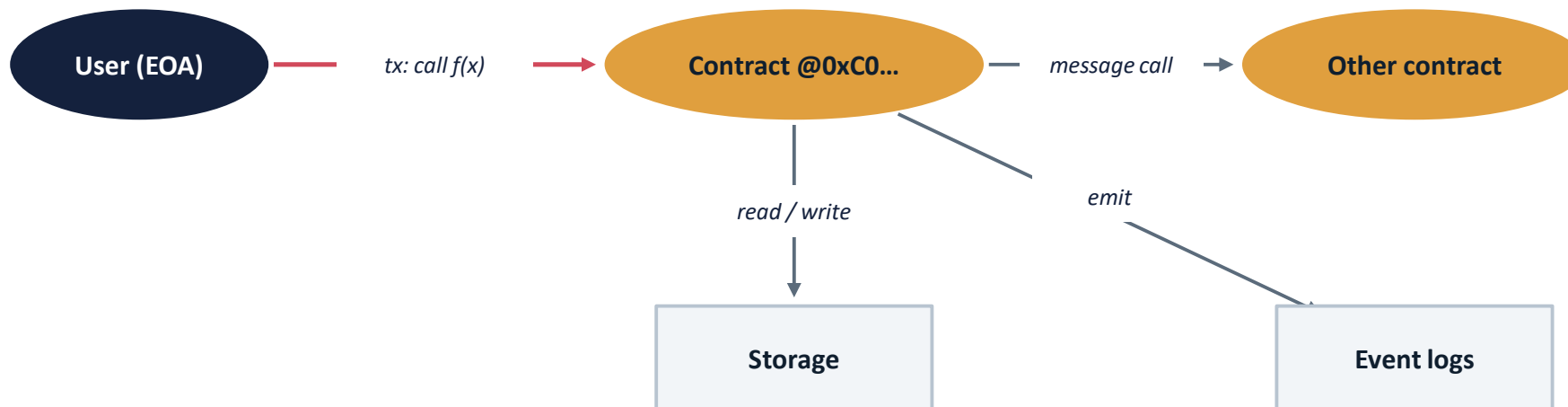
▸ Contracts ▸ EVM ▸ Gas & EIP-1559 ▸ Solidity ▸ Tokens ▸ Oracles ▸ Vulnerabilities ▸ Secure development ▸ State tries



What is a smart contract?

Code deployed at an address, executed identically by every node, whose state lives on-chain

"A set of promises, specified in digital form, including protocols within which the parties perform on these promises." — Nick Szabo, 1996



- **Deterministic**
- **Transparent** bytecode
- **Immutable** by default
- **Composable** (money legos)
- **Not** smart, not legal contracts

KEY INSIGHT

Every full node re-executes every contract call. That is what makes results trustworthy — and computation expensive.

The Ethereum Virtual Machine (EVM)

A quasi-Turing-complete, 256-bit stack machine with four kinds of data location

Execution context of one call

Stack

1,024 × 256-bit words; opcodes PUSH, ADD, SSTORE...

Memory

Byte array, zeroed per call; cost grows quadratically

Calldata

Read-only input bytes (function selector + args)

Program counter & gas

Remaining gas decremented per opcode

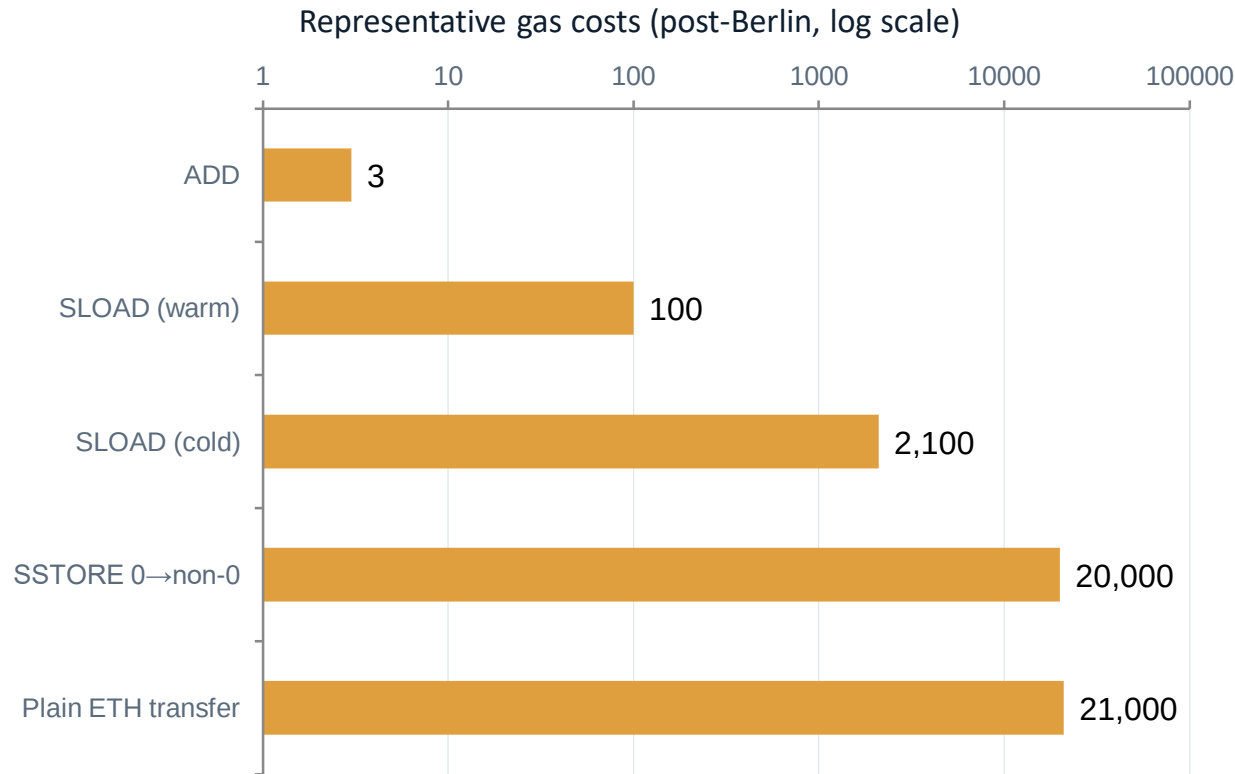
Storage (persistent)

256-bit key → 256-bit value, per contract; the only data that survives the tx

- **Bytecode** compiled from Solidity/Vyper; ~150 opcodes
- **Account types:** EOAs and contracts share one address space
- **Quasi-Turing-complete:** loops allowed, but gas bounds every execution → the halting problem becomes a budget problem
- **Precompiles** for ECRECOVER, SHA-256, pairings, etc.
- **EVM-compatible chains:** most L2s and many L1s reuse it

Gas: pricing computation

Every opcode has a gas cost; the sender pays gas used × price



- **Gas limit** (per tx): max you allow; unused gas is refunded
- **Out of gas**: state changes revert, but gas spent is kept → pays for wasted work
- **Block gas limit**: caps total computation per block; set by validators within bounds
- **Why storage costs most**: every node must keep it forever

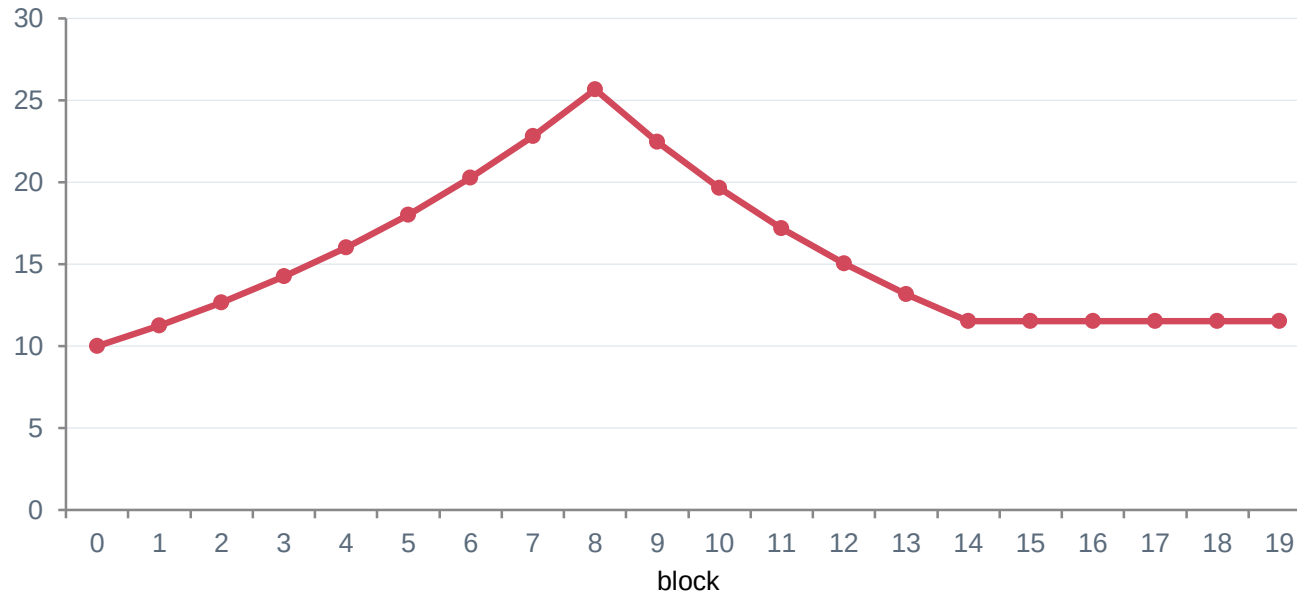
KEY INSIGHT

Gas separates the cost of computation (gas units, fixed by protocol) from its price (gwei, set by the market).

EIP-1559: the fee market redesign (London, August 2021)

Base fee (burned) adjusts algorithmically; a priority tip goes to the proposer

Base fee: 8 full blocks (+12.5% each), 6 empty (−12.5%), then on-target — computed



$$\text{base_fee}' = \text{base_fee} \times (1 + \frac{1}{8} \times (\text{gas_used} - \text{target}) / \text{target})$$

- **Target** = half the block gas limit; blocks can stretch to 2× target
- **Fee paid** = gas × (base fee + tip), capped by maxFeePerGas
- **Burn** removes base fee from supply → ETH can be net deflationary under high usage
- **UX win**: predictable fees; wallets no longer guess blindly

Solidity by example: a provenance registry

A contract we will reuse in Part III to anchor knowledge-graph snapshots

ProvenanceRegistry.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

contract ProvenanceRegistry {
    struct Anchor { bytes32 root; uint64 time; address by: }
    mapping(bytes32 => Anchor) public anchors; // graphId => anchor
    event Anchored(bytes32 indexed graphId, bytes32 root);

    function anchor(bytes32 graphId, bytes32 root) external {
        require(anchors[graphId].time == 0, "already anchored");
        anchors[graphId] = Anchor(root, uint64(block.timestamp),
msg.sender);
        emit Anchored(graphId, root);
    }

    function verify(bytes32 graphId, bytes32 root) external view returns
(bool) {
        return anchors[graphId].root == root;
    }
}
```

mapping = persistent storage (key → value); **public** auto-generates a getter

Events are cheap logs for off-chain indexers — not readable by contracts

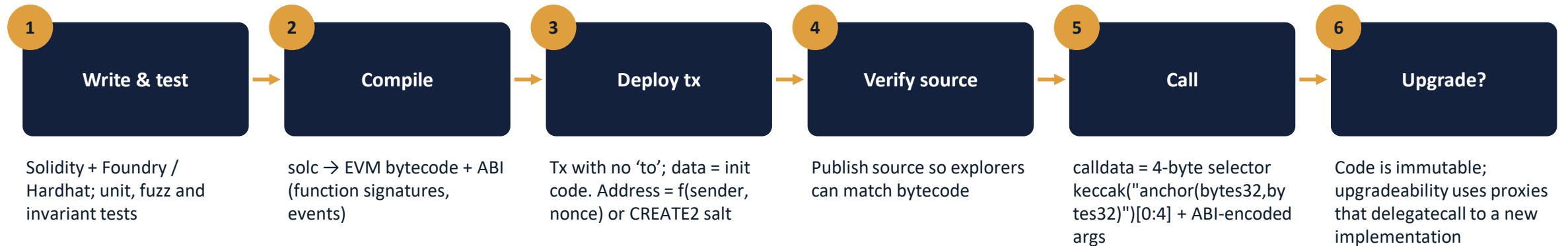
require reverts all changes if false (write-once semantics)

msg.sender = authenticated caller; **block.timestamp** set by proposer (±seconds)

view functions read state; free when called off-chain

Contract lifecycle: from source to call

What happens between writing Solidity and a user's transaction executing it



TRUST QUESTION

Proxy upgradeability reintroduces a trusted admin key. Always ask of any contract: who can change the code, and with what delay?

Token standards: fungible, non-fungible, multi

Interfaces that let wallets and exchanges handle any token without custom code

ERC-20

Fungible

```
balanceOf, transfer, approve,  
transferFrom, allowance
```

Stablecoins, governance & utility tokens

ERC-721

Non-fungible

```
ownerOf(tokenId), safeTransferFrom,  
tokenURI metadata
```

Art, tickets, domain names, deeds

ERC-1155

Multi-token

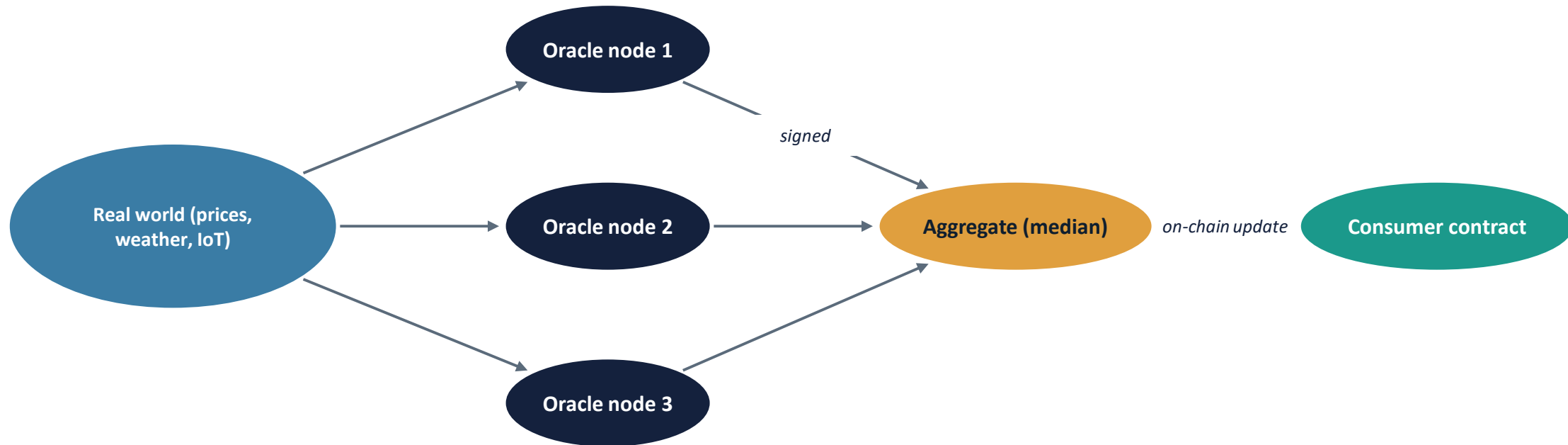
```
balanceOf(owner, id); batch  
transfers
```

Game items, editions, mixed inventories

A token is not a coin moving between wallets — it is an entry in a contract's mapping(address $\Rightarrow$ uint256). 'Transferring' updates two numbers in that contract's storage.

The oracle problem

Contracts are sealed from the outside world — how do real-world facts get in?



- **Trust shifts** from the chain to oracle operators — ‘garbage in, garbage forever’.
- **Mitigations:** many independent nodes, medians, staking/slashing, TWAPs, TEEs and TLS proofs.
- **Attack:** manipulating a thin on-chain market used as a price oracle (flash-loan attacks).

Reentrancy: the vulnerability behind The DAO (2016)

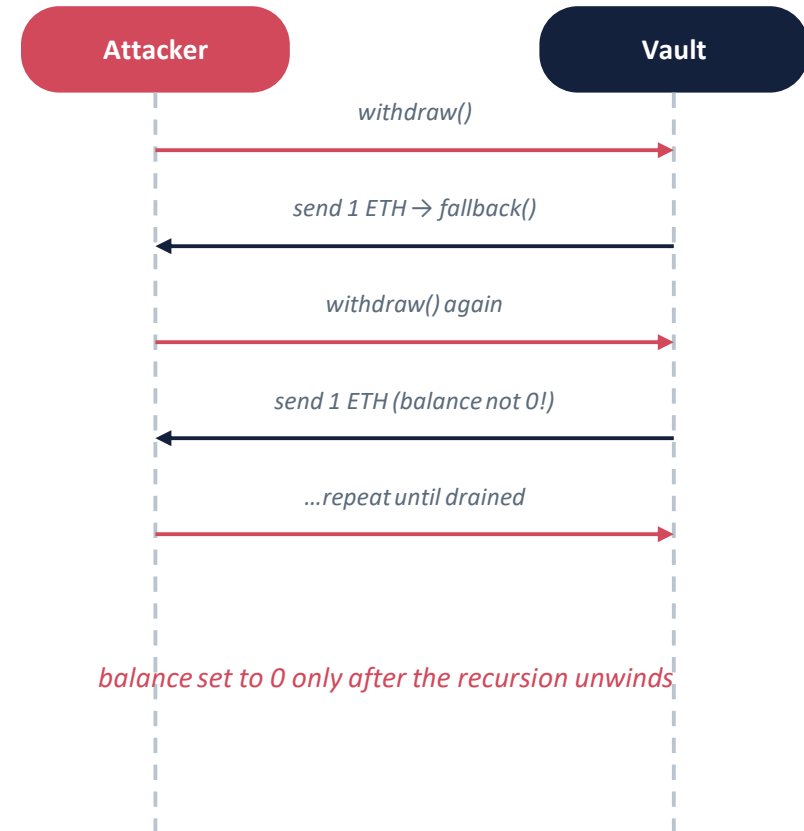
External call before state update lets the callee call back in and repeat the withdrawal

Vulnerable.sol

```
function withdraw() external {
  uint amt = balances[msg.sender];
  (bool ok,) = msg.sender.call{value: amt}(""); // ① call
  require(ok);
  balances[msg.sender] = 0;           // ② too late!
}
```

Fixed.sol – checks-effects-interactions + guard

```
function withdraw() external nonReentrant {
  uint amt = balances[msg.sender];
  balances[msg.sender] = 0;           // effects first
  (bool ok,) = msg.sender.call{value: amt}(""); // then call
  require(ok);
}
```



KEY INSIGHT

The DAO lost ~3.6M ETH; the community's rescue hard fork split Ethereum (ETH) from Ethereum Classic (ETC).

Smart-contract vulnerability catalogue

Frequent classes, with mechanism and primary defence

Class	Mechanism	Primary defence
Reentrancy	Callee re-enters before state update	Checks-effects-interactions, reentrancy guards
Access control	Missing onlyOwner / initialiser left open	Role libraries, audits of every external function
Oracle manipulation	Spot price from thin pool, moved by flash loan	TWAP / decentralised oracles, sanity bounds
Front-running / MEV	Mempool visibility lets others reorder txs	Commit–reveal, slippage limits, private order flow
Integer issues	Overflow/underflow (pre-0.8), precision loss	Solidity ≥ 0.8 checked math; careful rounding
Unchecked calls	Ignoring return value of low-level call	Check <code>ok</code> ; use safe-transfer libraries
Signature replay	Signed message reused on other chain/contract	EIP-712 domain separator, nonces, chain ID
Upgrade / proxy bugs	Storage collisions, unprotected upgrade	Standard proxy patterns, timelocks, multisig

Secure development lifecycle for contracts

Defence in depth — because deployed bugs are public, permanent and directly monetisable

Specification & threat model — Invariants written first: 'total supply = Σ balances'

Well-audited libraries — OpenZeppelin, Solmate — don't hand-roll tokens or access control

Testing — Unit, fuzz (Foundry), invariant & fork tests against mainnet state

Static analysis & formal verification — Slither, Mythril, SMTChecker, Certora / K framework

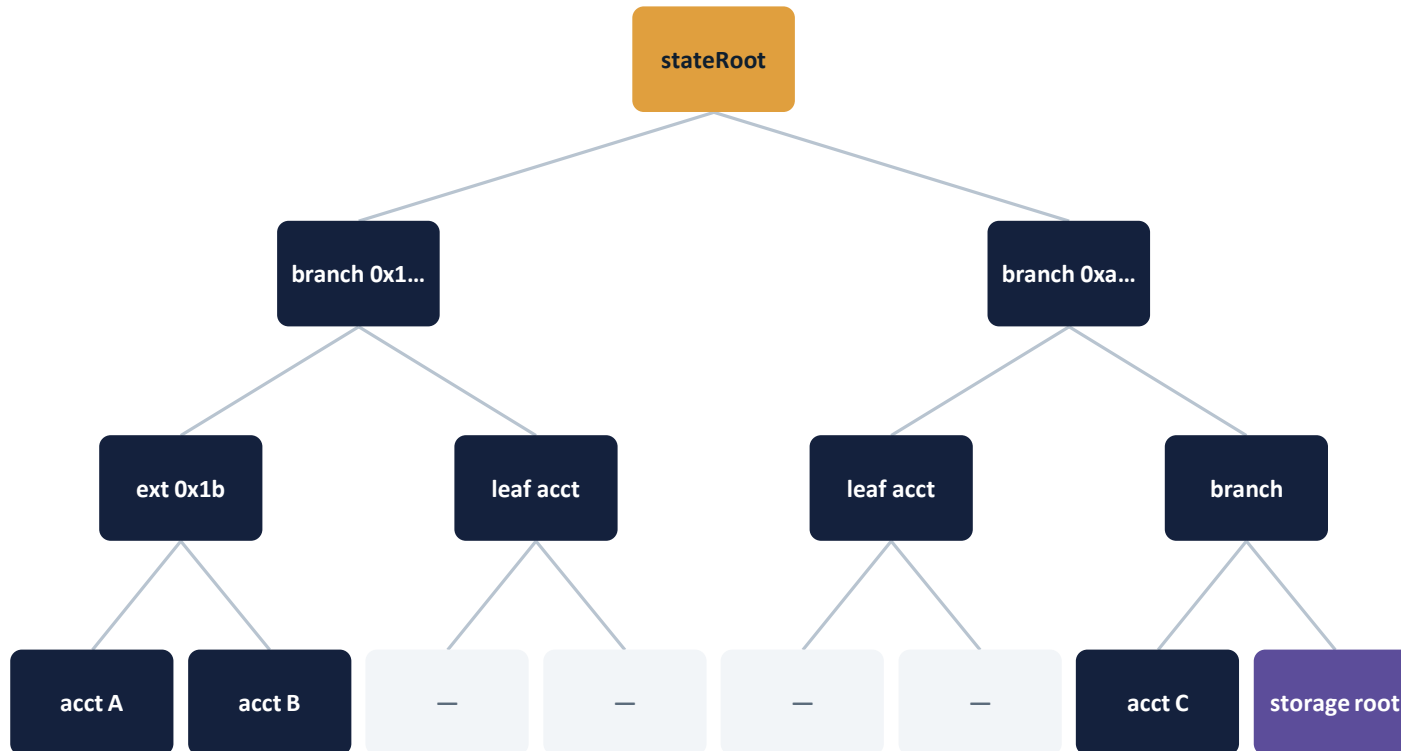
Independent audits & bug bounties — Multiple auditors; public bounty programmes

Operational controls — Timelocks, multisig admins, pausable circuits, monitoring & incident response

Each layer catches what the previous missed; none is sufficient alone.

Ethereum state: the Merkle Patricia Trie

Every block header commits to the ENTIRE world state with one 32-byte stateRoot



Simplified: real nodes are 16-way branches, extensions and leaves.

- **Patricia**: path-compressed radix trie keyed by keccak(address)
- **Merkle**: each node referenced by the hash of its RLP encoding
- **Four tries per block**: state, per-account storage, transactions, receipts
- **Proofs**: prove a balance or storage slot against a header — used by light clients and bridges
- **Future**: Verkle trees / binary tries to shrink witness size for stateless clients

Scalability & Interoperability

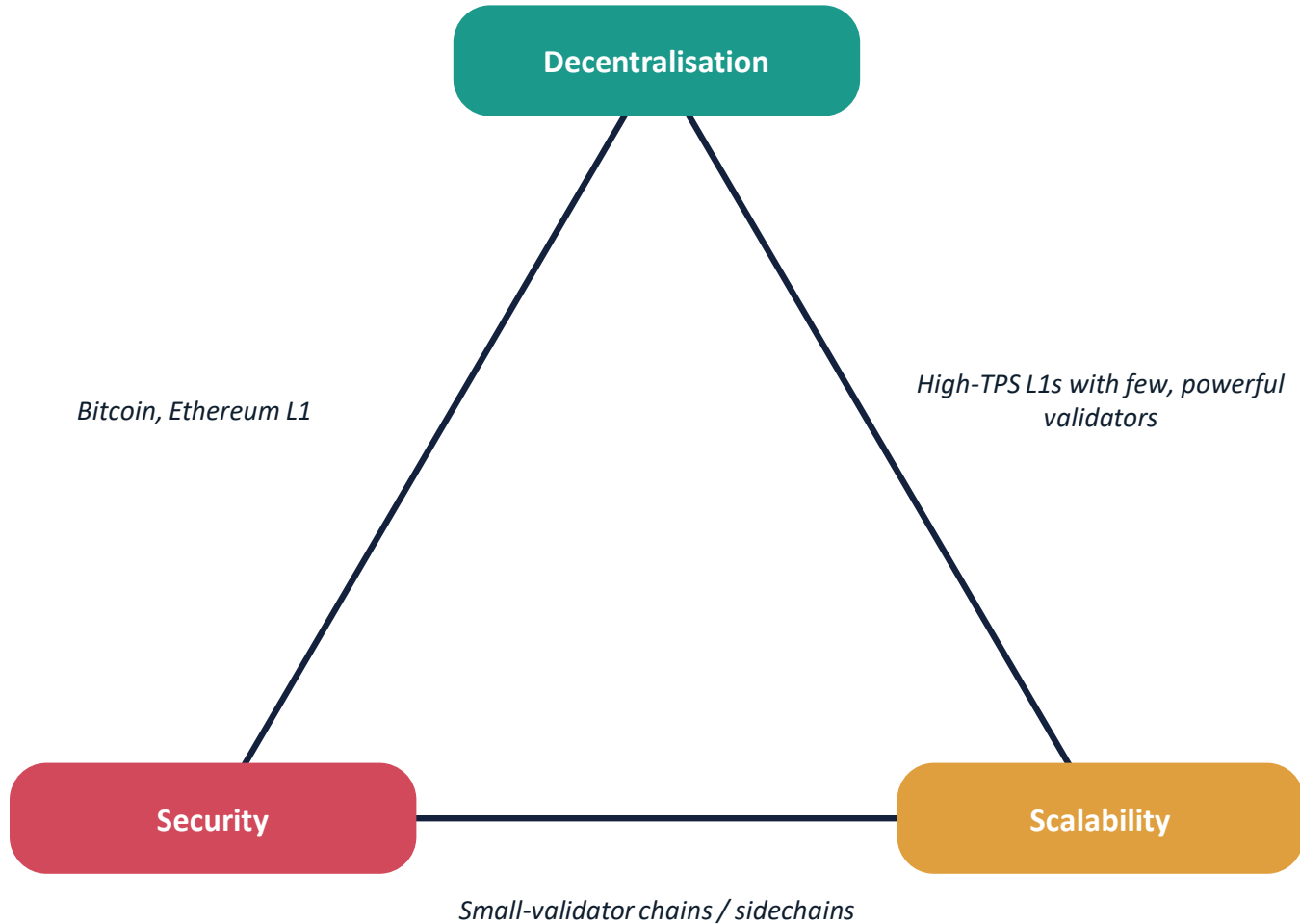
Doing more transactions without giving up what made the chain worth using

▸ Trilemma ▸ Throughput limits ▸ Layer 2 ▸ Channels ▸ Optimistic rollups ▸ ZK proofs & rollups ▸ Data availability ▸ Bridges ▸ Atomic swaps



The blockchain trilemma

A heuristic (Buterin): simple designs get at most two of three properties



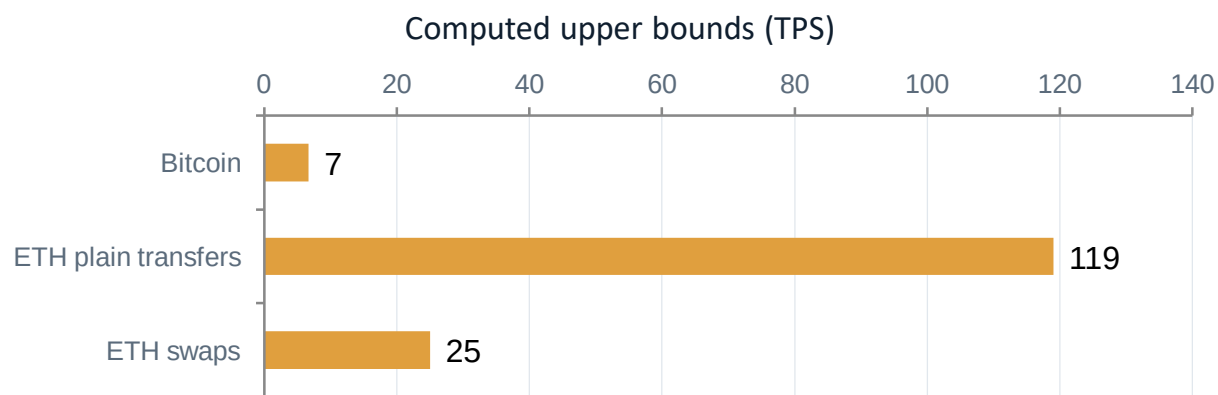
- **Decentralisation:** can ordinary hardware verify the chain?
- **Security:** cost to attack relative to value secured
- **Scalability:** throughput and cost per transaction
- **Escape routes:** move execution off-chain but keep verification on-chain (rollups), or split verification (sharding, data-availability sampling).

Where the throughput ceiling comes from

A back-of-envelope derivation you can reproduce

$$\text{TPS} \approx (\text{block capacity} \div \text{average tx size}) \div \text{block interval}$$

Chain	Capacity	Tx size	Interval	Upper bound
Bitcoin (pre-SegWit)	1,000,000 B	~250 B	600 s	≈ 6.7 TPS
Ethereum (30M gas limit)	30,000,000 gas	21,000 gas (plain ETH)	12 s	≈ 119 TPS
Ethereum, mixed workload	30,000,000 gas	~100k gas (swap)	12 s	≈ 25 TPS



Raising capacity or shortening the interval increases bandwidth, storage and propagation delay for every node — directly eroding decentralisation. Gas limits have been raised since; the method still applies.

Layered architecture: L1 and L2

Settle on a secure base layer; execute somewhere cheaper

Applications

Wallets, DeFi, games, identity, provenance

Layer 2 execution

Rollups · state channels · validiums

Data availability

L1 calldata & blobs · external DA layers

Layer 1 settlement & consensus

Ethereum / Bitcoin: ordering, finality, dispute resolution

cheaper, faster, more trust assumptions

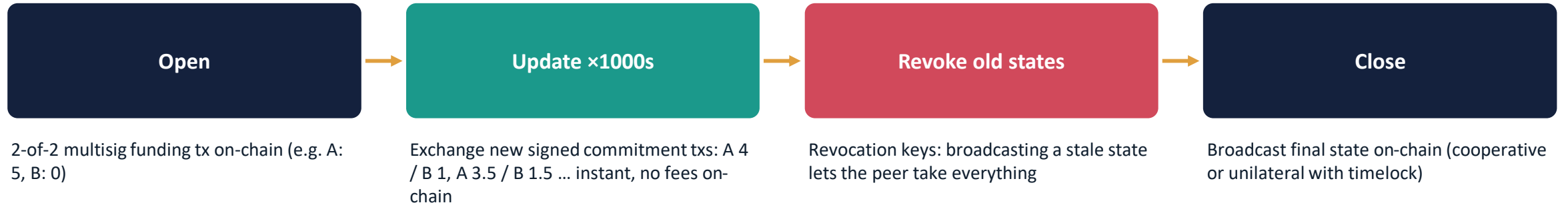
security & finality inherited downward

KEY INSIGHT

An L2 is a real L2 only if users can exit to L1 using L1 data alone, even if the L2 operator disappears.

Payment channels & the Lightning Network

Lock funds once on-chain, then exchange signed balance updates off-chain



Limits: liquidity must be pre-locked, receivers must be online (or use watchtowers).

Optimistic rollups

Assume batches are valid; allow anyone to prove fraud during a challenge window



- **Pros:** EVM-equivalent; simple proofs; cheap
- **Cons:** 7-day withdrawal to L1 (fast bridges add liquidity trust); centralised sequencers today
- **Examples:** Arbitrum, Optimism / OP Stack, Base

Fraud-proof path

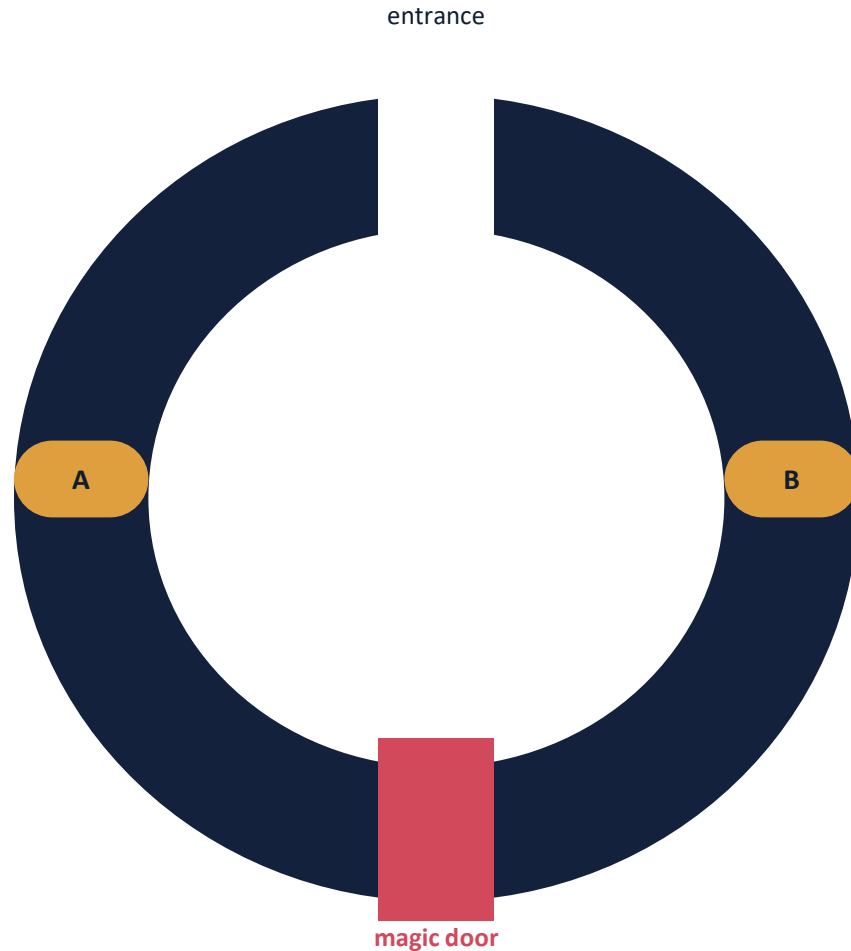
1. Verifier re-executes, finds a wrong state root
2. Interactive bisection narrows dispute to ONE instruction
3. L1 executes that step; loser's bond is slashed
4. Only one honest verifier needed (1-of-N trust)

KEY INSIGHT

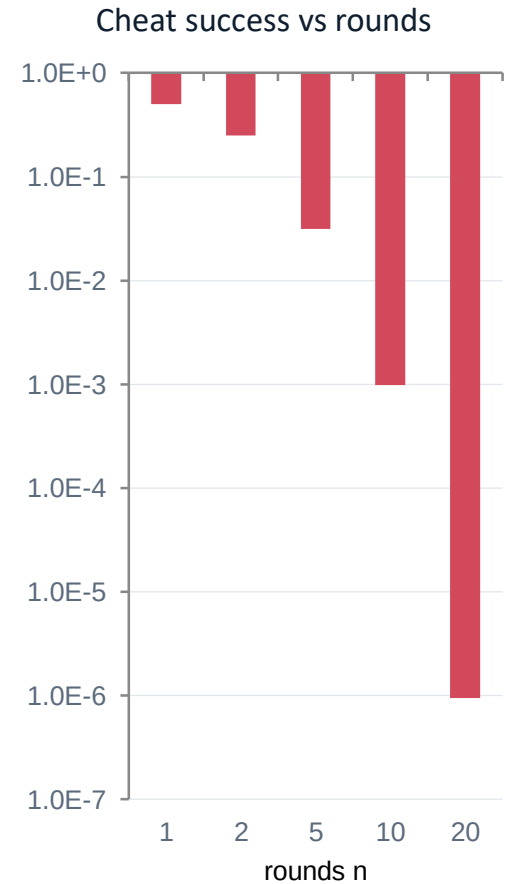
Security rests on data availability: if batch data were withheld, nobody could construct a fraud proof.

Zero-knowledge proofs: the intuition

Prove you know a secret without revealing anything else — the Ali Baba cave (Quisquater et al.)



- **Round:** Peggy enters a random path. Victor, at the entrance, shouts A or B.
- **If Peggy knows the password** she always emerges from the requested side.
- **If she doesn't**, she succeeds only with probability $\frac{1}{2}$.
- **Soundness** after n rounds: cheating succeeds with 2^{-n} .
- **Zero knowledge:** Victor learns nothing reusable — a video of the rounds could be faked.



ZK-SNARKs vs ZK-STARKs

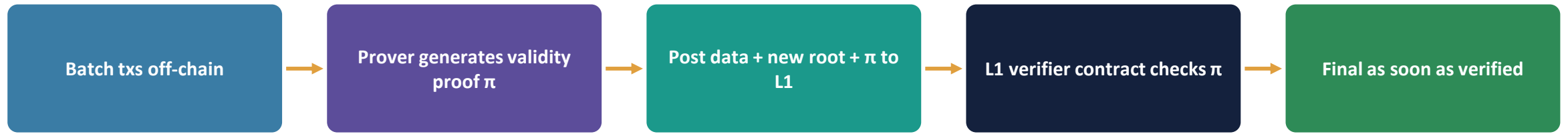
Succinct proofs that a computation was executed correctly

Property	SNARK (e.g. Groth16, PLONK)	STARK
Proof size	Tiny: ~200 B – 1 KB	Larger: ~tens to hundreds of KB
Verification	Milliseconds, constant	Fast, polylogarithmic
Trusted setup	Groth16: per-circuit; PLONK: universal	None (transparent)
Crypto assumptions	Elliptic-curve pairings	Collision-resistant hashes only
Post-quantum	No	Plausibly yes
Used by	Zcash, many zkEVMs, zkSync	Starknet, many zkVMs (FRI-based)

- **Completeness:** honest prover convinces verifier
- **Soundness:** false statements fail (except negligible probability)
- **Zero-knowledge:** reveals nothing beyond validity
- **Succinctness:** proof & verification $\ll$ computation

ZK rollups (validity rollups)

Every batch ships with a proof that the new state root is correct



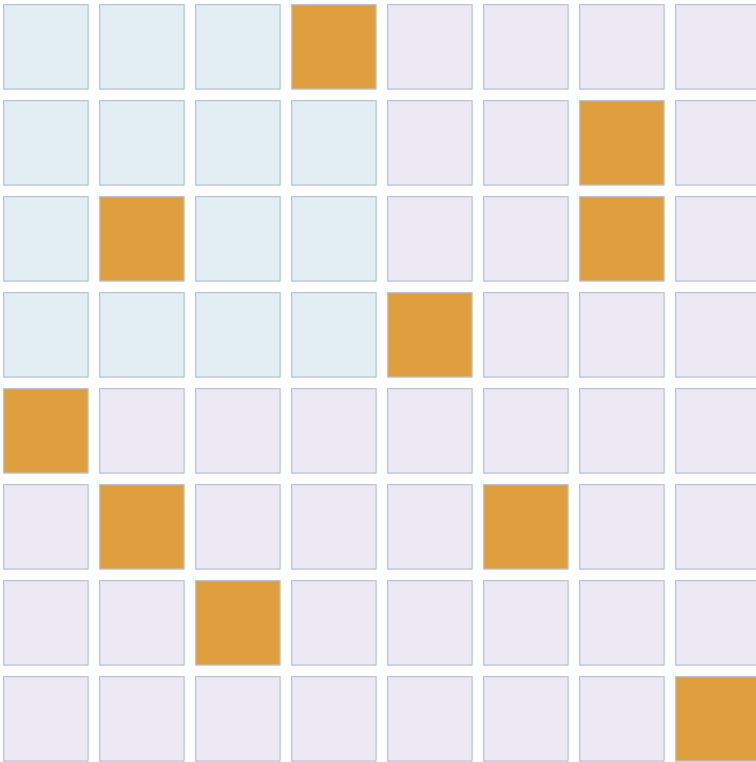
	Optimistic rollup	ZK rollup
Validity assumption	Innocent until proven guilty	Proven correct up front
Withdrawal to L1	~7 days	Minutes–hours (proof time)
Prover cost	Low	High (specialised hardware helps)
EVM compatibility	Easy	Hard; zkEVMs now mature rapidly
Trust model	1 honest verifier online	Mathematics + setup assumptions

KEY INSIGHT

Validiums/volitions keep data off-chain for lower cost — trading away L1 data availability.

Data availability and blobs (EIP-4844, March 2024)

Rollups need data to be published, not stored forever — so give them cheaper, temporary space



*Original data (top-left quadrant) is erasure-coded 2× in each dimension;
light nodes sample random cells (gold).*

- **Blob-carrying transactions** (proto-danksharding): ~128 KB blobs, separate fee market, pruned after ~18 days
- **KZG commitments** let anyone check a blob chunk against the header
- **Data-availability sampling**: if $\geq 50\%$ of the extended data is available, the whole can be reconstructed; random samples make hiding it overwhelmingly likely to be detected
- **Roadmap**: full danksharding scales blob count with sampling (PeerDAS)
- **Effect**: rollup fees dropped sharply after March 2024

Bridges: moving value between chains

Chain A cannot see chain B — someone or something must attest to events on the other side



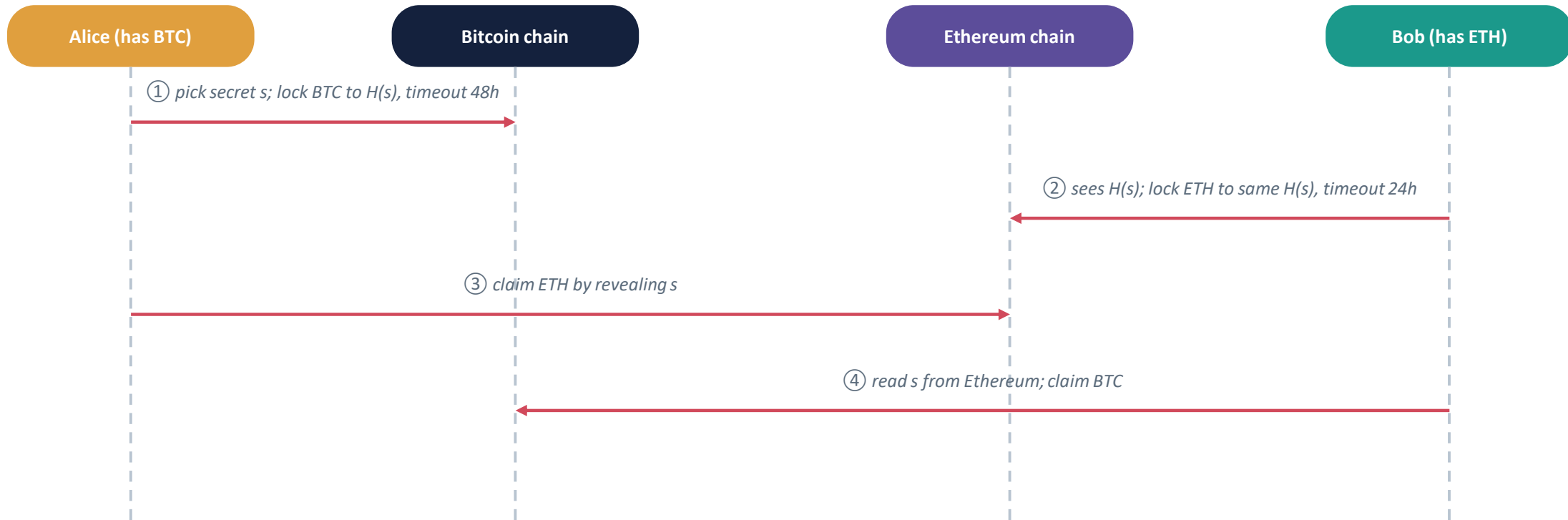
Verification model	Trust assumption	Example risk
External multisig / committee	k-of-n signers honest	Ronin (2022): 5 of 9 keys compromised, ≈\$600M+ lost
Light-client / ZK proof	Source chain consensus + math	Implementation bugs
Optimistic	1 honest watcher within window	Liveness of watchers
Native (rollup ↔ L1)	L1 itself	Slow exits

RISK

Bridges concentrate value behind the weakest verification in the path; they have been among the largest sources of losses in the industry.

Atomic swaps with hash time-locked contracts (HTLC)

Trade across chains with no intermediary: either both legs happen or neither does



Bob's timeout is shorter so Alice cannot wait until his refund passes and then claim both. If anyone stops, timeouts refund both parties.

The same hash-lock primitive powers Lightning multi-hop payments.

Permissioned & Enterprise Blockchains

When participants are known but still do not fully trust each other

▸ Public vs private vs consortium ▸ Hyperledger Fabric ▸ Execute-order-validate ▸ Do you need a blockchain?



Public, private and consortium chains

Two independent axes: who can READ, and who can WRITE / validate

Public permissionless

Anyone reads, anyone validates
e.g. Bitcoin, Ethereum

Public permissioned

Anyone reads, approved validators write
e.g. Some government registries, PoA networks

Private

One organisation controls access
e.g. Internal audit ledgers

Consortium

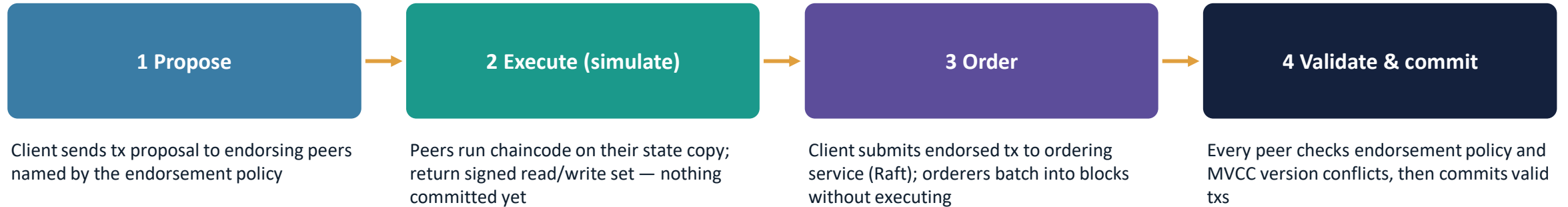
Members read and validate
e.g. Trade finance, supply-chain networks

	Permissionless	Permissioned
Identity	Pseudonymous keys	Known, certified (PKI)
Consensus	PoW / PoS	BFT / Raft
Throughput	Lower	Higher
Finality	Probabilistic / economic	Immediate
Censorship resistance	High	Depends on members
Privacy	Transparent by default	Channels, private data

open write →

Hyperledger Fabric: execute → order → validate

Reverses the order-execute pattern of public chains to allow parallelism and privacy



Why execute first?

Non-deterministic code is caught (endorsements disagree) instead of forking the chain.

Why order separately?

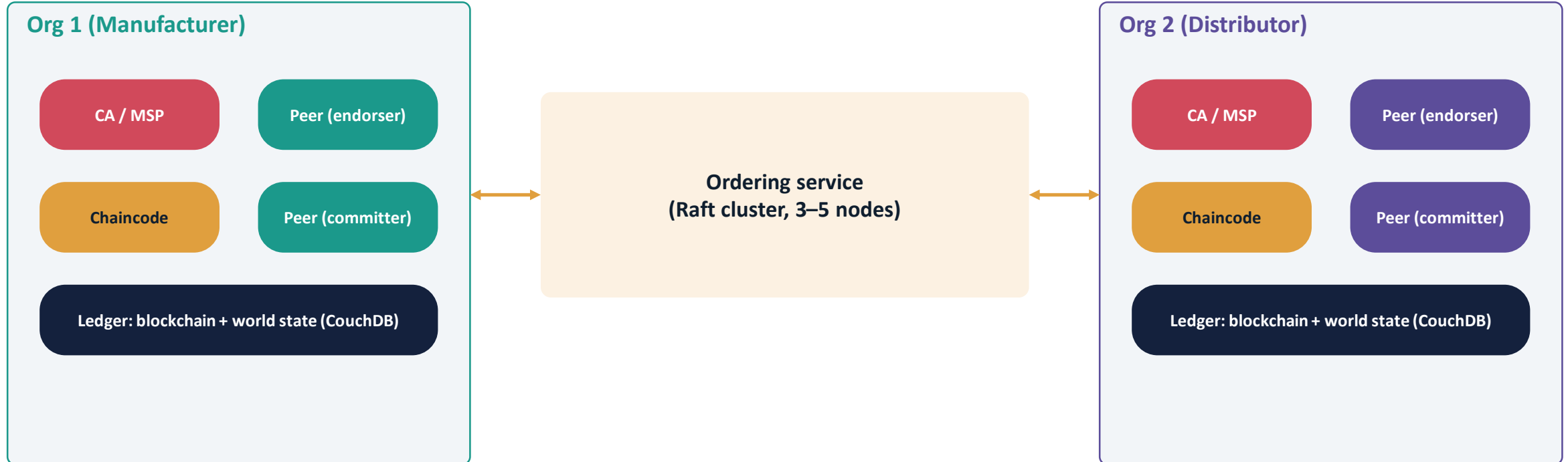
Pluggable consensus; orderers never see or run business logic.

Cost

Conflicting concurrent txs on the same key fail validation (MVCC) and must be retried.

Fabric network components

An annotated reference topology for a two-organisation consortium

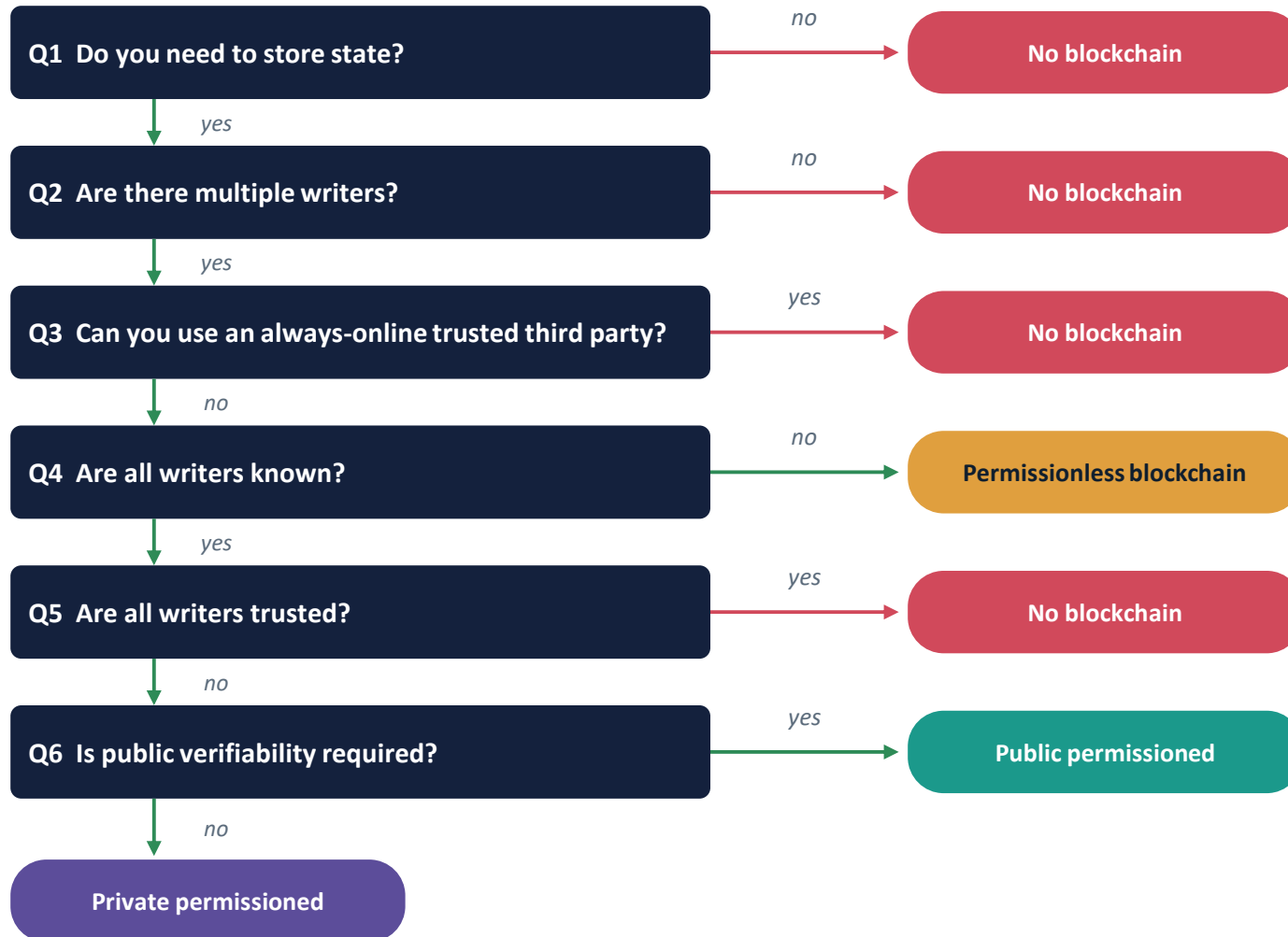


Channel 'supply-chain': a separate ledger shared only by its members; private data collections share hashes on-chain and data peer-to-peer

MSP = Membership Service Provider: maps X.509 certificates to organisations and roles.

Do you actually need a blockchain?

Adapted from Wüst & Gervais (2018), 'Do you need a Blockchain?' — read top to bottom



How to read it

- Each question is a **gate**: the first answer that exits the ladder is your answer
- Three of six exits say **no blockchain** — that is the common case
- Q3 is decisive: an always-online trusted party beats consensus on cost, speed and privacy
- Q4/Q5 separate open networks from consortia

KEY INSIGHT

Most failed enterprise pilots skipped this tree: if a trusted operator is acceptable, a signed append-only database with hash-chained audit logs is cheaper and faster.

Applications & Ecosystem

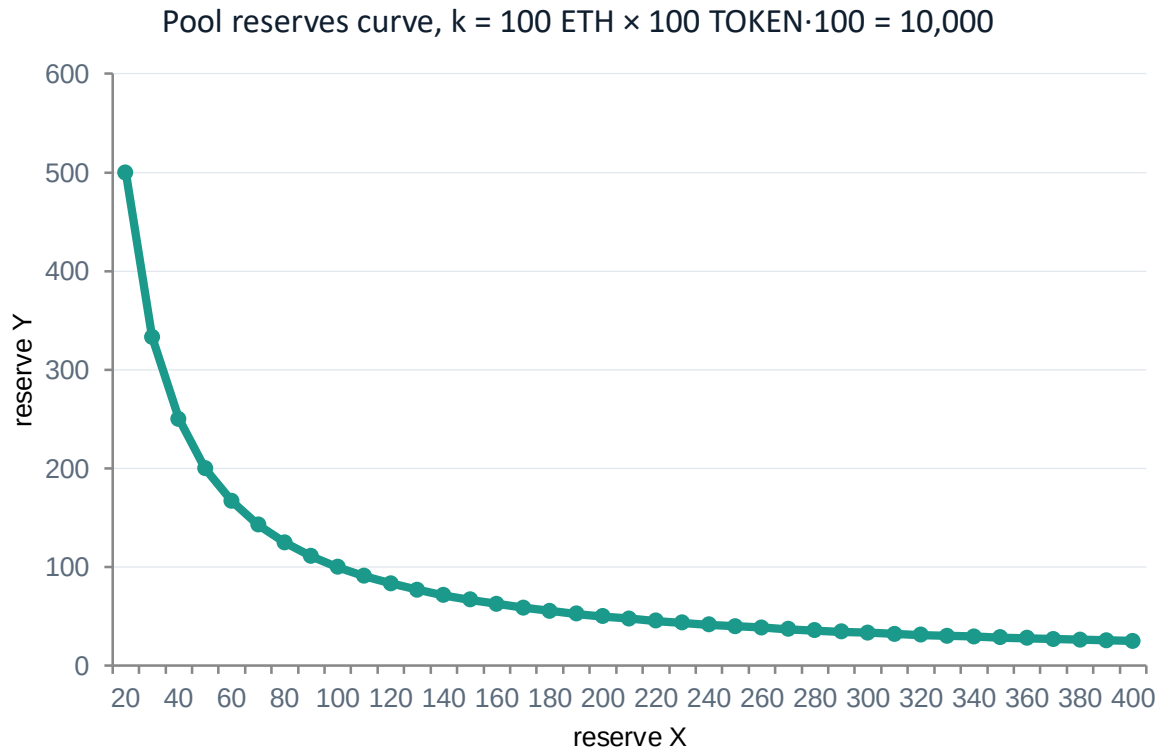
Where blockchains deliver value — and how the mechanisms work underneath

▸ DeFi & AMMs ▸ Stablecoins & lending ▸ NFTs ▸ Supply chain ▸ Decentralised identity ▸ DAOs ▸ Tokenomics ▸ Tokenisation



Automated market makers: $x \cdot y = k$

Uniswap-style constant-product pools replace order books with a pricing curve



Values computed from $\Delta y = y - k/(x + \Delta x)$, fees ignored.

Sell Δx into pool ($x=100, y=100$)	Receive Δy	Avg price	Price impact
1	0.990	0.990	1.0%
10	9.09	0.909	9.1%
50	33.33	0.667	33.3%

- Price = y/x ; every trade moves along the curve
- Larger trades $\rightarrow$ worse average price (slippage)
- LPs earn fees (e.g. 0.3%) but face **impermanent loss** when prices diverge
- Arbitrageurs keep pool price aligned with markets

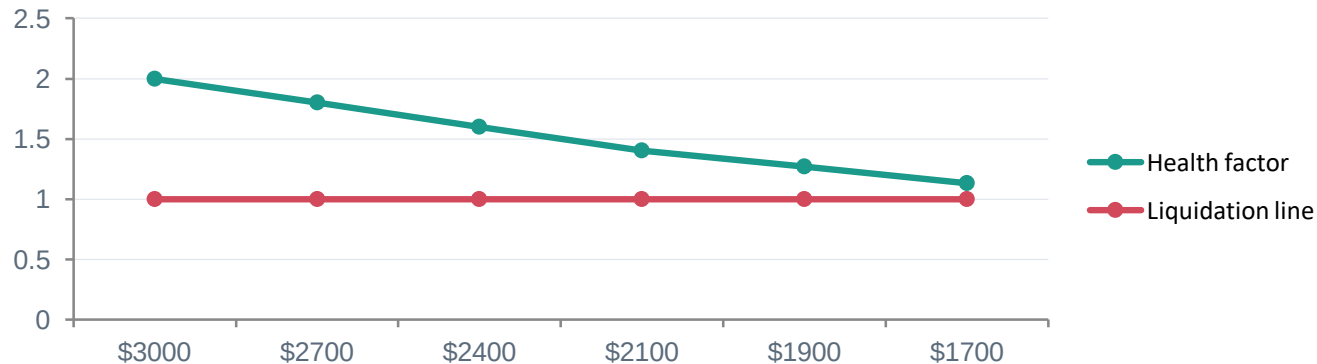
Stablecoins and over-collateralised lending

Two DeFi primitives most other protocols depend on

Stablecoin design	Peg mechanism	Key risk
Fiat-backed (USDC, USDT)	1:1 reserves held by an issuer; redemption	Issuer, custody & regulatory risk
Crypto-collateralised (DAI)	Over-collateralised vaults, liquidations	Collateral crashes, oracle failure
Algorithmic (e.g. TerraUSD)	Mint/burn arbitrage with a sister token	Death spiral — UST collapsed in May 2022

Health factor = (collateral value × liquidation threshold) ÷ debt

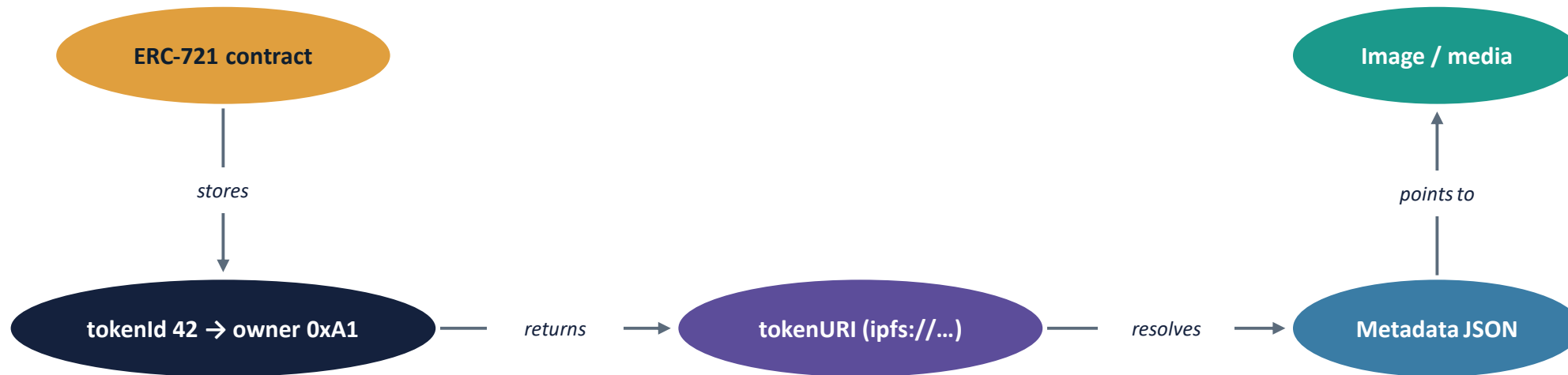
10 ETH collateral, 12,000 USD debt, threshold 80% — computed



- Below 1.0 anyone may **liquidate**: repay debt, seize collateral at a discount
- **Flash loans**: borrow any amount with zero collateral if repaid in the same tx — atomicity as a financial primitive

NFTs and digital ownership

What an NFT actually is — and what it is not



Is

A unique on-chain record of who controls a token ID, transferable and verifiable.

Is not

Copyright, or the image itself (usually stored off-chain). Licensing is a separate legal matter.

Durability

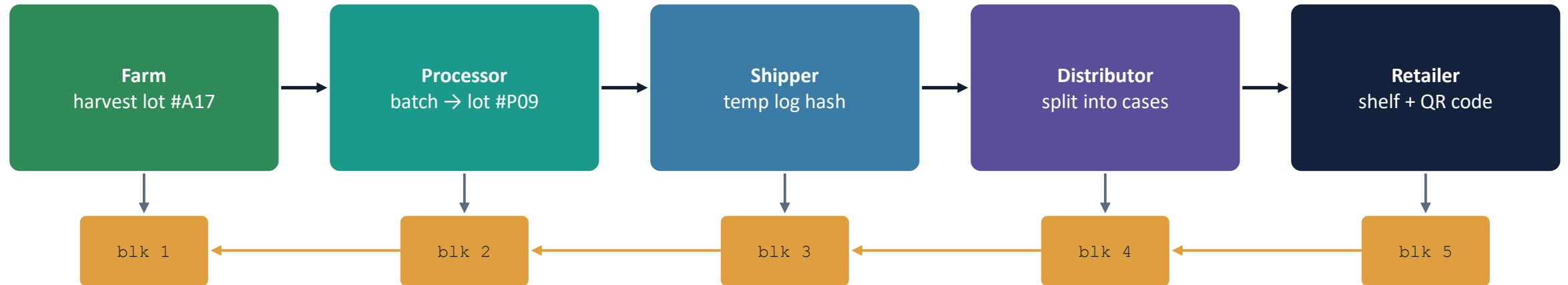
Use content-addressed storage (IPFS CIDs, Arweave) so the link commits to exact bytes.

Beyond art

Tickets, credentials, domain names (ENS), game items, real-estate deed pilots.

Supply-chain provenance

Shared, tamper-evident custody history across organisations that don't share a database



What the chain gives

Who asserted each custody event and when; no party can quietly rewrite it later.

What it cannot give

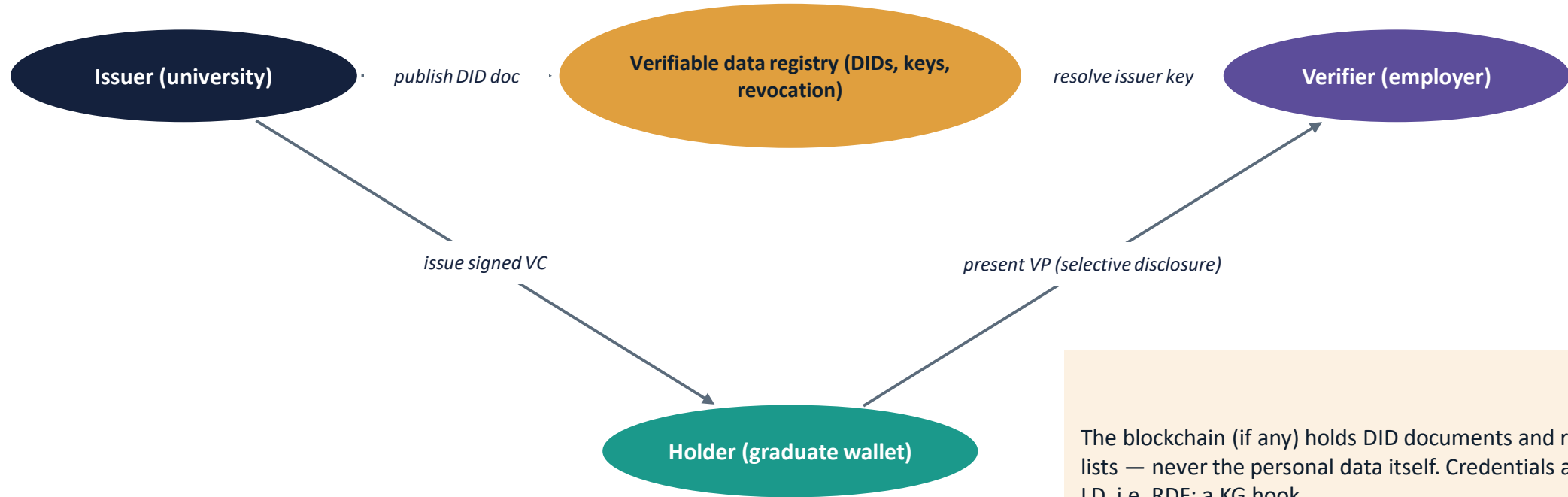
Truth at the physical boundary: a mislabelled crate is faithfully recorded as mislabelled (the 'oracle problem' again).

Complements

IoT sensors with signing keys, tamper-evident packaging, audits — and a knowledge graph to model products, lots and relations (Part III).

Decentralised identity: DIDs and Verifiable Credentials

W3C DID Core 1.0 (2022) and VC Data Model 2.0 (2025): the trust triangle

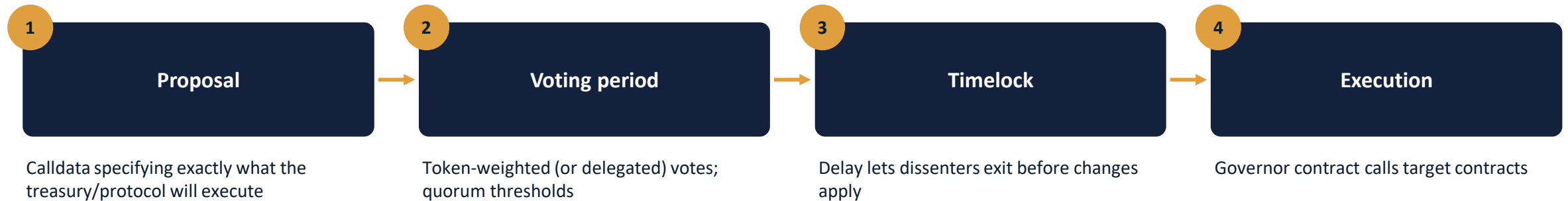


The blockchain (if any) holds DID documents and revocation lists — never the personal data itself. Credentials are JSON-LD, i.e. RDF: a KG hook.

```
did:example:123456789abcdefghi
└─ scheme : method : method-specific id
```

DAOs: governance encoded in contracts

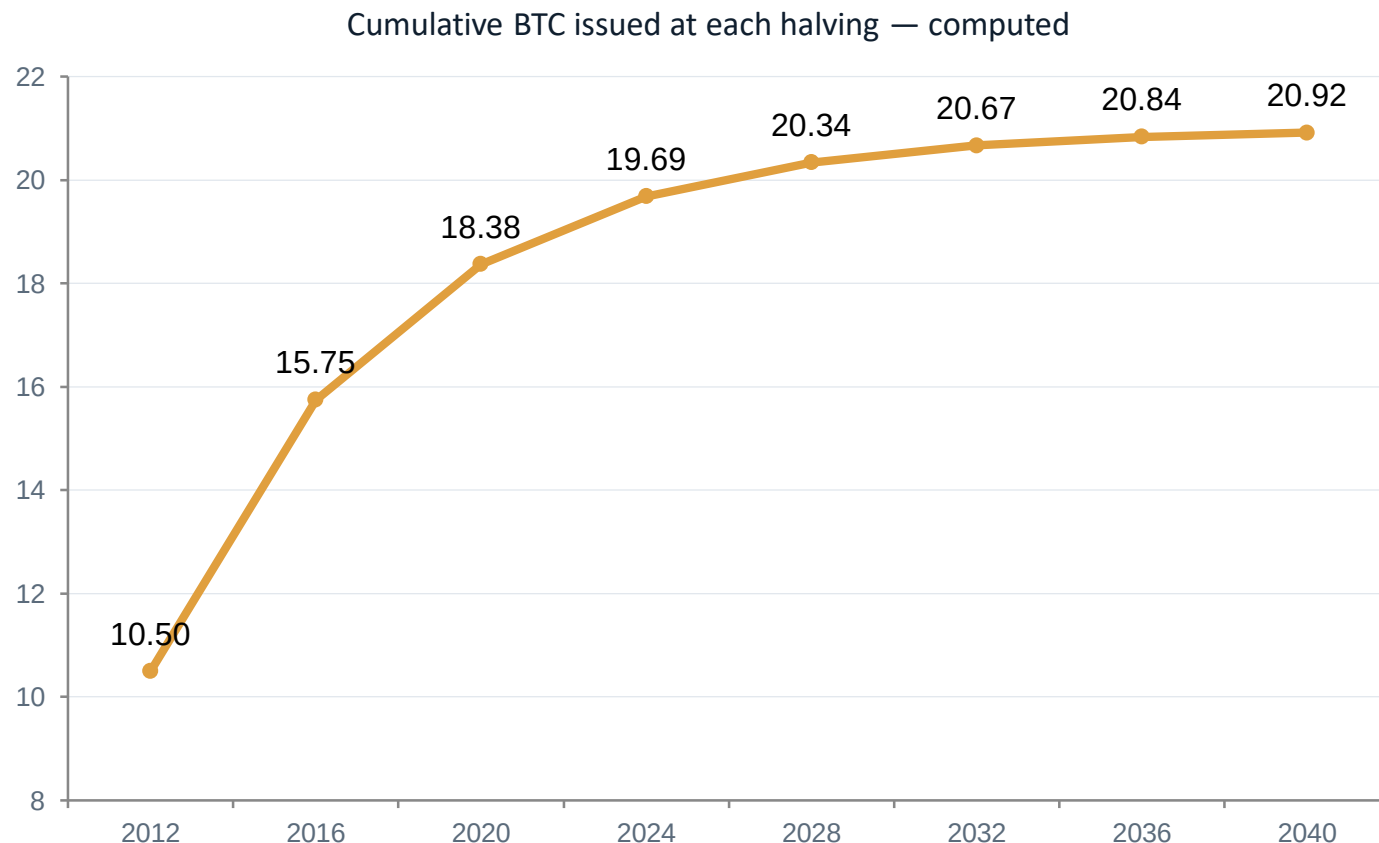
Decentralised autonomous organisations — proposal, vote, timelocked execution



Governance risk	Description	Mitigation
Plutocracy	1 token = 1 vote concentrates power	Delegation, quadratic or conviction voting
Low turnout	Small, active minority decides	Delegates, quorum floors
Governance attack	Borrow/buy tokens, pass malicious proposal	Vote snapshots at proposal block, timelocks, veto guardians

Tokenomics: Bitcoin's supply schedule

Fixed issuance rule: 210,000-block eras, subsidy halves each era, cap just under 21 million BTC



- $\Sigma = 210,000 \times 50 \times (1 + \frac{1}{2} + \frac{1}{4} + \dots) \rightarrow 21,000,000$ (minus satoshi rounding: actual cap $\approx 20,999,999.98$)
- **Halvings:** 2012, 2016, 2020, April 2024 (subsidy now 3.125 BTC)
- **Ethereum contrast:** no hard cap; issuance to validators minus EIP-1559 burn — supply can shrink
- **Design questions:** who gets initial tokens, vesting, inflation, sinks, incentive alignment

Tokenising real-world assets

Bonds, funds, deeds and invoices represented as on-chain tokens

Legal wrapper

SPV / trust holds the asset; token = legal claim

Compliance

Permissioned tokens check an allow-list (e.g. ERC-3643) before transfer

Oracle / servicing

NAV, coupon and custody data brought on-chain

Token & settlement

Atomic delivery-versus-payment with tokenised cash or stablecoins

Benefits

- 24/7 settlement, fewer reconciliations
- Fractional ownership
- Programmable corporate actions

Hard parts

- The chain does not enforce the off-chain legal claim
- Identity & transfer restrictions
- Interoperability with legacy rails

KEY INSIGHT

Tokenisation moves the RECORD on-chain; the ASSET's enforceability still lives in law. Model both explicitly.

Security, Privacy & Regulation

Threat models, privacy technology, the regulatory landscape and honest limitations

▸ Layered threat model ▸ MEV ▸ Privacy techniques ▸ Key management ▸ Regulation ▸ Limitations ▸ Part II review



A layered threat model

Attacks target every layer — most real losses happen far above consensus



Maximal Extractable Value (MEV)

Profit from choosing which transactions go into a block, and in what order

Sandwich attack, as ordered in one block:

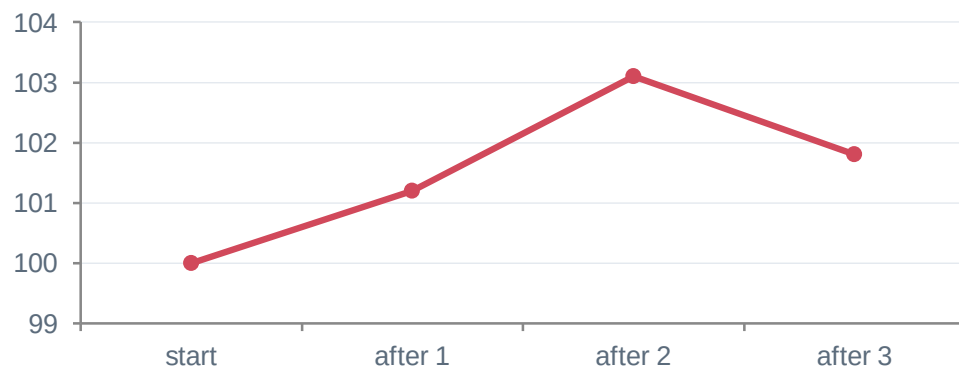
1. Attacker buy (front-run)

2. Victim buy (large, slippage 2%)

3. Attacker sell (back-run)

- **Benign MEV:** arbitrage aligning prices, liquidations keeping lenders solvent
- **Harmful MEV:** sandwiches, time-bandit reorg incentives
- **Proposer–builder separation** (MEV-Boost): specialised builders bid for block space; proposers pick the highest bid
- **Mitigations:** private order flow, batch auctions, encrypted mempools, slippage limits

Token price through the sandwich (illustrative)



Privacy: pseudonymity is not anonymity

Public ledgers are permanent, global, searchable — privacy must be engineered

Technique	Hides	Examples	Trade-off
Fresh addresses / HD wallets	Links between payments (partly)	Bitcoin best practice	Defeated by clustering heuristics
CoinJoin	Input–output links	Wasabi, JoinMarket	Coordination; regulatory scrutiny
Ring signatures + stealth addresses	Sender & receiver	Monero (RingCT)	Larger txs; decoy analysis
zk-SNARK shielded pools	Sender, receiver, amount	Zcash, privacy pools	Trusted setup (older), complexity
Confidential transactions	Amounts	Liquid, Monero	Range proofs add size
Permissioned channels	Data from non-members	Fabric channels	Trust in members

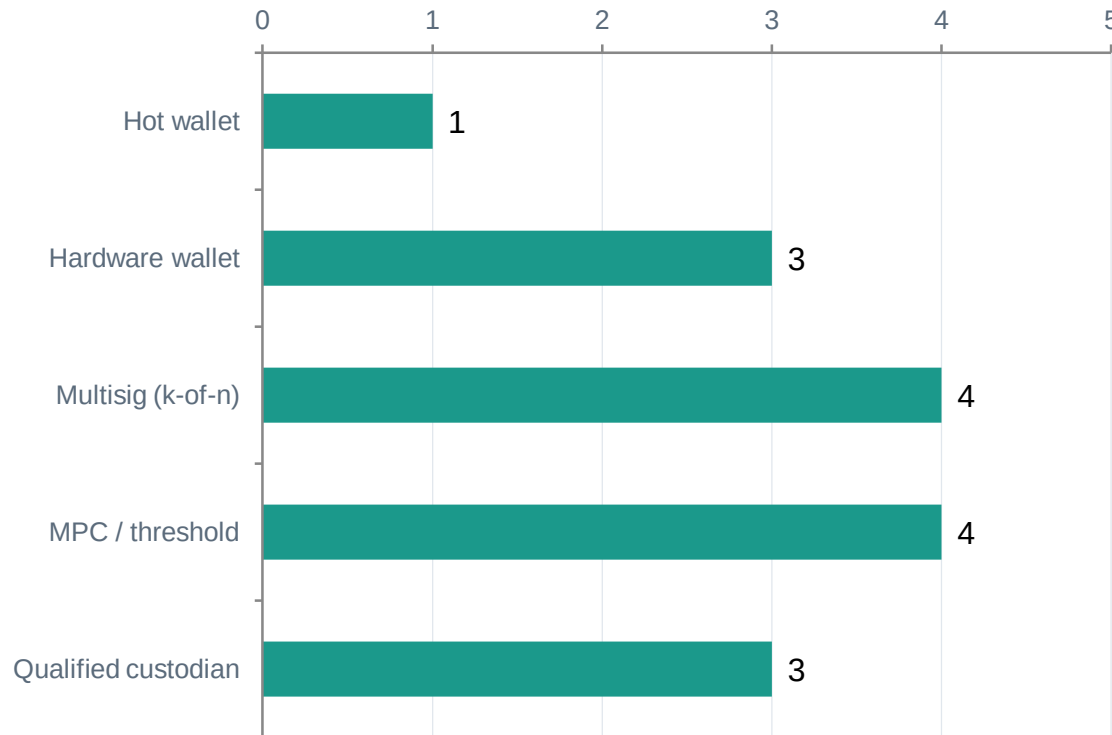
KEY INSIGHT

Blockchain data + GDPR: immutability conflicts with erasure rights — store personal data off-chain, anchor only salted hashes.

Key management: where security actually lives

Whoever controls the key controls the asset — there is no 'forgot password' link

Qualitative protection vs remote attack (author's scale)



Hot wallet — Convenient, online; exposed to malware

Hardware wallet — Key never leaves the secure element

Multisig (k-of-n) — No single point of failure; on-chain policy

MPC / threshold — Key shares; one signature; flexible policies

Qualified custodian — Institutional controls; counterparty risk

KEY INSIGHT

Quantum note: Shor's algorithm would break ECDSA/Schnorr; migration to post-quantum signatures (e.g. NIST's ML-DSA) is an active research and governance topic.

The regulatory landscape (as at mid-2026)

Fast-moving: always check primary sources before relying on any of this

EU European Union — MiCA

Markets in Crypto-Assets Regulation: stablecoin rules applied from 30 June 2024; full regime for crypto-asset service providers from 30 December 2024, with transitional periods.

UK United Kingdom

FCA registration for cryptoasset firms under money-laundering rules; financial-promotions regime since October 2023; a broader authorisation regime being legislated.

US United States

Spot bitcoin ETFs approved January 2024; federal stablecoin legislation (GENIUS Act) enacted 2025; market-structure rules still evolving.

G Global standards

FATF 'travel rule' for virtual-asset transfers; Basel Committee standards on banks' crypto exposures.

This slide is an overview, not legal advice. Engineers should design for auditability, identity hooks and data minimisation so products can adapt as rules change.

Honest limitations and criticisms

A rigorous practitioner can argue both sides

Critiques

- **Performance:** orders of magnitude slower than centralised databases
- **Energy** (PoW) and hardware waste
- **Oracle problem:** garbage in, immutable garbage
- **Usability:** irreversible errors, key loss
- **Speculation & fraud** dominate headlines
- **Centralisation creep:** pools, stakers, sequencers, front-ends

Responses & open work

- Use only where multi-party trust is the bottleneck
- PoS cut Ethereum's energy use dramatically
- Decentralised oracles, signed sensors, attestations
- Account abstraction, social recovery, passkeys
- Regulation, audits, transparency tooling
- Decentralised sequencing, DVT, client diversity

KEY INSIGHT

The right question is never 'blockchain or not?' in the abstract, but 'whose trust are we removing, and what does that cost?'

Part II summary: the blockchain stack in one picture

Each layer depends on the guarantees of the one beneath

Applications — DeFi · NFTs · identity · supply chain · DAOs

Smart contracts & L2 — EVM, gas, tokens, rollups, channels

Consensus — PoW / PoS / BFT: ordering + finality under attack

Transactions & state — UTXO or accounts; Merkle-committed

Data structure — Hash-linked blocks; Merkle trees

Cryptography — Hashes · signatures · ZK proofs

- **Tamper-evidence** comes from hashing
- **Authorisation** comes from signatures
- **Agreement** comes from consensus + incentives
- **Programmability** comes from deterministic VMs
- **Scale** comes from moving execution off L1 while keeping verification on it
- **Trust** is never eliminated — it is relocated. Always ask where.

Knowledge check — Part II

Q1

Why does altering a transaction in block 100 invalidate block 101?

Answer: It changes block 100's Merkle root and header hash, so block 101's prev-hash no longer matches.

Q2

Attacker with 30% hashrate, 6 confirmations: success probability?

Answer: $\approx 13.2\%$ (Nakamoto's formula) — six confirmations assume a much smaller attacker.

Q3

In PBFT with $n = 7$, how many Byzantine replicas are tolerated?

Answer: $f = 2$, since $n \geq 3f + 1$.

Q4

What single ordering change fixes the classic reentrancy bug?

Answer: Update state (effects) before making the external call (interaction).

Q5

Why must optimistic rollups publish their batch data on L1?

Answer: Without the data, no one can reconstruct state to build a fraud proof.

Q6

Selling 10 into an $x = y = 100$ pool yields how much?

Answer: $100 - 10000/110 \approx 9.09$ tokens ($\approx 9.1\%$ price impact).

PART III

Knowledge Graphs Meet Blockchains

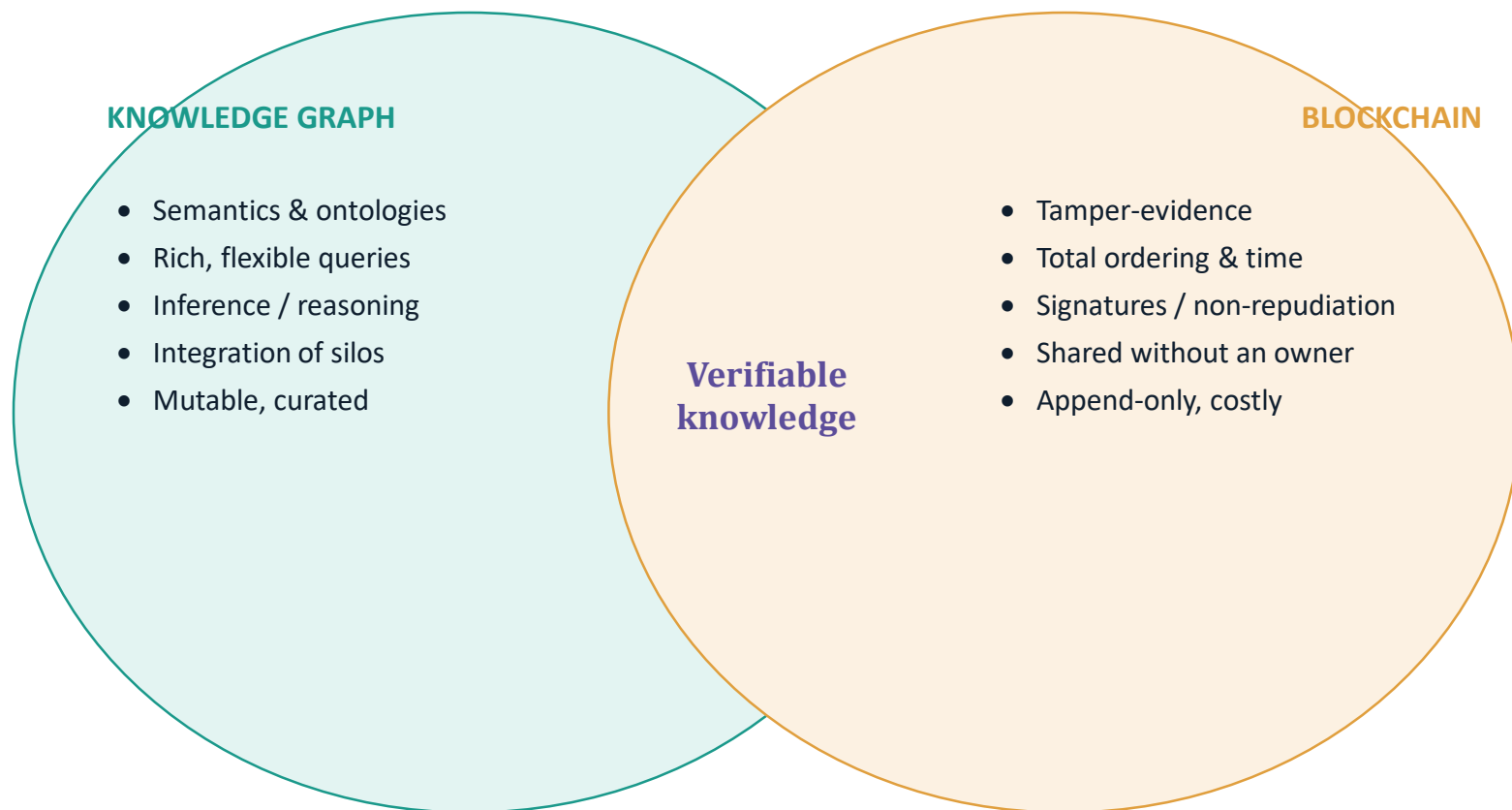
Meaning from graphs, integrity from ledgers — and how to engineer the combination

▸ Why combine ▸ Chain as a graph ▸ Ontologies ▸ ETL to RDF ▸ SPARQL lab ▸ Clustering ▸ PageRank ▸ Anchoring ▸ Triple proofs ▸ Credentials ▸ Reference architecture ▸ Case study ▸ Python lab



Why combine them? Two halves of trustworthy knowledge

A KG answers 'what does it mean?' — a ledger answers 'who said it, when, and has it changed?'



KG → chain

Blockchain data is a graph; model it semantically to query and analyse it.

Chain → KG

Anchor KG snapshots so any consumer can prove a fact existed unaltered.

Both → identity

DIDs and Verifiable Credentials are RDF graphs whose keys live on-chain.

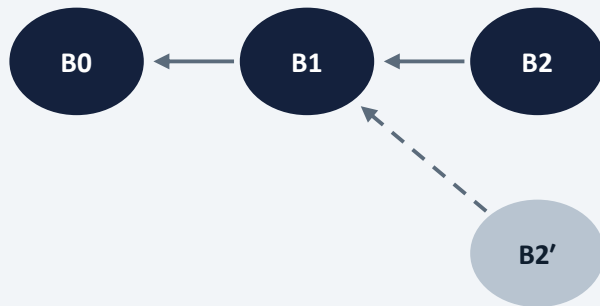
KEY INSIGHT

Put **meaning** in the graph and **integrity commitments** on the chain — never try to make one do the other's job.

A blockchain is already three graphs

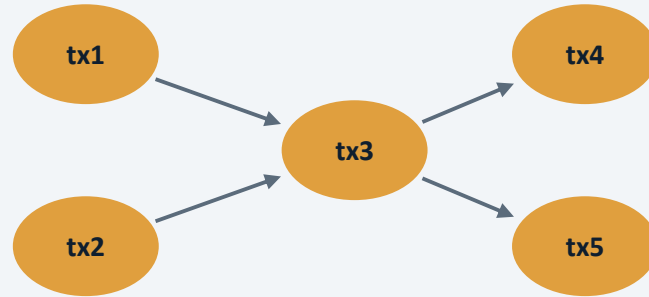
Every public ledger can be read as linked, typed data — the natural home of graph methods

① Block graph (a chain / tree)



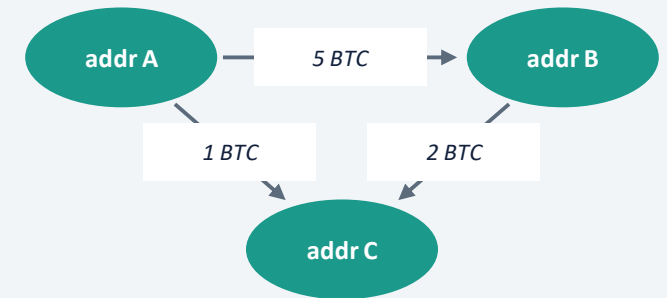
- Edges = `hasParent` (hash pointer)
- Forks create branches; consensus picks one
- Depth $\approx$ confirmations

② Transaction DAG (UTXO)



- Edge = output of one tx spent as input of another
- Acyclic by construction (can't spend the future)
- Supports taint & flow analysis

③ Address / entity graph



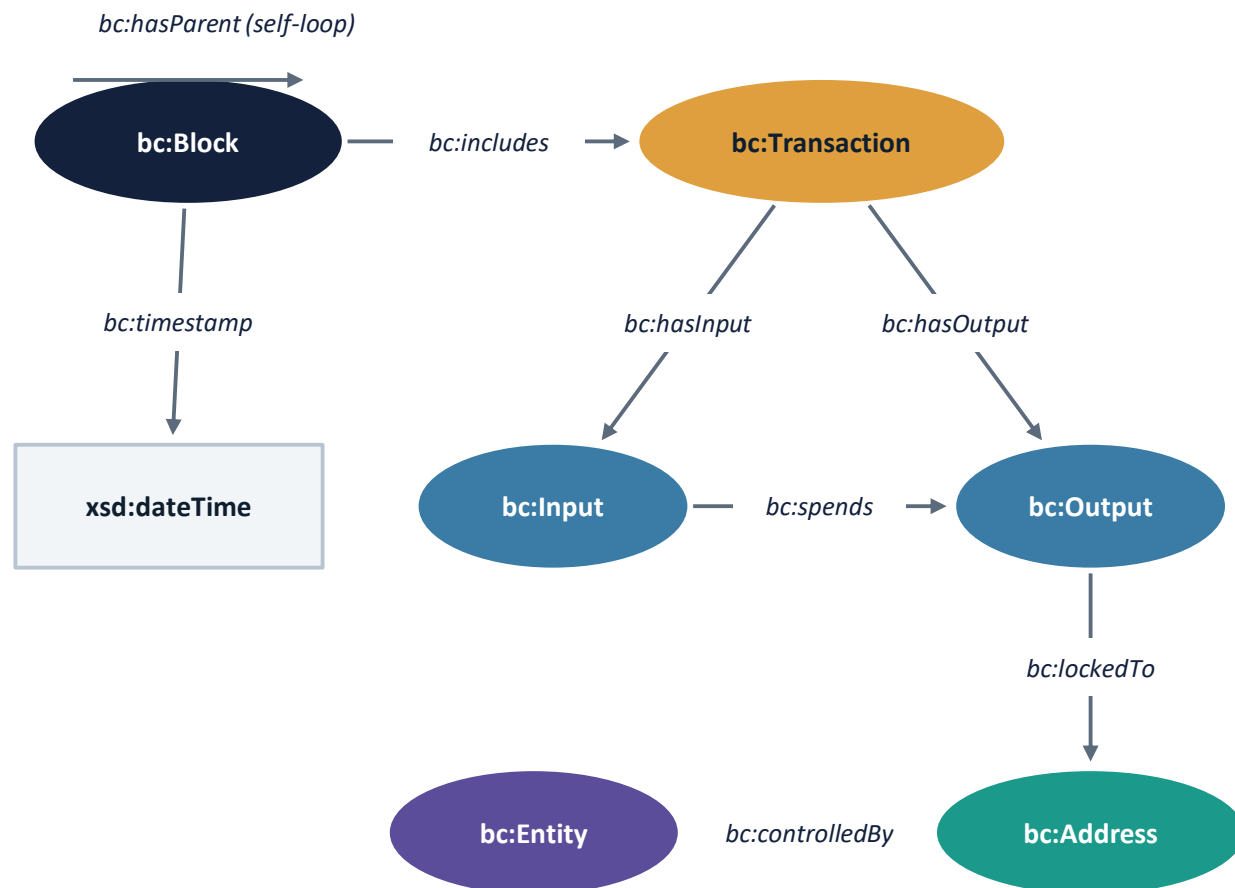
- Weighted, directed multigraph of value flows
- Clustering collapses addresses into entities
- Target of PageRank, communities, GNNs

KEY INSIGHT

Everything from Part I — RDF, SPARQL, graph algorithms, embeddings — applies directly to on-chain data.

Modelling a blockchain as a knowledge graph

A compact ontology (TBox) for UTXO and account chains — classes as nodes, properties as labelled edges



```

instance data (ABox) — Turtle

@prefix bc: <https://example.org/bc#> .
@prefix ex: <https://example.org/id/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

ex:b812 a bc:Block ;
  bc:height 812 ;
  bc:hasParent ex:b811 ;
  bc:includes ex:tx9f3a .

ex:tx9f3a a bc:Transaction ;
  bc:hasOutput ex:tx9f3a_0 .

ex:tx9f3a_0 a bc:Output ;
  bc:value "0.5"^^xsd:decimal ;
  bc:lockedTo ex:addr_bc1q7c .
  
```

- **Entity** is inferred, not recorded — clustering (next slides) materialises `bc:controlledBy`
- Keep on-chain facts and analyst **inferences** in separate named graphs for provenance

Existing blockchain ontologies and vocabularies

Reuse before you invent — these give shared IRIs for chain concepts

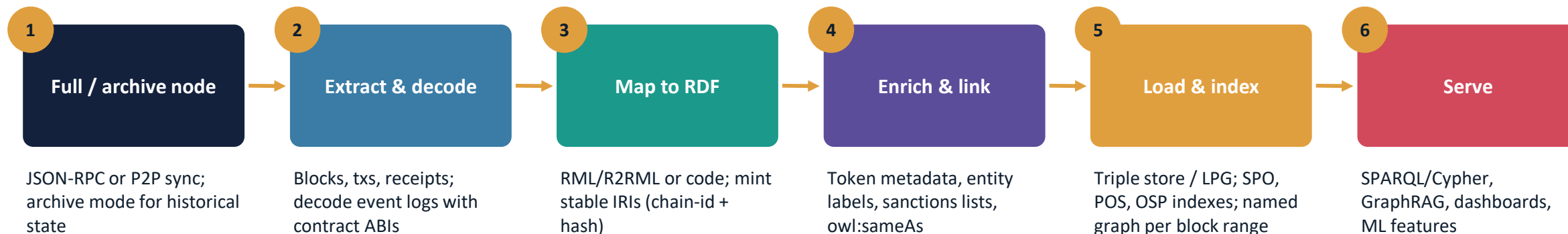
Vocabulary	Scope	Representative terms	Typical use
BLONDiE (Ugarte-Rojas, 2017)	Bitcoin, Ethereum, Hyperledger	Block, Transaction, Payload, Miner	Cross-chain semantic description of ledger structure
EthOn (ConsenSys)	Ethereum (OWL)	Account, Msg, Tx, ContractCall, gasUsed	Querying Ethereum state and messages as RDF
W3C PROV-O	Provenance of anything	Entity, Activity, Agent, wasGeneratedBy	Recording who/what produced a fact; pairs with anchoring
W3C DID Core / VC Data Model	Identity & credentials	DID document, verificationMethod, credentialSubject	Portable claims whose keys resolve via a registry or chain
schema.org / GS1 Web Vocabulary	Products & supply chain	Product, Offer, gtin, batch/lot	Describing goods tracked in provenance systems
FIBO (EDM Council)	Financial instruments	Security, Loan, Party, Obligation	Semantic view of tokenised assets and counterparties

KEY INSIGHT

Align your chain ontology to PROV-O: a block is a `prov:Activity`, a transaction's author a `prov:Agent`, its outputs `prov:Entity` — instant interoperability with provenance tooling.

From node to knowledge graph: the ETL pipeline

Turning raw blocks, receipts and logs into queryable, linked triples



Reorg-safety

Only finalise triples after N confirmations or finality; keep the tip in a separate, retractable named graph.

Scale

Ethereum has billions of transactions: use columnar staging, bulk loaders and partitioned graphs.

Identifiers

IRI = namespace + chain-id + hash, e.g.
`.../eth/1/tx/0x9f3a...` — globally unique and deterministic.

Idempotent loads keyed on content hashes make the pipeline safely re-runnable — a ledger's own trick, reused.

Lab: SPARQL over a blockchain graph

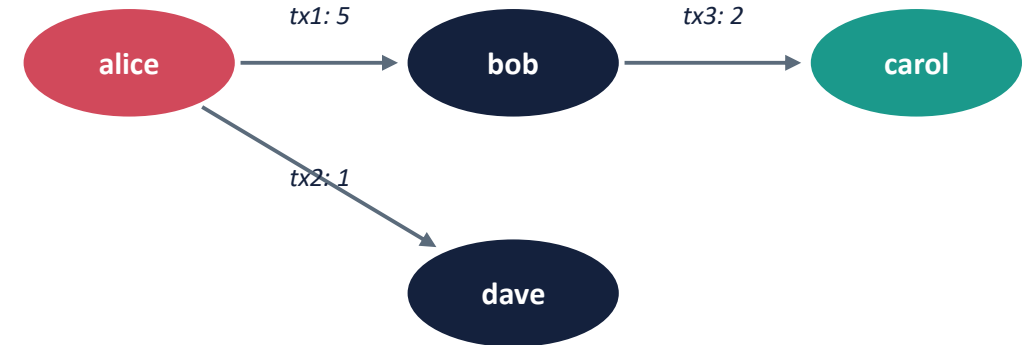
Two-hop value tracing — executed with *rdflib* on the 18-triple toy graph shown (output is real)

```
data.ttl (prefixes omitted)

ex:b1 a bc:Block ; bc:height 1 ; bc:hasParent ex:b0 ;
      bc:includes ex:tx1, ex:tx2 .
ex:b2 a bc:Block ; bc:height 2 ; bc:hasParent ex:b1 ;
      bc:includes ex:tx3 .
ex:tx1 bc:from ex:alice ; bc:to ex:bob ; bc:value 5 .
ex:tx2 bc:from ex:alice ; bc:to ex:dave ; bc:value 1 .
ex:tx3 bc:from ex:bob ; bc:to ex:carol ; bc:value 2 .
```

```
two_hop.rq — who received funds two hops from Alice?

PREFIX bc: <https://example.org/bc#>
SELECT ?who ?height (SUM(?v) AS ?total) WHERE {
  ?t1 bc:from ex:alice ; bc:to ?mid .
  ?t2 bc:from ?mid ; bc:to ?who ; bc:value ?v .
  ?blk bc:includes ?t2 ; bc:height ?height .
} GROUP BY ?who ?height
```



?t1 binds tx1 (alice→bob); ?t2 binds tx3 (bob→carol). tx2 dead-ends at dave.

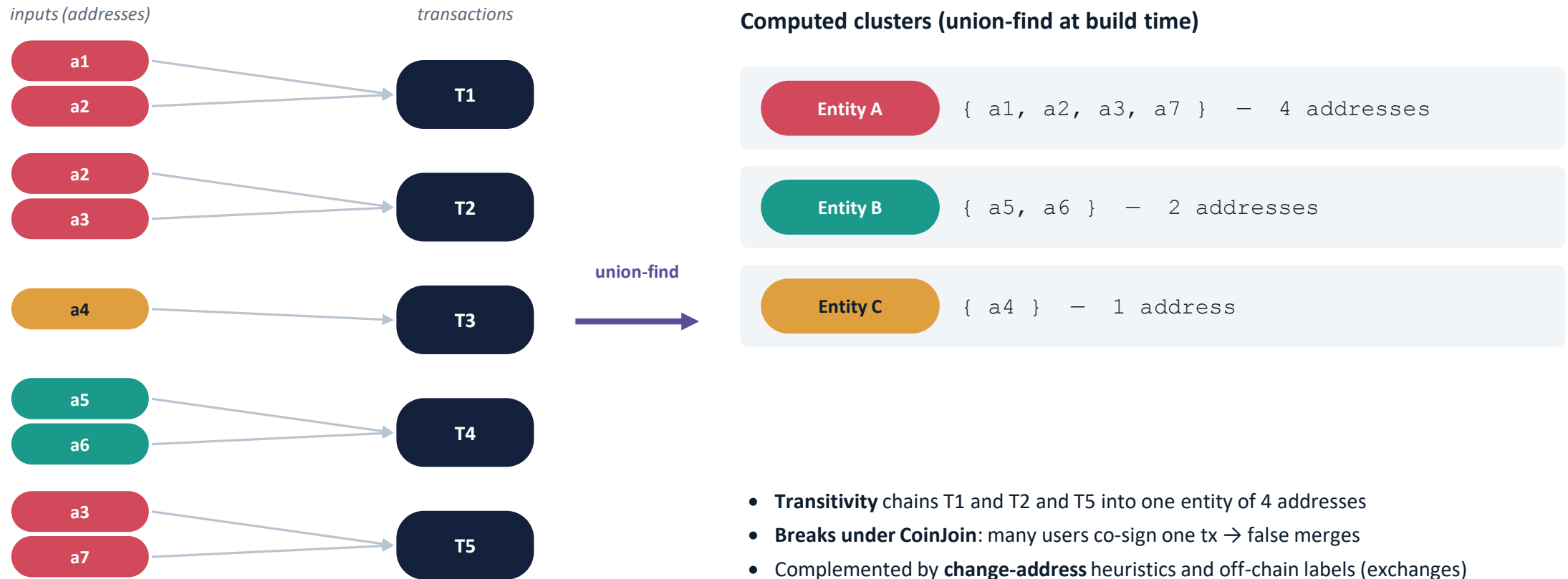
?who	?height	?total
carol	2	2

Same pattern scales to AML tracing; add property paths (*bc:to/^bc:from*)+ for arbitrary depth.

III

150 Address clustering: common-input ownership

If several addresses sign inputs of one transaction, assume one entity controls them (Meiklejohn et al., 2013)

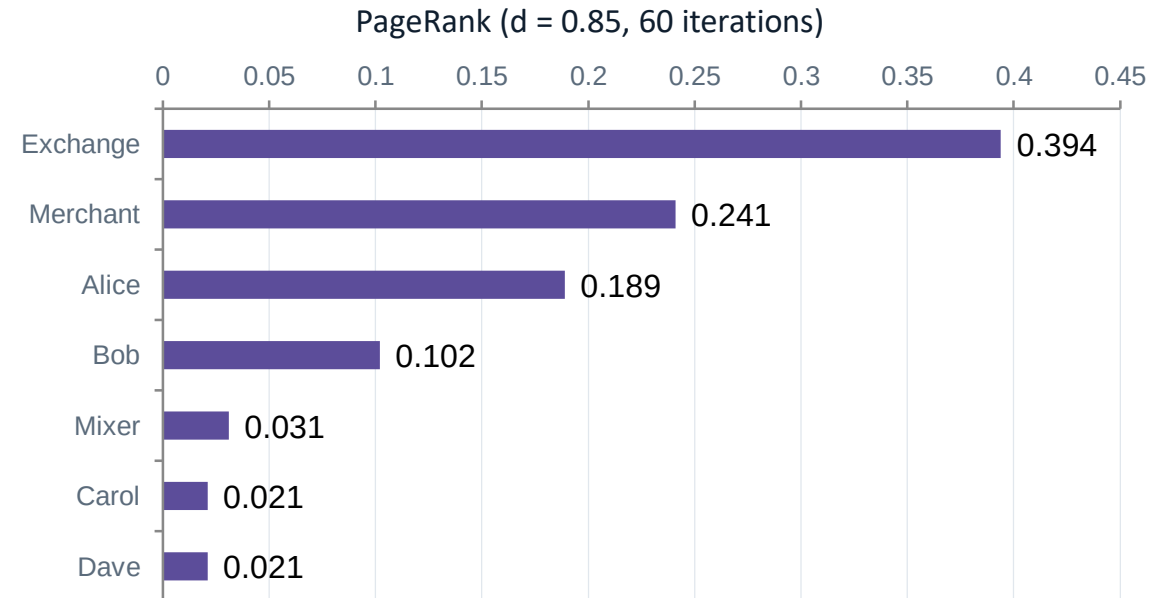
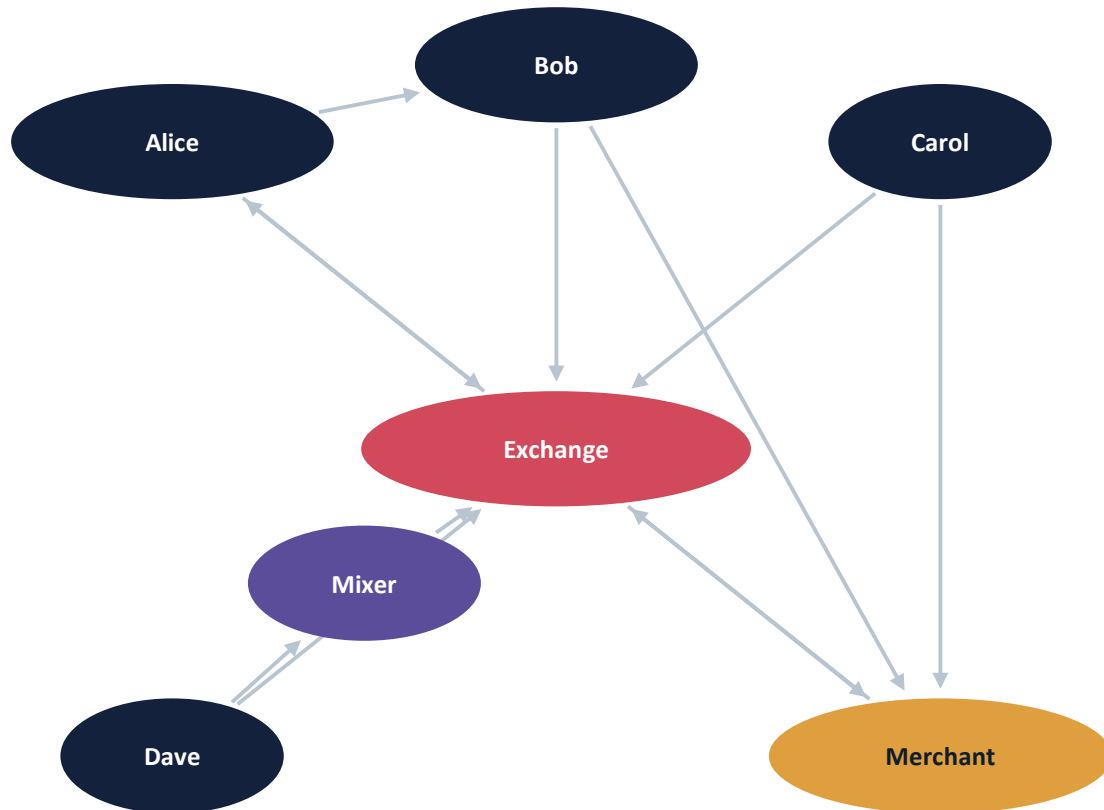


KEY INSIGHT

Clustering is KG entity resolution (Part I) applied to pseudonyms — with the same precision/recall trade-offs.

Graph algorithms on value flows: PageRank

Which addresses are structurally central? Power iteration computed at build time on a 7-node toy transfer graph



- Node width $\propto \sqrt{\text{PageRank}}$; the **Exchange** hub dominates because most flows pass through it
- Useful features for fraud/AML classifiers alongside degree, betweenness, motifs
- Weighting edges by value, or time-decaying them, changes rankings — state your choice

Anchoring a knowledge-graph snapshot on-chain

Commit to a whole named graph with one 32-byte hash; verify it later without trusting the publisher



```
canonical N-Quads (sorted, blank nodes relabelled)
```

```
<ex:batch42> <ex:maxTempC> "7.9" <ex:g1> .  
<ex:batch42> <ex:producedAt> <ex:plantA> <ex:g1> .  
<ex:batch42> <ex:shippedBy> <ex:acmeLogistics> <ex:g1> .
```

```
root = 065b22b62775c438fb1fcfc3399c340c  
       73d33b771ecdb0c17e7a58d9bc5629a0
```

```
7.9 → 4.9 ⇒ 2612916847569ba2bef7f0ec19597c74  
             db524baca17976595123423120fcdc5f
```

Both digests computed with SHA-256 at build time. One changed character → an unrelated digest (avalanche, Part II-B).

What anchoring proves — and doesn't

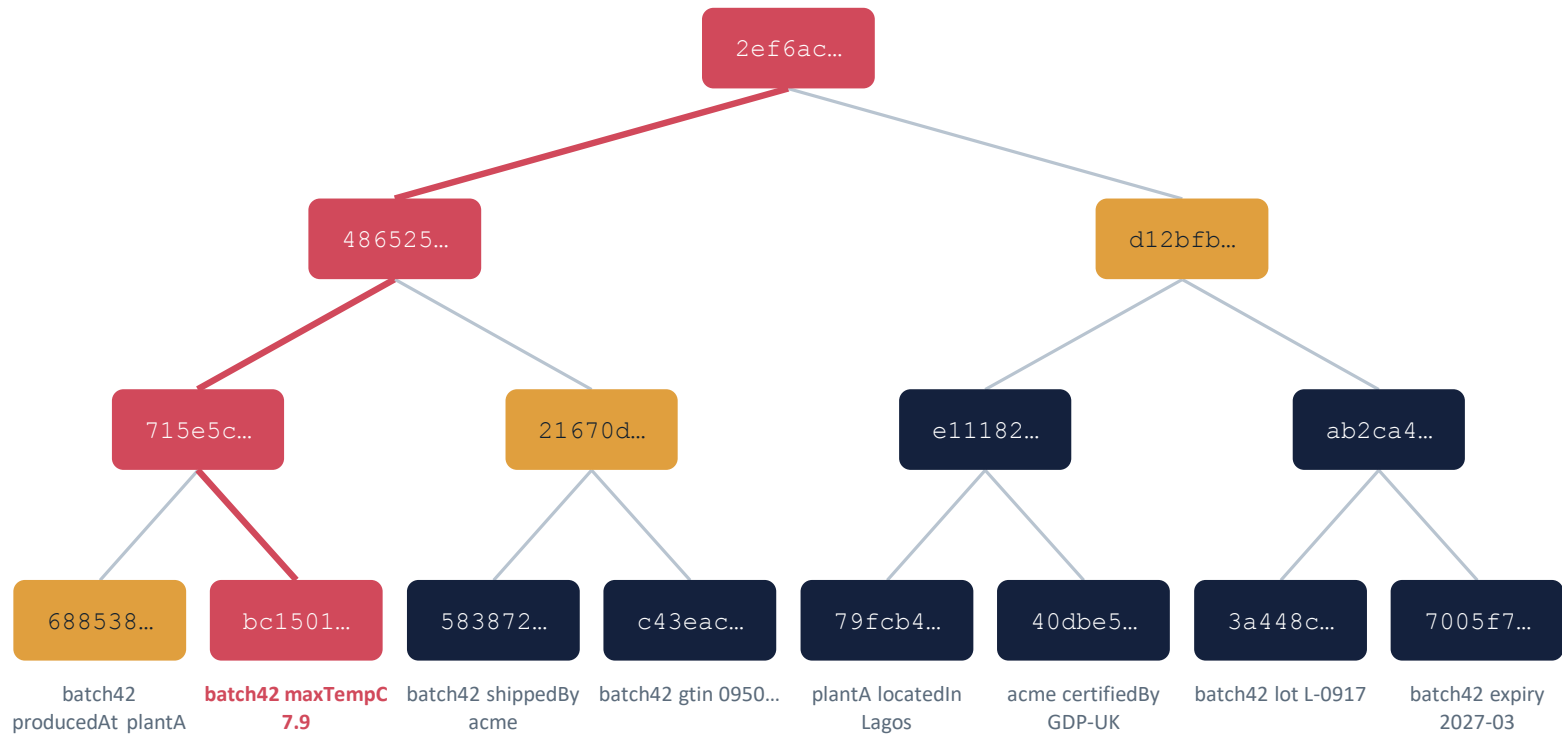
- **Proves** the exact graph existed no later than the anchoring block
- **Proves** who anchored it (tx signature)
- **Does not** prove the facts are true (oracle problem)
- **Requires** canonicalisation: same graph, different serialisation, must hash the same
- Uses the `ProvenanceRegistry` contract from Part II-F

KEY INSIGHT

Cost is $O(1)$ on-chain per snapshot regardless of graph size — the data stays off-chain, the commitment goes on-chain.

Proving a single triple: Merkle trees over a graph

Anchor the Merkle root of the triples; reveal one triple plus $\log_2(n)$ sibling hashes to prove membership



proven leaf/path

sibling (in proof)

- To prove **'batch42 maxTempC 7.9'** the prover sends the triple + **3** gold hashes
- Verifier recomputes the coral path and compares to the anchored root
- 8 triples → 3 hashes; 1 billion triples → 30 hashes (~1 KB)
- Other triples stay private — selective disclosure

Root (anchored) : 2ef6acffb5940b6d7c4a12d9593794bc788e2f240bf3db60c2d714b86d8d714a

All hashes are real SHA-256 values computed from the leaf strings shown.

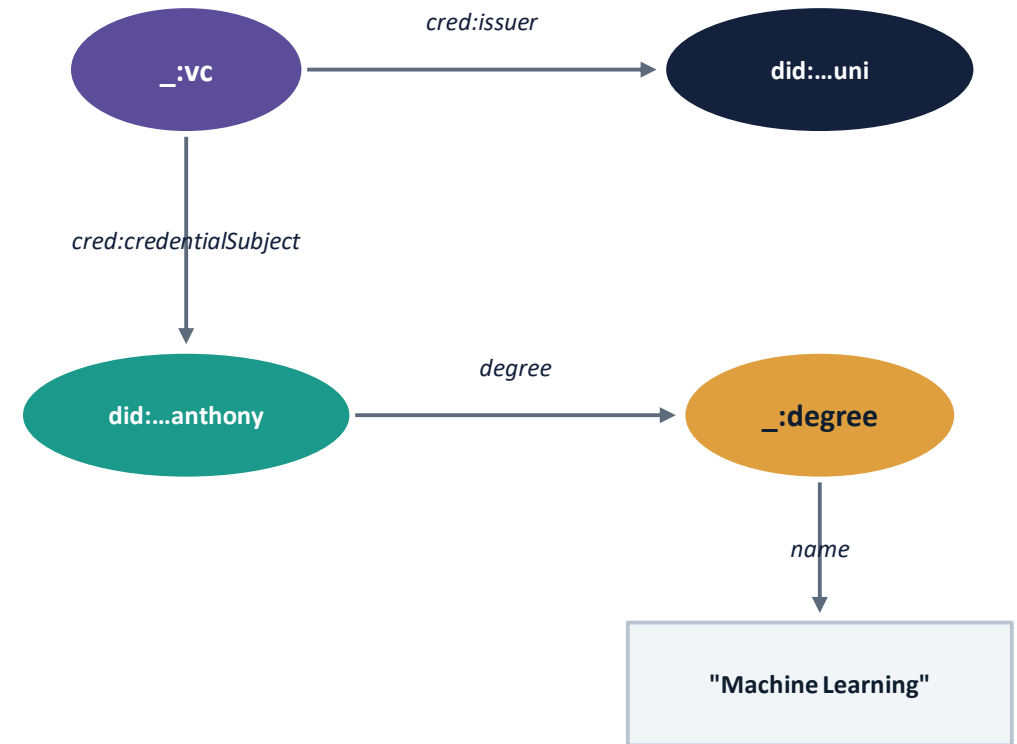
III 154 Verifiable Credentials are knowledge graphs

A VC is JSON-LD — i.e. RDF — signed by an issuer whose key is resolvable through a DID

credential.jsonld (illustrative)

```
{
  "@context": ["https://www.w3.org/ns/credentials/v2"],
  "type": ["VerifiableCredential", "DegreeCredential"],
  "issuer": "did:example:uni-manchester",
  "validFrom": "2026-07-01T00:00:00Z",
  "credentialSubject": {
    "id": "did:example:anthony",
    "degree": { "type": "MSc",
                 "name": "Machine Learning" }
  },
  "proof": { "type": "DataIntegrityProof",
             "cryptosuite": "ecdsa-rdfc-2019", ... }
}
```

Minimise on-chain data: never write the credential itself to a ledger.



- The **proof** signs the **canonicalised RDF** — the same RDFC-1.0 step as anchoring
- Verifier resolves the issuer's DID → public key (a registry or chain holds only keys & revocation lists)
- Credentials a holder collects form a **personal knowledge graph** they control

Reference architecture: an anchored KG

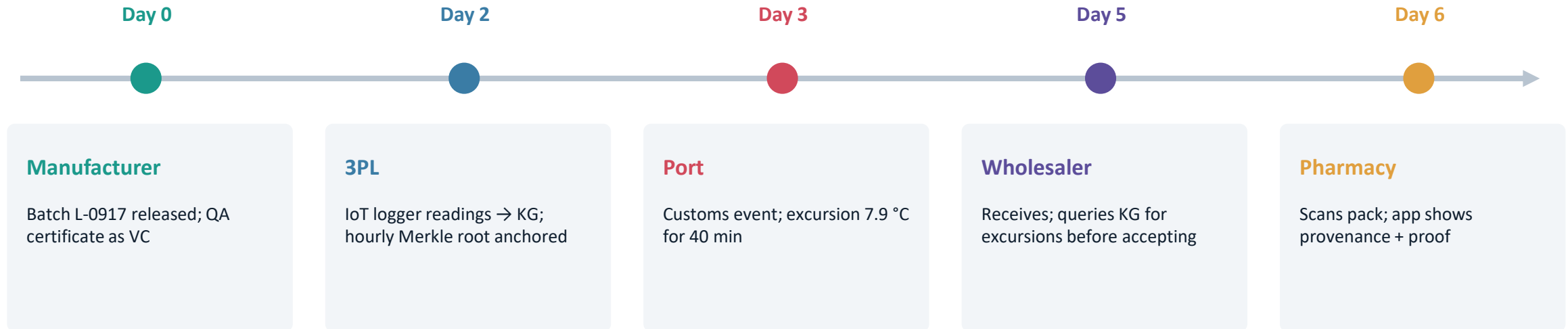
Off-chain meaning, on-chain commitments, identity in between



Design rule: the system must keep working if the ledger is slow or down — anchoring is asynchronous and batched.

Case study: a cold-chain pharma network

A composite scenario showing every layer working together



excursion check at the wholesaler

```
SELECT ?batch ?max ?anchorTx WHERE {  
  ?batch ex:lot "L-0917" ; ex:maxTempC ?max .  
  FILTER(?max > 8.0 || ?max < 2.0)  
  GRAPH ?g { ?batch ?p ?o }  
  ?g prov:wasGeneratedBy/ex:anchoredIn ?anchorTx .  
}
```

- 7.9 °C sits inside a 2–8 °C window → **accept**, and the answer is provable
- A regulator later verifies the same graph against the anchor tx — no need to trust the 3PL's database
- Recall scope = one SPARQL query over batch → shipment → pack edges

Illustrative composite; mirrors requirements of pharmaceutical track-and-trace regimes such as the US DSCSA and EU FMD.

Lab: a working blockchain in 30 lines of Python

Hash-linking, proof-of-work and validation — the whole of Part II·C–D in miniature

minichain.py

```
import hashlib, json
def h(o): return hashlib.sha256(json.dumps(o, sort_keys=True).encode()).hexdigest()

def mine(body, d=4):
    # proof-of-work
    n = 0
    while True:
        dig = h({**body, "nonce": n})
        if dig.startswith("0"*d): return n, dig
        n += 1

chain = []
def add_block(txs):
    prev = chain[-1]["hash"] if chain else "0"*64
    body = {"height": len(chain), "prev": prev, "txs": txs}
    nonce, dig = mine(body)
    chain.append({**body, "nonce": nonce, "hash": dig})

def verify(chain):
    for i, b in enumerate(chain):
        body = {k: b[k] for k in ("height", "prev", "txs", "nonce")}
        if h(body) != b["hash"] or not b["hash"].startswith("0000"):
            return f"block {i}: bad hash / PoW"
        if i and b["prev"] != chain[i-1]["hash"]:
            return f"block {i}: broken link"
    return "chain valid"
```

Canonical JSON (`sort_keys`) makes the hash deterministic — the same idea as RDF canonicalisation

Difficulty 4 hex zeros $\Rightarrow 16^4 = 65,536$ expected hashes per block

`prev` is the **hash pointer**: it binds each block to all history

Validation re-derives every hash and link — **anyone** can run it; no trust in the miner needed

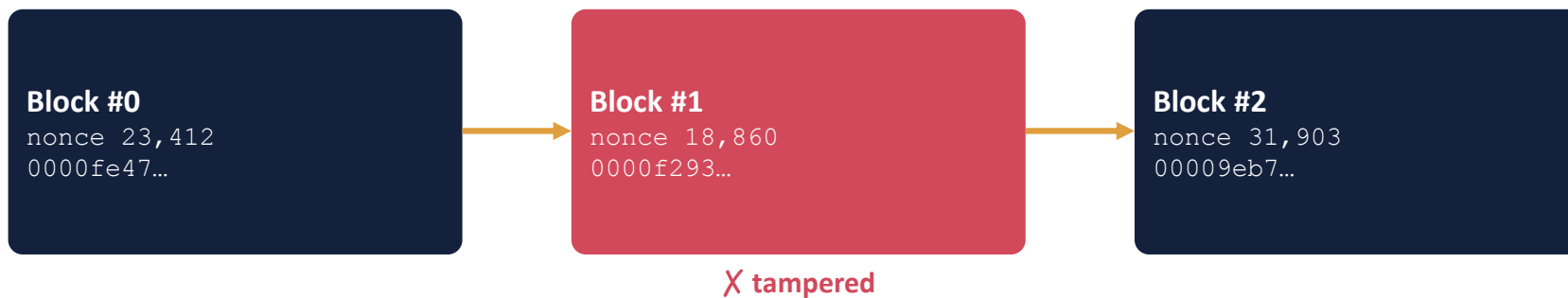
Next slide: real output from running this code.

III 158 Lab output: mining, validating and tampering

Actual console output from minichain.py (Python 3, difficulty 4)

```
console

>>> add_block(["genesis"]); add_block(["alice->bob:5"]); add_block(["bob->carol:2"])
#0 nonce= 23412 hash=0000fe47b7a9bb7c... prev=0000000000...
#1 nonce= 18860 hash=0000f293fd23025f... prev=0000fe47b7...
#2 nonce= 31903 hash=00009eb769697406... prev=0000f293fd...
>>> verify(chain)
'chain valid'
>>> chain[1]["txs"] = ["alice->bob:500"]      # tamper with history
>>> verify(chain)
'block 1: bad hash / PoW'
```



- Nonces 18,860–31,903 sit near the 65,536 expectation — PoW is a lottery
- Re-mining block 1 alone would still fail: block 2's **prev** no longer matches
- Exercise: add Merkle roots of **txs**, then a longest-chain fork-choice rule

Design patterns and anti-patterns

Hard-won lessons from KG + ledger projects

✓ Hash on-chain, data off-chain

Anchor roots; keep triples in the KG. Scales, preserves privacy and GDPR erasure.

✓ Separate facts from inferences

Chain-derived facts, analyst labels and model outputs in distinct named graphs with PROV-O.

✓ Canonicalise before hashing

RDFC-1.0 or deterministic JSON; otherwise equal graphs produce unequal hashes.

✗ Personal data on a ledger

Immutable + public = permanent exposure. Even hashes of low-entropy data are guessable — salt them.

✗ A blockchain as the database

Slow, expensive, poor at queries. Ledgers are for commitments and coordination, not analytics.

✗ 'On-chain means true'

The chain proves integrity of a record, not correctness of the world. Keep attestations and audits.

KEY INSIGHT

Rule of thumb: if you cannot say which party you no longer have to trust, the ledger is decoration.

Research frontiers

Open problems where knowledge graphs and blockchains are converging (as of 2026)

1 Verifiable query answering

ZK proofs that a SPARQL answer is complete and correct against an anchored graph.

2 Decentralised KGs

Graphs co-owned by many parties with on-chain integrity (e.g. OriginTrail DKG).

3 LLM agents + provenance

GraphRAG answers that cite anchored triples; agents that hold keys and sign actions.

4 GNNs for chain forensics

Temporal graph networks for fraud, phishing and laundering detection on transaction graphs.

5 Semantic smart contracts

Contracts whose conditions reference ontology terms and verified credentials.

6 Post-quantum integrity

Migrating signatures (ML-DSA, SLH-DSA) and preserving long-lived anchors' meaning.

Part III summary & knowledge check

Q1

Why anchor a Merkle root rather than the triples themselves?

Answer: $O(1)$ cost, privacy, erasure-compatibility; single triples still provable via $\log_2 n$ siblings.

Q2

Why must RDF be canonicalised before hashing?

Answer: Equivalent graphs can serialise differently (order, blank-node labels) $\rightarrow$ different hashes.

Q3

What does the common-input-ownership heuristic assume, and when does it fail?

Answer: Co-signed inputs share an owner; fails with CoinJoin / collaborative transactions.

Q4

Anchoring proves which properties of a fact?

Answer: Existence and integrity at a time, and who anchored it — not that the fact is true.

Q5

Where should a Verifiable Credential itself be stored?

Answer: With the holder (wallet); only DIDs/keys and revocation status may be on a ledger.

Q6

How many sibling hashes prove one of 1,024 triples?

Answer: $\log_2 1024 = 10$.

PART IV

Capstone, Glossary & Further Reading

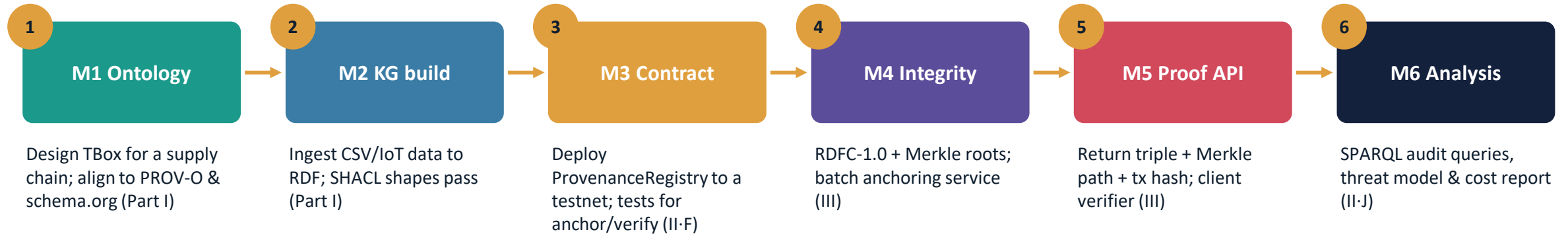
Consolidate everything by building a provable knowledge graph end to end

▸ Capstone brief ▸ Assessment rubric ▸ Glossary ▸ Final check ▸ Reading list



Capstone: a provenance KG with on-chain anchors

Six milestones; each one reuses a named part of this course



Deliverables

- Git repository with README and one-command setup
- Ontology (Turtle) + SHACL shapes + sample data
- Contract, tests and deployment address
- Demo: tamper with a triple → verification fails
- 4-page report: architecture, trust analysis, costs

Suggested stack

- Python: rdflib / pySHACL, or Apache Jena
- Foundry or Hardhat; Solidity ≥ 0.8
- A public testnet or a local devnet (Anvil)
- Optional: Neo4j + n10s for LPG views; GraphRAG demo
- Stretch: ZK proof of a query answer

Assessment rubric

How the capstone is judged — rigour is weighted above feature count

Criterion	Weight	Excellent (A)	Adequate (C)
Ontology & data quality	20%	Reuses standards; SHACL catches seeded errors	Ad-hoc terms; partial validation
Correctness of integrity layer	25%	Canonicalised, Merkle proofs verified by independent client	Hashes serialisation only; no per-triple proofs
Smart-contract quality	15%	Tests, events, access control, gas notes	Works on happy path only
Trust & threat analysis	20%	States who is trusted for what; oracle and key risks mitigated	Generic security list
Queries & insight	10%	Non-trivial SPARQL answering real audit questions	Simple lookups
Communication	10%	Clear diagrams, honest limitations, reproducible	Hard to reproduce



weight distribution (stacked to 100%)

Glossary (1/2): 51% attack to Mempool

Precise one-line definitions for every core term in the course

Term	Definition
51% attack	Majority of hashpower/stake rewrites recent history
ABox / TBox	Instance assertions vs schema/ontology axioms
Account model	Global state of balances per address (Ethereum)
Blank node	Anonymous RDF node, local to a graph
BFT	Consensus tolerating $f < n/3$ arbitrary faults
Canonicalisation	Unique serialisation so equal data hashes equally
DID	Decentralised identifier resolving to keys
Double spend	Spending the same funds twice
EIP-1559	Ethereum base-fee burn + tip fee market

Term	Definition
Embedding (KG)	Vector for entities/relations, e.g. TransE
Entailment	Triples that logically follow under RDFS/OWL
Finality	Point after which a block cannot be reverted
Gas	Unit metering EVM computation
Genesis block	First block; hard-coded in clients
Hash pointer	Reference + digest of the referenced data
IRI	Internationalised global identifier in RDF
Light client	Verifies headers & proofs, not all txs
Mempool	Node's pool of unconfirmed transactions

Glossary (2/2): Merkle proof to ZK proof

Precise one-line definitions for every core term in the course

Term	Definition
Merkle proof	Siblings on the path from leaf to root
MEV	Value from ordering transactions in a block
Nonce	PoW counter, or per-account tx counter
Open-world assumption	Missing $\neq$ false (RDF/OWL)
Oracle	Service bringing off-chain facts on-chain
OWL	Web Ontology Language for rich semantics
Proof of Stake	Validators bond capital; slashed if misbehave
Proof of Work	Leader election by hash-lottery
Rollup	L2 posting data + proofs/claims to L1

Term	Definition
SHACL	Constraint language validating RDF shapes
Slashing	Destroying stake for provable misbehaviour
Smart contract	Code at an address executed by all nodes
SPARQL	W3C query language for RDF
Triple	(subject, predicate, object) statement
UTXO	Unspent output; Bitcoin's state unit
VC	Signed, verifiable claim (W3C)
ZK proof	Proof revealing nothing beyond validity
zk-SNARK / STARK	Succinct ZK proofs; STARKs need no trusted setup

Final knowledge check: across the whole course

Q1

Express 'block 812 includes tx 9f3a' as an RDF triple.

Answer: `ex:b812 bc:includes ex:tx9f3a` .

Q2

Why can a light client trust a balance without downloading the chain?

Answer: Headers carry state/tx Merkle roots; a proof path links the leaf to a header it has verified.

Q3

Under the open-world assumption, what does a missing 'bc:controlledBy' triple mean?

Answer: Unknown — not that the address is uncontrolled.

Q4

Give one reason a consortium might prefer PBFT-style consensus to PoW.

Answer: Known members → immediate finality, high throughput, no energy race.

Q5

What single artefact links an off-chain KG to a blockchain in this course?

Answer: The anchored (Merkle) root of a canonicalised named graph.

Q6

State the trilemma's escape route used by rollups.

Answer: Execute off-chain, verify and publish data on-chain (L1).

Further reading: primary sources

Read the originals — most are short and remarkably clear

Reference	Why read it
Nakamoto, S. (2008). Bitcoin: A Peer-to-Peer Electronic Cash System.	The 9-page origin; attacker-success maths (Part II·C)
Haber, S. & Stornetta, W. S. (1991). How to time-stamp a digital document. J. Cryptology.	Hash-linked timestamping — the direct ancestor of block chaining
Lamport, L., Shostak, R. & Pease, M. (1982). The Byzantine Generals Problem. ACM TOPLAS.	Defines the fault model every consensus protocol answers
Castro, M. & Liskov, B. (1999). Practical Byzantine Fault Tolerance. OSDI.	PBFT, the basis of permissioned consensus
Buterin, V. (2013). Ethereum White Paper; Wood, G. Ethereum Yellow Paper.	Programmable blockchains; formal EVM specification
Eyal, I. & Sirer, E. G. (2014). Majority is not enough: Bitcoin mining is vulnerable. FC.	Selfish mining (Part II·D)
Narayanan, A. et al. (2016). Bitcoin and Cryptocurrency Technologies. Princeton UP.	The best rigorous textbook introduction
Antonopoulos, A. M. Mastering Bitcoin; Antonopoulos & Wood, Mastering Ethereum. O'Reilly.	Practitioner-level internals
Wüst, K. & Gervais, A. (2018). Do you need a blockchain? CVCBT.	The decision tree in Part II·H
Hogan, A. et al. (2021). Knowledge Graphs. ACM Computing Surveys 54(4).	The definitive KG survey — covers all of Part I
Bordes, A. et al. (2013). Translating embeddings for modeling multi-relational data. NeurIPS.	TransE (Part I)
Meiklejohn, S. et al. (2013). A fistful of bitcoins. IMC.	Address clustering (Part III)

Standards: W3C RDF 1.1, SPARQL 1.1, OWL 2, SHACL, RDF Dataset Canonicalization (RDFC-1.0, 2024), DID Core 1.0, VC Data Model 2.0.

The chain continues with you.

Meaning lives in the graph. Integrity lives on the chain. Trust lives in what you can verify.

Knowledge Graphs & Blockchain — Foundations to Advanced

